C4 and ADRs of the production backend of FinApp1.0.

Author: Thomas Costa

Date (dd/mm/yyyy): 01/10/2026

Main body text word count: 37,422.

Citation Style: ACM (Association for Computing Machinery)'s style.

Text Font: Times New Roman. Text Size: 11. Text Interline: 1.5.

Language: British English.

This work is protected by copyright and other intellectual property rights,

which are retained by the author.

Table of Contents

Introduction....1

1. C4 System Context....2

1.1. Diagram....2

1.2. ADR: monitoring....3

1.3. ADR: Stripe....3

1.4. ADR: email notification service....3

1.5. ADR: security systems and in-container/in-component security solutions....4

2. C4 Containers Context....8

2.1. FinApp1.0. System's Containers Diagram....8

2.2 FinApp1.0. System's container ADRs....9

2.2.1 ADR: data and rules storage solution....9

2.2.2. ADR: structuring as a modular monolith....11

2.2.3. ADR: usage of AI models....12

2.2.4. ADR: using a RESTful API....15

2.2.5. ADR: using FastAPI....15

2.2.6. ADR: where does Jinja2 run?....16

3. C4 Components Context....18

3.1. FinApp1.0 System - FastAPI Container's Components....18

3.1.1. Diagram....18

3.1.2. ADR: DDD bounded contexts as some of the app container's components....19

3.1.3. ADR: data caching....19

3.1.4. ADR: rendering and frontend solution....20

3.2. FinApp1.0 System - AI container's Components....22

3.2.1. Diagram....22

3.2.2. ADR: AI models choice....22

3.2.3. ADR: AI-FastAPI communications and merger?....24

3.2.4. ADR: AI models' AI API, engine and management....24

4. C4 Code Context....26

4.1. ADR: user requests' processing strategy 26
4.2. ADR: Clean Architecture and DDD division and annotation of the code 28
5. C4 Deployment Context: the US deployment case 32
5.1. Diagram 32
5.2. ADR: OS 33
5.3. ADR: using Python-Oracle DB driver in Thick Mode 33
5.4. ADR: user input format validation solution 34
5.5. ADR: endpoint caching? 35
5.6. ADR: system failure management 35
5.7. ADR: mechanism used to redirect HTTP requests to the standby FastAPI instance 37
5.8. ADR: hosting and isolation 38
Options 38
Decisions 39
Rationale 39
Solutions retained based on legal and industry requirements 39
Solutions retained based on installation/access requirements 40
Solutions retained based on backup plan and app's statelessness 40
Capacity consumption estimates by program container 41
Expected service demand 41
Duration of a usage cycle 42
Times of usage and impact on capacity calculation 42
HTTP requests per second per million inhabitants 42
Concurrent users, and new usage cycles per day per million inhabitants 43
Needed capacity of the DB, its backup space and a standby db 43
Disk type 44
Optimisation 44
Nature of queries, number of writes and number of 'user write' commits 45
Peak datafile reads 47
Peak data file random writes (physical writes) 48
Peak redo write size 49
Online redo log files, their size, and redo archive space size 50
DB Physical disk storage space size 51

DB Peak Storage bandwidth 55
DB Peak memory 55
DB CPU 56
Needed capacity for the AI container 58
Parameters disk storage needs 60
per-inference-engine-instance Disk storage needs 60
AI container's context storage needs 60
Per-model-replica parameters' VRAM 61
Task 1 context VRAM, and CPU and GPU usage 61
Task 2 context VRAM, and CPU and GPU usage 63
Number of machines needed and final capacity count 64
Needed capacity of the FastAPI container 64
Number of FastAPI instances per deployment 65
Disk storage 65
FastAPI container's CPU 65
FastAPI container's RAM 66
Solutions retained based on capacity requirements 67
Solutions retained based on the level of control I need 67
Solutions retained based on processing speed and performance 68
Solutions retained based on the level of isolation I want, and containerisation 70
Solutions retained based on price 71
Prices of capacity aside from membership fees 71
Membership fees per deployment option: 76
Consequences 78
5.9. ADR: cloud provider 80
5.10. ADR: by-FastAPI/in-FastAPI TLS decryptor 81
6. Codebase specifications 82
6.1. CA elements' classification by code form 82
6.2. Design patterns and SOLID Principles 85
6.3. UV as dependency manager 87
6.4. DB queries code 87
Bibliography 89

Appendix 1: probabilistic tree of write and read loops happening during 1 loop of the main use case 130

Appendix 2: samples of the AWR Report of a DB of my Oracle contact: 131

Appendix 3: US revenue, profit, and pricing model.....135

Glossary

Acronym	Term	Definition
ADR	Architectural Decision Record	Structured written trace of a decision taken regarding the architecture of software.
ACL	Anti-corruption layer	Translation layer between a bounded context and an external service, ensuring both can communicate even when their language/terms vary [188].
	AI container	The entire group of components deployed to run specific tasks with AI. This fleet includes all the model replicas, the inference engines, the model repositories.
	App context	The pool of data of an app.
	Bounded Context	A Domain-Driven Design concept designating a specific subtask within the general task that a computer program achieves.
CA	Clean Architecture	An architectural guideline commonly used in software development.
	Cohesion	Cohesion is often cited with adjectives which describe how it is achieved specifically, but it basically always means the same thing: “it is the degree to which all elements directed towards performing a single task are contained in the same component” [140].
CORS	Cross-Origin Resource Sharing	“CORS determines which external domains are allowed to access your API. By restricting this to only your trusted frontend application, you prevent other websites from making requests” [231].
	Coupling	Coupling knows specific declinations and has one unified meaning: it is the action of making multiple elements work co-dependently, or one depending on the other two elements which cannot work without one

Acronym	Term	Definition
		another are coupled [140].
CRSF	Cross-Site Request Forgery	“A malicious website tricks an authenticated user into unknowingly submitting a request to your application. This is a classic attack vector known as Cross-Site Request Forgery (CSRF). If your authentication relies solely on browser cookies, your app could be vulnerable to actions like changing a user's email or submitting data without their consent” [231].
CSR	Client-side rendering	the client generates the HTML content. Opposed to server-side rendering [224].
	Customer	A customer is the legal person receiving my service, via one or multiple users, against payment.
DDD	Domain-driven design	A framework used to conceptualise the core code layer elements of a computer program [220].
DI	Dependency injection	“a programming technique that makes a class independent of its dependencies. It achieves that by decoupling the usage of an object from its creation. This helps follow SOLID’s dependency inversion and single responsibility principles” [187].
DIP	Dependency Inversion Principle	“A software design approach that prioritises the business domain at the core of development, coined by Eric Evans” [380].
	Head user	A customer is one or multiple users, with one of them acting as the ‘head user’, meaning he/she sets the other users under that customer id and the permissions they have.
JWT	JSON Web Token	“JSON Web Token (JWT) is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object” [40].
	Model replica	Sometimes, a single model cannot quantitatively sustain the entire workload the program needs. To address that issue, one can run multiple instances of that model. Each instance is called a replica.
OLTP	Online transaction	A type of data management system optimised for programs which update a lot of data and/or insert a lot of new data constantly [12, 344].

Acronym	Term	Definition
	processing	
	Read loop	Read loop: e.g., “for each …, read…”, but sometimes there is no “for each”, just “read..”, usually at the beginning of the use case loop.
	Process	At the program scale, the process (or ‘OS process’) is the OS’s execution of the program [204]. In FastAPI, a process is the unit of work a Uvicorn/Gunicorn worker executes [216]. The same logic applies to other contexts: in an AI system, a ‘process’ is short for ‘AI process’ and is what an AI engine executes, etc.
	Rule	A rationale a program applies upon a context (not the application itself, just the rationale). For example, “if cacti, water once per month” is a rule for watering plants. That rule can be stored as code or as a relationship in a table.
	Serving/in serving mode	Designates an element of a program when it is actively in charge of delivering the service to the user as the single or co-deliverer (e.g. a DB co-delivers a service with the FastAPI app and other services). This is opposed to a dormant, or in-dormant mode element, which can be part of the architecture, can do some work of its own (e.g. a DB mirror instance which also does some writes) when dormant or no active work, but which does not directly provide the service to the users (in this example, the primary instance of the DB would be serving, while the mirror copy would be dormant).
SSR	Server-side rendering	Content is rendered before being sent to the browser.
SSO	Single sign-on	“Single Sign-On (SSO) streamlines user authentication by allowing access to multiple applications through one set of credentials” [412].
	Target	a table’s column in which a certain user answer must be stored
	Sub-workflow	In the context of my program, this designates the part of the app’s workflow that a bounded context runs.
TLS	Transport Layer security	an encryption security used so that signals going over a network are not readable.

Acronym	Term	Definition
	Sub-workflow	the next smaller logical unit of a workflow. A sub-workflow is never just a given rule, because, as evoked in the rule definition, the latter is a rationale, while a sub-workflow is a step or multiple execution steps. Therefore, for example, a (sub-)workflow to water plants can execute watering based on watering rules such as “if cacti, water once per month”, but not the other way around.
	Usage cycle	A usage cycle is a cycle of customer activity during which that customer (via its unique or multiple users) uses all the features of the app that he/she must use to receive the service 1x in its entirety. The usage cycle contains the completion of 1 user project, users working on this user project logging in/out, and setting user rights for working/consulting this user project.
	Use case	This is a Clean Architecture concept, designating workflow-orchestration as-code rule specific to the application [222].
	Use case loop	Use cases can contain code loops, that id dub use case loop.
	User	An individual who interacts with my program’s website to receive the online service it offers. The ‘head user’, i.e. , the individual who manages the customer profile and rights of users who are part of the customer, is also a user.
	User project	A logical Metadata container for contexts and inference runs applied upon users’ interactions to provide financial case information. The completion of a user project is one part, and the main part of the usage cycle.
	User session	Authenticated (logged-in) user activity. Visitors who only view the homepage or signup page but do not access any personal space are not part of a user session.
VLM	Vision-language Model	Type of Language Model.
XSS	Cross-Site	A form of attack where “an attacker injects malicious scripts into your

Acronym	Term	Definition
	Scripting	application through user input” [231].

Introduction

In this paper, I discuss the production-stage design of FinApp1.0's backend and explain my design choices. FinApp1.0 is a paid financial web service applicable to financial cases worldwide. I use Python and SQL as backend languages.

I organise the paper as follows. First, I present the architecture using a C4 template, as C4 helps best picture the roles and relationships of a software's components at each granularity level [53]. I dedicate a chapter to each C4 context layer (Sections 1 to 4) and to the C4 deployment detail (Section 5). Each section contains the related architecture diagram and Architectural Decisions Records (ADRs) [64]. I then present noteworthy non-architectural domain code details (Section 6).

Before diving in, the reader must note the following.

First, I have completed the sections of this paper iteratively until all was coherent and accounted for. Thus, there is no strict order of reading; one can jump directly to any section of interest.

Secondly, in Sections 4 and 6, I focus on general decisions I took for the code and domain, as I want to keep the domain and code details private.

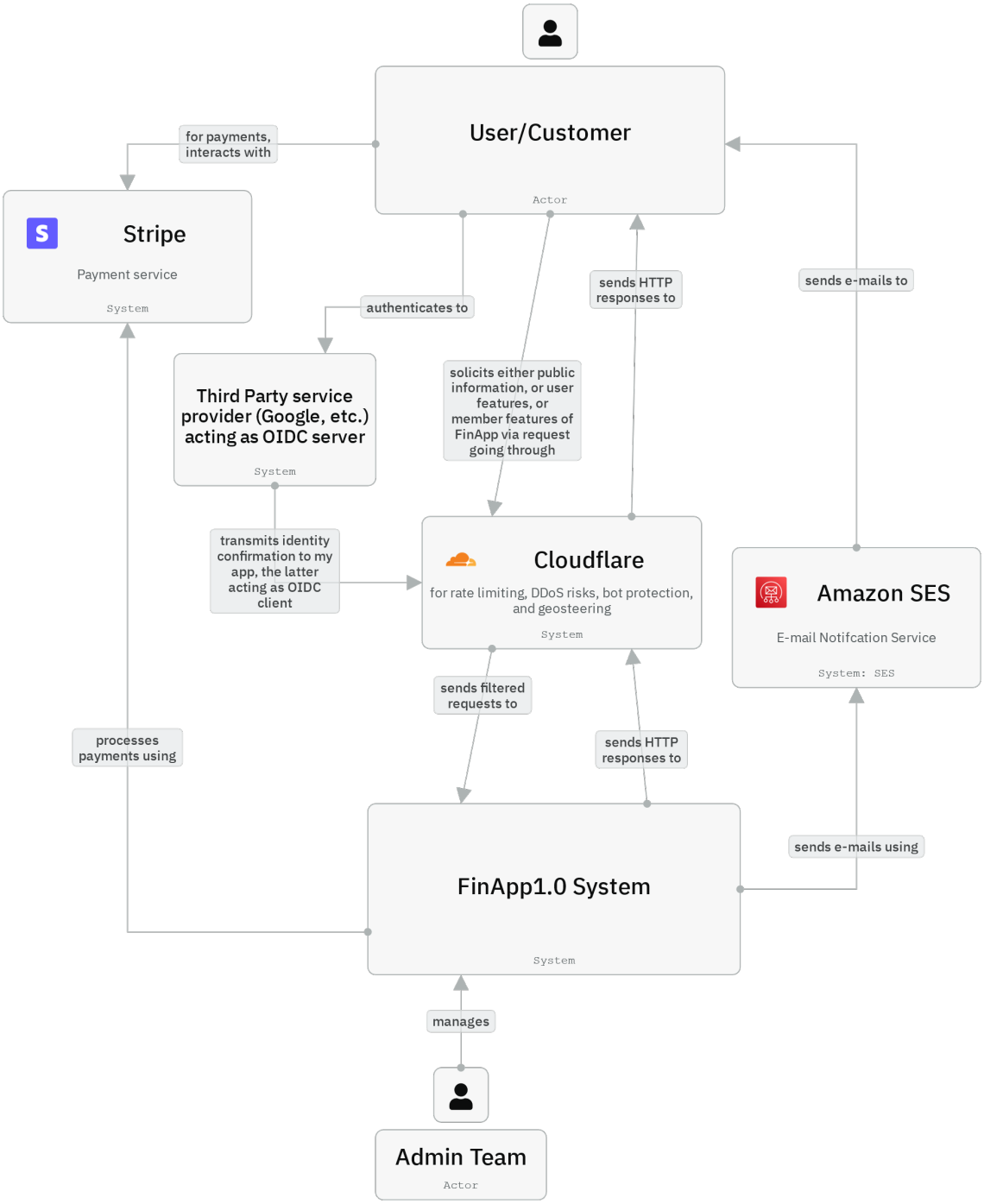
Thirdly, for reasons I discuss later, I must deploy once in each country with a clientele. Each local deployment influences architectural choices beyond the deployment strategy. For that reason, I here focus on a single deployment. I choose to focus here on the US deployment because I have found sufficient market data to forecast US demand, and because it would be one of the service's main markets.

Fourthly, I am currently improving the code and domain. Thus, I don't know the eventual capacity requirements of most processes. However, these requirements partly determine the ideal deployment solution. I still want to train in choosing the right deployment. Therefore, when I lack an exact metric to calculate capacity, I use an arbitrary value or derive it from third-party projects.

Lastly, this paper is strictly about the backend of FinApp1.0, but when relevant to backend choices, I also discuss the frontend.

1. C4 System Context

1.1. Diagram[1]



[1] I drew diagrams with Ice Panel, besides the deployment diagram which I drew using Diagrams.net.

1.2. ADR: monitoring

Date (dd/mm/yyyy): 20/07/2026

Status: Accepted.

Context: To be alerted of performance issues and system/component failures, I need monitoring.

Options: External or self-coded monitors.

Decisions and Rationale:

Oracle has its integrated performance monitor called AWR, and internal alert systems. So I do not need a specific supplementary monitor for my Oracle DB.

To measure the CPU performance of other components, I will use tools such as UserBenchmark, GeekBench, or CineBench [139, 381].

Finally, the chosen load-redirecting technology checks website availability, thus not requiring any additional monitors (see Section 5.7).

1.3. ADR: Stripe

Date (dd/mm/yyyy): 25/05/26

Status: Accepted.

Context: The program needs a service fee payment processing system.

Options: I can craft the system or use a service provider's.

Decision: I choose Stripe.

Rationale: It is a common robust solution that does not require installation, since it is an external service.

Consequences: The program must call/redirect to this service to process payments.

1.4. ADR: email notification service

Date (dd/mm/yyyy): 25/08/26.

Status: Accepted.

Context: My program needs a tool or component to send emails to customers/users for personal info, password setting/resetting, confirmations, exceptional security warnings, company newsletters, etc.

Options: I can craft that tool or use a service provider's.

Decision: I choose Amazon SES.

Rationale: It is a common secure system ideal for apps sending more or less sensitive e-mails, which is my case [13].

Consequences: The FinApp1.0 container sends email requests to it.

1.5. ADR: security systems and in-container/in-component security solutions

Date (dd/mm/yyyy): 20/07/2026.

Status: Accepted.

Context: I want to protect the intellectual property behind the program as well as user data, and profit interests. I must determine whether I need any specific security system, or if all security features are part of a given system/container/component.

Here are the main security concerns:

- Unauthorised requests: I want no individual to receive the service without explicit authorisation. To do so, I need to enforce authentication/authorisation mechanisms. Moreover, “modern cybersecurity standards require passwords to be stored using strong hashing algorithms. The primary reason is to limit damage if (or when) a breach happens. A hashed password database, in theory, forces an attacker to identify each password hash rather than simply reading passwords in plain text. Thus, hashing is essential for compliance and security best practices.” The solution must be strong enough against the usage of rainbow tables and others [287].
- SQL injections: risks exist whether the queries are in ORM or raw SQL (although in my case only in raw SQL, since it is coded this way) [409].
- XSS attacks: Each usage cycle (which includes activities such as filling up the user registration page, completing a user project, paying, editing user info, user rights, editing user project name and extracting user project summary) comprises a per-customer[2] process. There is no blog-like or comment-like shared space on the website. De facto, a user has no input cell it can fill, that another untrusted user will be able to see (thus also unable to edit or hover over, or click on). That eliminate cross users forms of XSS, but not all XSS, since a user can access what he/she entered via the Q&A history ribbon, and because change of display (in our case, from the input cell, to the ribbon) does not neutralise the XSS risk. To neutralise the remaining threat, what matters is that the browser interprets the user's value as data, and never as code/HTML [221].

[2] I say "per customer", because the service is engineered so that someone can supervise the work of other users completing the user projects for him

- Rate limiting: “Rate limiting is essential for any public API. Without it, a single client can consume all your resources, affecting other users” [105].

Options:

- For password hashing: Bcrypt, scrypt, Argon2id, Argon2i, Argon2 [282].
- For blocking SQL injection in raw SQL: adding raw SQL blockers.
- For authentication and authorisation: JWTs or Session Cookies, SSL, SSO (single sign-on) with external providers like OAuth2; request-level authorisation with request middleware (while keeping authorisation logic in the core logic code), specific protocols.
- for additional data security: secrets management systems (e.g. Amazon Secrets Manager, Microsoft Key Vault, Google Secret Manager), WAFs (e.g. Cloudflare, Amazon WAF), HMAC, secure random strings
- Against XSS attacks : WAFs (e.g. Cloudflare, Amazon WAF), auto escape mechanisms [231].
- To achieve rate limiting : either SlowAPI with in-memory backend [218, 331], or using Redis [317], or Flint [236], or pre-written pure Python code algorithms [105], using the FastAPI-RateLimiter module [213]. Rate Limiters can sit either in the App, at the edge or in an API gateway, but each can implement the most optimal limiting algorithms known, namely Token Bucket and Leaky Bucket [205].

Often, tools or libraries can implement solutions to address multiple of these concerns.

Decisions:

- For authentication: OAuth2.0 authentication (with authentication made by an external service provider like Google or other, with the identity user uses to access to that third-party service) with in-app OIDC.
- In-DB secrets management.
- For Password hashing: I use Argon2id.
- To block SQL injection: I use SQL injection blockers within the raw SQL queries.
- Against XSS attacks: I use the Jinja2 auto-escape mechanism.
- For rate limiting, DDoS risks, bot protection, and geo-steering: Cloudflare.

Rationale:

As we will see later, I develop the app as a FastAPI project.

Because FastAPI “does not have built-in protection against CSRF in the way a traditional web framework like Django does”, I use a JWT provider and a Cross-Origin Resource Sharing (CORS) policy to protect against CSRF [231]. For logging in and providing such a system, I can either execute it all in-app, or delegate JWT creation and identity/authorisation creation and management. Creating an authorisation policy is optimal in my case, since I want to fix exactly what a user can do depending

on who that user is (do they manage user projects and who can access them, their payments, can they modify the projects, or just read them, etc.).

OAuth2.0 is a standard protocol to enforce an authorisation policy. It attaches authorisations to a JWT, whether my app or an external provider creates that JWT. Also, OAuth's separation of the Authorisation and the API sides induces that users send requests with the correct permissions without the program having to check the user's rights at each HTTP request [2, 156, 176].

For security purposes, having another service create the JWT is a great option. That way, with OAuth2.0, users can log in to my app using identity credentials they use to access third-party services (Google, Microsoft, company websites, and many others) without sharing login data with my app. Consequently, identity management and security are out of my app, making log-in very safe and free for me (since I do not run the identity management and security implementations; these are just attached to the identity of the user with these providers like Google etc., and I just run FastAPI as an OIDC client), while my users still have an app user account which I store in my DB to link activity and payments with users [2, 156, 176].

Additionally, for secrets management, my Oracle DB (which I choose as explained in Section 2.2.1), has sufficient features [112, 255].

For password hashing: Argon2id is the ideal solution, with parameters that take 250 to 500 ms on verification servers, with OWASP minimum configuration settings, and moving up from there if hardware allows. But it only works if the platform has a well-maintained Argon2 library (e.g. argon2-cffi for Python), and the system is not subject to a compliance regime that mandates something else [282]. FastAPI supports argon2-cffi [249]. My system's compliance regime will not mandate something else. Thus, I can use Argon2id.

For SQL injection prevention: because my queries are in raw SQL (see ADR: Queries code, in code context chapter), the SQL injection will be prevented via SQL text tuning [231, 233].

Next, Jinja2's (the rendering solution I chose; see rendering ADR later) auto-escape mechanism sufficiently addresses XSS risks, as it renders user inputs as data, never as HTML. This is a kill-all-risks solution, unlike WAFs. And it does not add C4 components [190, 221, 231].

For rate limiting, DDOs risks, bot protection and geo-steering, I want to limit requests per specific computer because many concurrent users might share an IP (typically colleagues in an office) [205]. Only those rate limiters sitting in an API gateway or in the App can provide per-user limiting [205]. SlowAPI with an in-FastAPI memory cache only provides per-Uvicorn-worker rate limiting [342]. I expect dozens/hundreds of requests per second in the US deployment, and most HTTP requests get a response in less than a second. Thus, if I use only a few Uvicorn workers, I would need to limit requests per Uvicorn workers/min to a very high number. If I use plenty Uvicorn workers, I would

need to enforce a much smaller per-worker limit, but the compound workers' maximum limit would remain thousands of requests/min. Thus, in either case, one individual could send thousands of requests per minute if I only use per-worker rate limiting. That activity would be far above the normal user activity. Therefore, my program requires a centralised external cache to count a user's requests across Uvicorn workers. Putting the rate limiter in an API Gateway is often recommended at production [287]. This is because the gateway sits between user requests and the FastAPI, ensuring that FastAPI does not get flooded and then realise this internally. I know that I also need a specific tool enforcing that user IPs from a given country go only to the program instance deployed in that country. And I know I also need to protect the app from bots and DDoS attacks. Cloudflare provides all 4 features exactly as I need them [74–78]. Another advantage of Cloudflare is that it works with any hosting solution [72]. And Cloudflare decrypts and re-encrypts signals using the encryption method common browsers use (TLS) [336]. This is convenient since I want that users can access my service via common browsers.

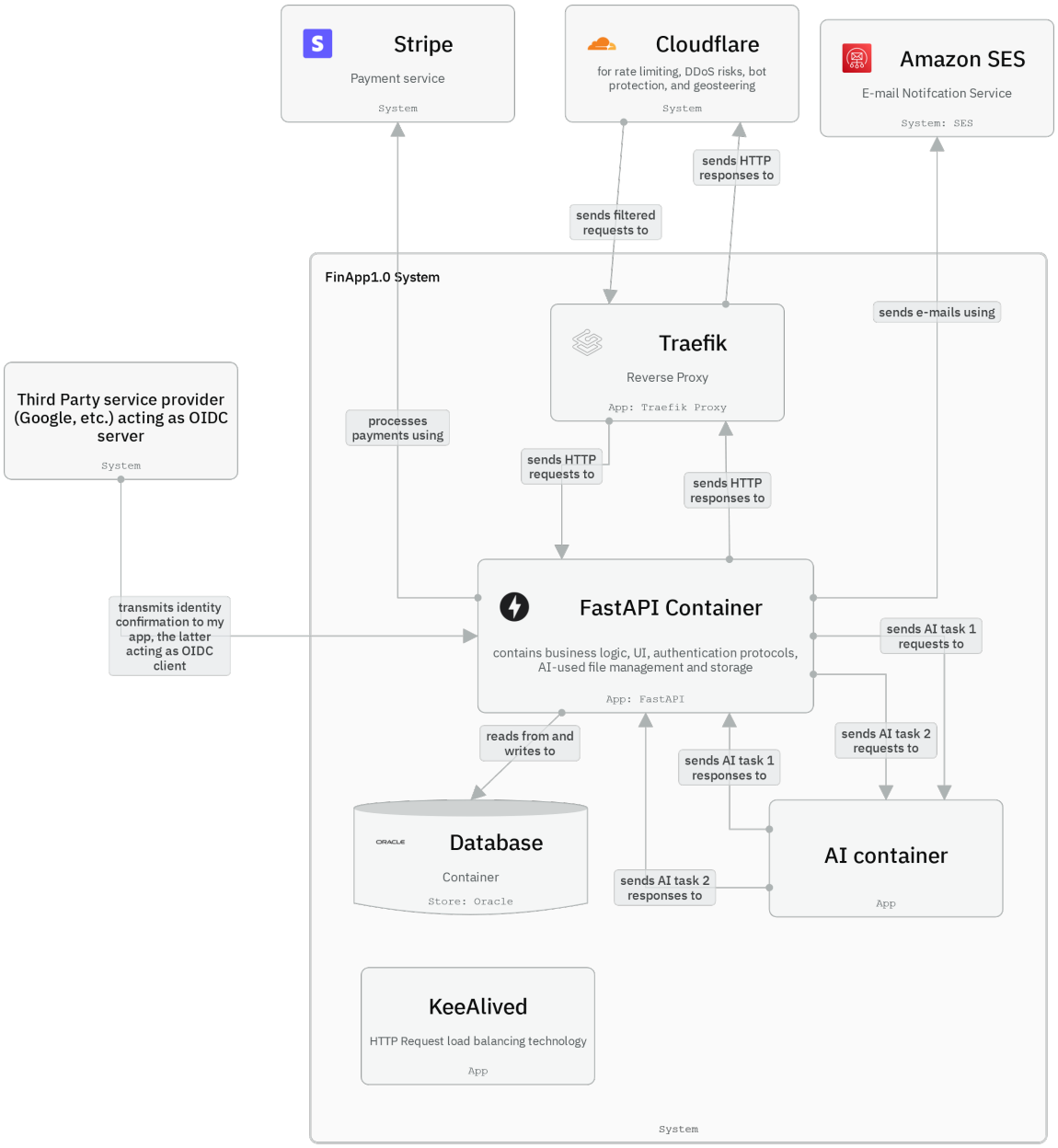
Consequences: I must configure Oracle DB secrets management, and the OIDC and OAuth2.0 part I manage myself are simply the protocol files within my FastAPI C4 component. Also, I need to find an in-app-container TLS decryptor (since Cloudflare re-encrypts the signals before sending them to FastAPI), and I must add Cloudflare in my C4 diagrams. Since Cloudflare is an online service that I purchase, there is no capacity to provision for it. Finally, I must later determine whether anything/what sits between FastAPI and Cloudflare (see 5.8 and 5.10).

2. C4 Containers Context

I here detail only the FinApp1.0 system's containers, as it is the only system with multiple containers.

Also, I do not discuss here why I use KeepAlived, as I do so in the Deployment Section.

2.1. FinApp1.0. System's Containers Diagram



2.2 FinApp1.0. System's container ADRs

2.2.1 ADR: data and rules storage solution

Date (dd/mm/yyyy): 10/09/2025

Status: Accepted.

Context:

I must decide how my program stores user, user project and user project execution data, as well as business-domain and application-level static and dynamic rules.

Options: Stored as code, stored in a database (with multiple possible providers, e.g. PostGres, MongoDB, DynamoDB etc.), held in live memory.

Decision: Static business domain rules for the user project workflow, and user data, both stored in an ORACLE database. App/orchestration level rules, as well as the dynamic business domain rules for the user project workflow, are stored in code.

Rationale:

I must retain thousands of 1kb rows per user project, for thousands of users and user projects over durations from a day to months (see hosting ADR). And I must store tens of thousands of legal, financial, and math rules I have to create myself (because non-existent). Holding all this data in memory would be more costly than storing it in a database, and it carries risks of losing in-memory data if a failure occurs [162]. And my large amount of static rules is a great fit for database storage, as a DB provides a single source of truth, which eases audit, tracking, versioning, and rule search.

Domain design showed that the program uses structured data representing relationships between domain objects (e.g., "this sanction applies to this person under these conditions"), and that one can store that data in tables in a highly logical and query-efficient way, with only a minority in JSON and JSON data would be structured (in ways I designed). However, JSON data is critical and read and written at a high volume. The DB must process requests in the order of their arrival, ideally processing multiple requests concurrently (given the workload volume discussed in the deployment ADRs).

'Relational databases are the best for storing structured data, i.e., data organised in a way which highlights relationships between entities and/or objects [235]. 'Relational databases are recommended for accounting/financial applications, as well as for managing user or user project data [234, 359]. "Relational databases use SQL to store structured data" [234]. 'One can scale a relational database vertically' and horizontally ("by partitioning the data or replicating the database among more database

servers, and using caching and load balancing” [333]) to fit international-scale business needs [12]. This is practical because I can code table creation and other queries in SQL.

The program revolves around users registering, paying service fees, and answering questions, with AI models providing failure messages or small amounts of data per call, storing information the user or the AI models provide, building a response based on the read information, the user’s answer, or small aggregations (at most a few hundred lines), and then returning that response (either stopping the user’s workflow if an error is raised, or not stopping if no error is raised) before the user’s next interaction. This type of program activity is called OLTP (online transaction processing) [12, 344].

Because user project-specific rows are continuously altered, the DB must ensure consistency by accounting for these changes in real time. And the data pool must be secured, as it contains valuable domain knowledge, confidential legal cases, and user information. I must retain all data in a way so users can close and reopen user projects at any progress stage with ease and fast.

SQL DBs such as PostgreSQL, Oracle, SQL Server, and MySQL are the most well-known and commonly used DBs for commercial projects and provide consistency and rapid data task execution. I want to use one of these to learn about a DB I can find in other projects in my career and because they have a strong user community (some also have good customer support). But which one to choose?

“With the right extensions, tools, and configurations, open-source databases like PostgreSQL and MySQL can meet enterprise security standards”, and they each have a strong community, which helps with finding solutions to given problems [12].

Also, I want a DB which has data validation features, because this is key to avoid permanent storage errors [113]. Additional features of interest include the DELETE ON CASCADE command [277].

PostgreSQL runs faster and supports more data types and features (which is beneficial to meet unexpected needs derived from scaling or adding features to the app) and does JSON reads and writes better, and scales horizontally more easily than SQL Server, and all for free [349]. PostgreSQL also supports concurrent table queries, unlike SQL Server, which makes it better suited to high-traffic scenarios [349]. PostgreSQL can run on most widely used OSes, while SQL Server is more closely tied to Windows [12]. But I use Linux machines (see Section 5.2.).

Oracle includes batch processing tasks integration features [361]. PostgreSQL too [99].

PostgreSQL and Oracle rather match in terms of performance, but security features might be a little more extensive on Oracle, and “for highly regulated industries, such as finance and healthcare, Oracle is the preferred database management system. It ensures compliance with industry-specific requirements, such as SOX, PCI DSS, and HIPAA" [114].

Oracle enables data distribution for horizontal scaling [334]. This is great for transitioning from a small to a larger workload as the service gets more popular or when I use the DB's tables for new services (which is totally possible and likely given the nature of my tables).

Finally, Oracle provides its own Linux OS versions, which enable better DB maintenance and DB performance while also allowing other software on the same machine (which I need; see the hosting solution ADR) [253]. Note that putting it all on the same machine might slow down the DB if the other elements' RAM consumption spikes or any other capacity bottleneck occurs. However, for this deployment, I know the required capacity to support the maximum workload (see hosting ADR). Therefore, same-machine cohabitation is perfectly fine.

Consequences:

- The application is stateless [162].
- Conceptualising optimal tables that can store all needed rules and user data optimally is key.
- Communication between the DB and the Python app requires a communication intermediary [203]. In my deployment, I must discuss which intermediary I use and where it lives.

2.2.2. ADR: structuring as a modular monolith

Date (dd/mm/yyyy): 03/03/2026.

Status: Accepted.

Context: The software's structure to meet business needs and optimise scalability.

Options: One can structure the app either as a monolith, a modular monolith, or microservices.

Decision: I structure the app as a modular monolith.

Rationale:

In DDD, bounded contexts are the general tasks that a software runs to deliver a service [67]. I might use some bounded contexts (for other digital services, either as such or with minor adjustments. Whether or not I do so, I want my current service to perform great anyway. In a modular monolith, one breaks code into 1 module per function (in DDD terms, we can say one module per bounded context). Modular monoliths perform great at large scales and under high load if correctly designed. They execute faster, notably because everything is in one place, so they have way less network-call-induced latency than with microservices. They are also 2-3x cheaper to maintain. I can separate each module from the others because each has its own interface. And thus, I can 1) troubleshoot one function without affecting the entire app, 2) paste with minor adjustments some modules in other projects' deployments or 3) easily switch to a microservices deployment (if other projects require similar functions, or if scaling shows few modules require more CPU and ram than the others

significantly enough so that scaling the entire monolith would be cost inefficient, or if the dev team grows to 50+ members and coordination becomes complicated), because the modules correspond already to each microservice I would deploy [158, 215, 281].

An OS's process scheduler can dynamically allocate incoming processes to the free cores/RAM space of a machine. But from the moment that machine reaches full capacity and one needs to scale horizontally, even if only one entry point drives most of the traffic, one must replicate the entire app when scaling, because the inter-machine dispatching technology (often a load balancer) can only spread loads per app instance, not per bounded contexts/entry point [215, 399]. Up to a certain point, this replication constraint is not an argument for switching from a monolith to microservices. But if the costs of deploying and maintaining replications exceed the microservices deployment costs for the same capacity, then and only then can microservices become an interesting option. But this is a rare situation, because, on average, monoliths are 2-3x cheaper to maintain, so microservices are more profitable only at a scale of solar proportions. In practice, even apps driving billions of daily interactions often favour a monolith because it costs them 50%+ less and offers performance benefits [399].

Consequences: Unlike for a microservices app which shows as multiple separately deployable smaller apps at the C4 container level, a modular monolith shows as a single block at this C4 level [54].

2.2.3. ADR: usage of AI models

Date (dd/mm/yyyy): 22/07/2026

Status: Accepted.

Context: My program's workflow includes one big task I dub 'financial case info fetching' (here also shortened as 'info fetching'). My system needs software that reads documents to facilitate that task.

Options: I can build that software myself, or use a third-party provider's [203].

Decision: I use AI models, more precisely a Vision-Language Model (VLM) for a task I dub 'task 1' and an embedding model for a task I dub 'task 2'.

Rationale:

The software here communicates only with the core domain code, not with users. The core code sends a standardised request to the software as follows: "*table_name.column_name for id = x, needs info I in measurement unit U (if I is a numerical value), from target document D, which uses formatting convention F (e.g., the financial record model from an accounting organisation, or any imaginable formatting convention from a third party or that I enforce)*".

The target document D is either an original source that the user attached to the user project, or a file summarising info from the original source(s). Original sources can contain information written in text format or embedded in images. Thus, there are two general workflow options for the software to complete its job:

- 1) At every request, the software reads the original source(s);
- 2) When the user attaches the sources, the software extracts everything it might need, then for every info request, the software parses the extracts to find the target info.
- 3) Same as option 2, but instead of parsing the entire extracts, the software parses only a section of the extracts.

In all sub-workflows of the core code app that need 'financial case info fetching', the app repeatedly (from 10 to thousands of times) needs info from the target documents. And each info fetching request is for a single info; before advancing to the next step, the workflow always needs to know if the info is found (and if found, what is its value). In the main sub-workflow of my service (80-90% of FinApp1.0's activity), the program does not know what information it will have to fetch in the next step of that sub-workflow, nor how many next steps there will be. Thus, there isn't a way for the app to disqualify info to extract from original sources as the workflow advances. And using a rule such as "*when the core code app is asked to find info more than X times, fetch everything you might need out of the original sources*" is not better than option 2 or 3, for it implies more full-original sources parses than option 2 and 3, which would make the software's activity more expensive because the parsing of potential images found in the original sources is more expensive and latency heavy than parsing extracts such as JSON extracts. On the other side, I know just how many times it can ask to fetch info from documents if extractions are separate, and I know that the core code app usually needs 60% or more of a document. Lastly, since each request for info is a request for a single info, there is no need to reparse the entire extracts each time; the extraction should categorise extracted info in logical sections that the code knows the logic of, so the code can ask for a specific section to parse only (this is all totally possible). So I opt for option 3: a one-time extraction organising info logically (in a "{infoname:infovalue}" and sections-based way) in one or multiple JSON files (herein referred to as "Task 1"), and then reads of a section of the JSON file(s) (each read is an instance of what I dub "Task 2"). Once that one-time extraction completes, the program can flush original sources.

I implement a "*if software does not find the info in explicit terms, ask user*" fallback. And users are liable of the usage they make of the software's delivered service. Users can also choose to provide all info manually rather than via input sources, by not uploading documents. Thus, software-assisted info extraction does not need to be 100% exact, but something as exact as possible makes the user experience easier.

Task 1 must merge all extracts of a user project as a single JSON file, so that task 2 looks within a single file, thus eliminating the need of JSON file selection logic.

Due to the fallback aforementioned, the software does not need to know very rare formulations of the target info. But the software must know common formulations in each possible language, since it must analyse documents in any country's language, regardless of the customer's country (because users work on international or domestic financial cases that documents in different languages might provide info about). Thus, the software must either contain all needed knowledge, or my FastAPI app must send it the sought formulations.

Also, the software must find target info which are numerical values, regardless of the measurement unit, whether the measurement is written in Arabic numerals or in words, and whether the unit is written in full or in abbreviated form.

Many input documents or parts of them will be images. An LLM-VLM must perform task 1 because VLMs can extract text from images and order it in a structured way due to general comprehension of given guidelines and context, whereas the other text-extraction technology alone, namely legacy OCR, can only extract word-for-word with no context comprehension and with no info re-ordering [59, 211, 212, 283, 370]. To do task 2, an embedding model is the only technology I found capable of retrieving info from a JSON file when that info's formulation is not 100% predictable (but structure is, since the parsed JSON follows a strict known structure) [14].

Note, I need only one model for each task for the following reasons. AI models are made of one or more parameters (more or less like the model's tools) and a context (the pool of info that the tools can work on at once, we could say it is its 'reality') [115, 397]. RAG is a form of answer-building logic that relies on data external to the model's context [144]. RAG is useful when one cannot fit all the data into the context window required for correct inference, either because the model's context window is too small or due to other obstacles to inserting that data into the context. This is what the main LLMs on the market (form Anthropic or OPENAI for example) use to find info from online sources in real time. There is an argument for RAG for task 1 because the parsed documents, though following a general theme (legal or financial info), might also contain some industry-specific info (e.g., trade rates, merchandise information, chemistry/physics indexes, etc. But when a document contains such info, the info is named, and that is all my task 1 model really needs to extract that info according to the guideline JSON structure my code specifies. To me, this disqualifies the need of RAG. There is an argument to use RAG for task 2 as well, because the models' parameters and context might not contain all needed info about the expected potential formulations of a target info (storing all this represents millions of words or more because of the large variety of info the model might search in my case due to the nature of the service). However, despite the eventual overall big size (see US deployment ADRs), I can load all needed info for a reasonable price by sending task 2 requests with

the needed formulation info (from a table queried pre-AI request which indicates common formulations of an info in a given jurisdiction) and an order for task 2 to load only a small JSON section based on date index and JSON file sections names (e.g. “{‘financial records’;’13 August 2023’}”) that the core code specifies. Thus, I do not need RAG. Thus, and knowing RAG needs multiple models but embedding alone can exist as a single model [223], I do not need multiple models for each task.

Consequences: I must determine which models to use, and where do they live and operate in my architecture.

2.2.4. ADR: using a RESTful API

Date (dd/mm/yyyy): 10/04/2026

Status: Accepted.

Context: Domain code-HTTP users’ communication requires a clear interface [203].

Options: That interface can be any web application programming interface (API) [394].

Decision: I choose a RESTful API.

Rationale:

I can use a RESTful API with stateless apps, and my app is stateless. Also, that type of API separates client and server sides clearly, so that I can modify each independently of the other at any time [63, 108, 152]. And a RESTful API pairs greatly with the usual medium-to-large-production-stage-quality cybersecurity solutions, namely, JSON Web Tokens (JWTs), Session Cookies, a TLS (transport Layer Security) or a SSL, SSO (single sign-on) with external providers like OAuth2, request-level authorisation with request middleware (while keeping authorisation logic in the core logic code), a secure random string, HMAC, secrets management systems (e.g. Amazon Secrets Manager, Microsoft Key Vault, Google Secret Manager), WAFs (e.g. Cloudflare, Amazon WAF), can ensure the security of sensitive (notably for authentication) data transiting in the stateless app, and API keys enforced within an API gateway .(e.g. Amazon API gateway, Google Apigee, Cloudflare API Shield) [58, 200, 231, 332, 355].

Consequences: I must determine how I deploy it.

2.2.5. ADR: using FastAPI

Date (dd/mm/yyyy): 10/05/2026

Status: Accepted.

Context: I want to use a web deployment framework, as it is software that structures and simplifies the deployment and code writing of a web app [369].

Options: I can use any Python web framework designed to code and deploy RESTful APIs, e.g., Django, Flask, FastAPI, etc.

Decision: I choose FastAPI.

Rationale:

FastAPI can work with all the elements I yet chose: the RESTful API for web-app communication, the Python-OracleDB driver, and vLLM.

Also, FastAPI allows me to code an async RESTful web API benefiting from the ASGI server Uvicorn, which I chose (see ADR concurrency in the C4 level of the paper), but also to benefit from all the routing, serialisation, Pydantic data validation, OpenAPI spec automatic writing, and dependency injection “Depends” features coming along with the Starlette-based FastAPI implementation (features that I need, and that Uvicorn alone does not provide) [117, 368]. This ensures that I already benefit from these features on top of Uvicorn, in their most performant form, without having to manage or write them. Overall, this web app framework proved faster than Python competitors like Flask and Django [366].

Also, FastAPI allows me to use both asyncio (which I selected: see ADR concurrency in the C4 level of the paper) and synchronous handling in Python within user-project sequentiality, to use the async/io libraries as well as threads/multi-processes for CPU-bound tasks, and to use type hints to facilitate code writing [33, 118, 119, 387].

FastAPI is faster than other Python frameworks for my program, whether or not those frameworks use asyncio. Also, FastAPI automatically generates the OpenAPI spec from the code, which is helpful because the spec is a key file for understanding the app's client-server communication. The Depends and Pydantic Models one can use in FastAPI simplify dependency injection and validation, respectively. And the documentation is exhaustive and detailed [171, 386].

Consequences: I must determine how the other key architectural elements, supposed to interact with the core code, interact with FastAPI, and how FastAPI receives requests.

2.2.6. ADR: where does Jinja2 run?

Date (dd/mm/yyyy): 02/05/2026

Status: Accepted.

Context: In the components context chapter, I explained I chose Jinja2 rendering. I need to determine where it lives (what runs it).

Options: In the FastAPI app, or in a separate frontend app [54, 97].

Decision: The frontend lives in FastAPI.

Rationale: This frontend can physically live within the FastAPI container in terms of deployment, thus including in the C4 deployment diagram [120, 121]. The nature of the frontend (almost only static HTML) induces that I should put this frontend within the FastAPI c4 container rather than as a separate c4 container [54].

Consequences: At the C4 container level, one will not see the frontend as a distinct container, since it is a part of the FastAPI container, and each component must own all the responsibilities it includes [54].

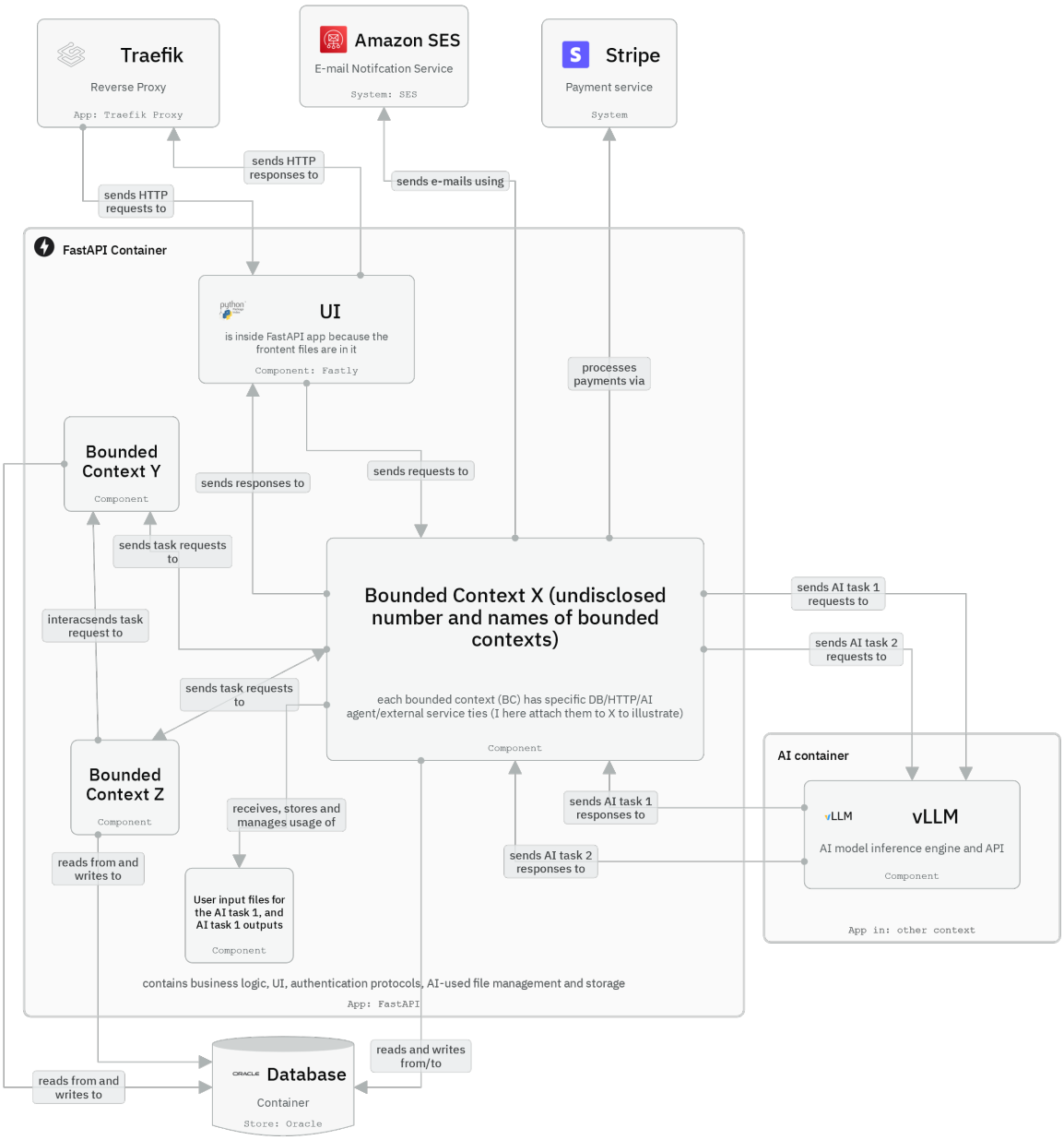
3. C4 Components Context

Here, I only dissect the FastAPI and the AI containers, as the other elements' detail are not different from one program to another, or are deployment-relevant only.

3.1. FinApp1.0 System - FastAPI Container's Components

At this level, C4 presents the fragments of runtime units [55]. I thus do not detail the FastAPI files (e.g., Dockerfile, toml file, uv.lock, test.DB) which are not fragments of runtime units.

3.1.1. Diagram



3.1.2. ADR: DDD bounded contexts as some of the app container's components

Date (dd/mm/yyyy): 10/06/2026.

Status: Accepted.

Context: I must determine the number and nature of C4 components of my FastAPI app container.

Decision: I divide the FastAPI app container into components, which are its bounded contexts (whose names I keep private).

Rationale:

My code must create the lowest coupling and highest cohesion [140]. "Coupling refers to the degree of interdependence between software modules. […] Cohesion refers to the degree to which elements within a module work together to fulfil a single, well-defined purpose" [140]. Obtaining the lowest coupling and highest cohesion means that a change in one part of the code must entail the fewest changes to other parts of the code, that each part of the code performs a single task, and that there is no redundant code. Achieving this concretely happens in code, but conceptually starts at a higher level, when determining the general tasks of the FastAPI app, because determining non-overlapping tasks reinforces cohesion.

DDD is a framework for conceptualising the invariant core code of a software [67]. It particularly fits complex finance software projects [7, 220, 292]. In DDD, bounded contexts are the general tasks that a software runs to deliver a service [67]. Bounded contexts correspond to the modules of a modular monolith (in C4 language, they correspond to the components of an app container) [154]. My app is a modular monolith that provides a complex financial service, and I want to facilitate modules' migrations to other projects/microservice setups.

3.1.3. ADR: data caching

Date(dd/mm/yyyy): 03/06/2026

Context: I want to minimise I/O and DB-related latencies beyond DB caching solutions [346].

Options: I can add a remote cache (outside the app, e.g., Redis) or a local cache (in the app) [346]. And of course, I can also choose not to add other caches.

Decision: I will use only Oracle DB buffer caching, thus adding no other cache.

Rationale:

My app is a modular monolith. A modular monolith suits a local cache better than a remote cache. Local caching improves performance, is easy to set up, and does not introduce additional network calls, infrastructure, or costs, unlike a remote cache. It is also more secure than a remote cache [92, 346]. Therefore, let’s focus on whether a local cache is suitable.

Only a single FastAPI instance serves at a time (see hosting ADR). Thus, data changing due to user requests remains consistent within a local cache. For rather simple queries, whether calling on new data or on the same data already optimally fetched by indexing and in-DB caching, the in-app cache is overkill, because checking the cache, de-serialising data, and managing cache keys would actually be slower than just hitting the database directly [217]. But many of my queries are grouped and analysing the same data pool: a use case analyses a certain data pool, and depending on the result, it usually inserts or updates a row/rows in a table (other than form that data pool) and provides to these/this row(s) values that can be found in the data pool. These two characteristics make close caching attractive. However, I must also consider cases of cache staleness, and security concerns.

The DB admin’s data manipulations often are writes of legal/financial rules for a more recent period not yet found in the database (e.g. rules for 2027). Therefore, they do not make the cache incorrect, assuming that the FastAPI app still performs a DB query to query that data in the DB if it does not find it in its local cache. But some DB admin manipulations will include updates to correct mistakes, and in these cases, if the info was already in the local cache before the update in the DB, the in-app cache has out-of-date data. Oppositely, when a db admin changes a data in the db directly, that change naturally hits the DB's cache, ensuring data consistency.

Moreover, the data pulled is always sensitive: it contains either user info, billing info, or financial/legal info specific to a business discussed in the user project. This is too sensitive to keep in an in-app cache [217].

3.1.4. ADR: rendering and frontend solution

Date (dd/mm/yyyy): 01/05/2026

Status: Accepted.

Context: I must determine my rendering solution, then pick an HTTP request traffic system and security features compatible with its security and functionality.

During the questionnaire part of the user project completion, questions are answered one after another. Answer confirmation redirects the user to the next question/question group.

For user creation and payment, the display follows the usual standard for these subservices: a single form-like window with all required information, and a single confirmation.

During the info fetching part of the user project's completion, which takes the form of a questionnaire from the user's point of view,[3] the user sees a "Q&A answer history ribbon", in which each info the user or AI model fetch during the questionnaire part, is displayed in chronological order of fetching. This ribbon enables the user to find and edit information already provided about the financial case. I do not require an expand/collapse behaviour for this ribbon. The list of info fetched from questionnaires tends to grow long, and reloading the entire ribbon every time might be time- and energy-intensive. Therefore, the rendering of the ribbon's contents would benefit from a dynamic or dynamic-like solution that loads only the new info fetched since the last HTTP request after each HTTP response, while all previous info stays on display.

Decision:

I use Pure SSR with Jinja2 to render the entire app, except for the financial case info ribbon and the question placeholders (which will rely on a Jinja partial template, e.g. _ribbon_item.html and HTMX).

Rationale:

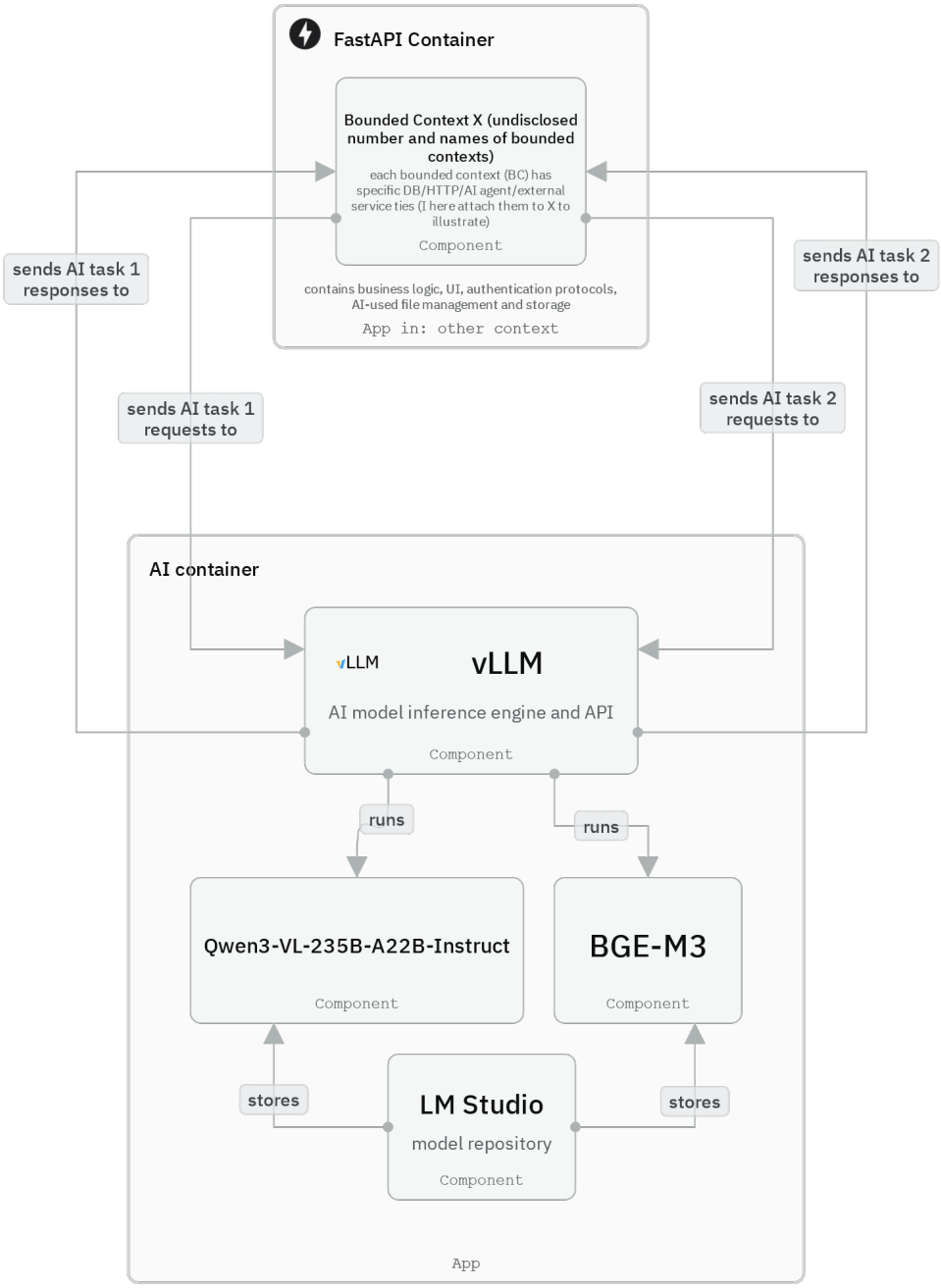
- Pure SSR with Jinja2 works great for static rendering [284].
- Jinja protects from XSS attacks via its auto-escape system [231].
- The advantage of HTMX, is that it allows, without using .js frontend, questionnaire frontend to be a questionnaire page, with the only thing that changes on this page being the question and the history ribbon, which are added to the frontend page as HTMX fragments. Notably, for the Q&A answer-history ribbon, hx-swap="beforeend" appends the newly obtained information while preserving previously displayed entries [164, 165].
- I found no evidence that this rendering solution prevents rendering on both computers and smartphones. Sources seem to defend that this flexible rendering (dubbed Responsive Web Design) works with HMTL/HTMX rendering (which is what I rely on here) [226]. This thus allows users to access their user account and the service on all their devices, which is valuable in my case.

Consequences: I must decide where this frontend resides/what runs it.

[3] It is not strictly fetching answers from the user's answers, in so that the AI container contributes to finding info as well, hence me saying "a questionnaire from the user's point of view".

3.2. FinApp1.0 System - AI container's Components

3.2.1. Diagram



3.2.2. ADR: AI models choice

Date (dd/mm/yyyy): 22/07/2026

Status: Accepted.

Context: As we recall from the previous C4 layer ADRs, an LLM-VLM must perform a task named AI task 1, and an embedding model must perform a task named AI task 2. Which model to choose?

Decision: I use Qwen3-VL-235B-A22B-Instruct for task 1, and BAAI's BGE-m3 for task 2.

Rationale:

Many models yet exist to perform tasks 1 and 2. Coding models myself would be unnecessarily bothersome, and I have no expertise in that field. Therefore, I focus on third-party AI models that I can deploy co-locally to my core code/app instance, ideally open-source.

For task 1, the best VLM seems to be Qwen3-VL, because it recalls extraction rules and previously-parsed info well during a long content OCR task, 'supports the most common 32 languages', and because its max context window accepts up to a million tokens (that includes the input and output tokens) per task and the context window per task 1 request (see Section 5.8) is under that 1M limit but over the limit input tokens per task of the competitor VLMs [405]. In benchmarks (like MMLongBench-Doc) of performing OCR on long documents but dressing relationships during the OCR, the Qwen3-VL-30B version performed significantly worse (at least 10% worse) than the 235B version [308]. For parsing financial documents specifically, which is the most of the documents the model will have to parse in my program, other benchmarks also report 10 points of performance difference between the two models on every ground of evaluation [410]. Also, only the 235B version seems to be at the level of commercial models [405]. In my case, because the amount of proofreading the user must do on the resulting finapp1.0's service delivery summary (a document which summarises to the user exactly what info the program used from the sources it used to deliver the financial service) is an important part of the user experience and can last some hours for the longest projects. Therefore, I select the 235B version of Qwen3-VL. But I stress that if future benchmarking (which I cannot do at the moment) proves the 30B version is sufficient, I will adopt this version.

For task 2, the most performant and open-source visual embedding model able to extract from both images and text files is BAAI's BGE [47, 223]. Its maximum sequence length (how many tokens it can handle at once) and other aspects can vary, and choosing the right config depends on request load and size [47]. But as discussed in the deployment ADRs later, I choose BGE-M3 for task 2. This model has a context window of 8,192 tokens max [47] which is sufficient for task 2, since each task 2 request has a 1,220 tokens' context window per task (see Section 5.8).

Consequences: I must determine how to run, and deploy these models.

3.2.3. ADR: AI-FastAPI communications and merger?

Date (dd/mm/yyyy): 22/07/2026

Status: Accepted.

Context: I must determine how the AI models I use communicate with the core FastAPI code, and where they run and are stored.

Options: AI models sit inside or outside the FastAPI c4 container. If sitting outside FastAPI, they can communicate with FastAPI via either an Application Programming Interface (API) or a Model Context Protocol (MCP) [203].

Decision: AI models sit outside the FastAPI instance and communicate with it via an AI API.

Rationale:

Models simplify case info fetching. If the code cannot reach them, it can still ask the user for info. This way, the app does not stop. This workflow aspect works whether I embed models in code or not.

AI models perform well only with the type of context they were trained with [223]. A model trained with financial documents dating up to 2026 might not perform as well in 2028 if the documents' formats and layouts change by then. Thus, I might change models or their versions in the future.

Not embedding my agent in code eases AI model changes and allows loading the model on its own server or Docker container. This way, dependencies remain separate, which simplifies debugging and modifications and capacity consumption (big for the AI runs, small for the FastAPI code runs) remains separate too, giving more deployment flexibility (notably for providing a more tailored hardware specifically for the AI activity of the program).

An API enables communication between a program's components by exposing endpoints: an endpoint specifies who can talk to whom and about what. An MCP is specifically used for software with AI models. The MCP enables more informed communication between an AI model and another component. To do so, the MCP exposes the component's full context to the model [150].

In my case, my program contains the AI models. And the task 1/ task 2 requests contain all the info the models need to do their tasks. The model does not need to understand how the entire core code works to answer the request appropriately. Thus, I don't need an MCP and stick to an AI API.

Consequences: I need to select an AI API. Also, since FastAPI does not host the models, something else must (commonly a model repository). And it also needs its own engine to run. And I need a GUI.

3.2.4. ADR: AI models' AI API, engine and management

Date (dd/mm/yyyy): 22/07/2026

Status: Accepted.

Context: I must select an AI API, an AI model repository, an IA model engine, and a GUI.

Options: There are APIs, different engines, GUIs and repositories on the market. The list is long.

Decisions: I choose vLLM (which is both an engine and API) and LM Studio as repo/GUI.

Rationale:

My models are a VLM and an embedding model. vLLM, SG Lang, and TensorRT-LLM are the server engines for running large-scale embedding models and VLMs. While their performances are equivalent, vLLM is free, deploys more easily and is compatible with more hardware brands [178]. vLLM also serves its own API, even if the deployment is local and offline [390].

That server engine vLLM and these 2 models are available together only in the Hugging Face AI repo (the other repos have one or the other) [47, 166]. Hugging Face seems to offer an AI API, but it's not free. LM Studio is another repo. It is free, has an intuitive interface, and I can import the two models from Hugging Face into LM Studio [167, 397].

Some businesses provide GUI, API, model selection, and storage features as one service. They seem to support my task 1, task 2 system, because they let me define different request types that the core code app can call [379]. However, they are not free, even when deployed on my own hardware.

Consequences: I must now determine how to deploy the AI container (see Section 5).

4. C4 Code Context

I here discuss architecturally-relevant code choices which do not reveal the nature of FinApp1.0's code/domain.

4.1. ADR: user requests' processing strategy

Date (dd/mm/yyyy): 10/05/2026 (with an additional source added in August, to mention worker anomaly and re-creation features of Uvicorn, plus with a later confirmation that the decision in this ADR matches the capacity plans).

Status: Accepted.

Context:

Together with the hardware, user requests' handling (herein referred to as the "processing strategy") determines the app's performance and scalability [322, 383]. My program revolves around two 'scales of execution': the "user" workflow (the stream of requests-execution-responses for a specific user), which is sequential (because the prior HTTP response determines the possible next HTTP responses), and the "across-users" workflow (the stream of requests-execution-responses for the entire app) whose processes can run simultaneously (because the prior HTTP response of one user does not determine the possible next HTTP responses of other users). I need to find the processing strategy that best implements these two workflows. For reminder, I code in Python.

Options:

Parallelism is the practice of executing tasks simultaneously, each in a separate thread. Concurrency is a complementary approach that improves the scheduling and execution of tasks on a given processor thread [204].

Background tasks, Celery, and Asyncio are 3 ways to achieve effective concurrency of I/O-bound tasks in Python (concurrency programming) while ensuring each user's workflow remains sequential [98, 207, 309].

Spreading requests processing across multiple workers or threads on a single CPU core, or multiple CPU cores, is the lower foundation of parallelism [204]. Then, the app must split tasks across 'app threads', which it can spread across the CPU threads available. In Python, these 'app threads' are called CPython processes, e.g., in FastAPI, the unit of execution a Uvicorn/Gunicorn worker executes. Python threading alone, due to a mechanism called the GIL, creates concurrency within a CPython process, thus exploiting a single CPU thread. Therefore, to ensure parallelism in Python, one must use multiple CPU threads and create multiple CPython processes instead of trying to create Python threads within a CPython process [216].

Decisions: I use asyncio and multiple Uvicorn workers to enforce sequentiality within each user workflow and true parallelism of the user workflows.

Rationale:

First, for the deployment I discuss here, the program expects hundreds of requests per second that a CPU with multiple threads must handle (see Section 5.8). This means that I can benefit from splitting the work across multiple CPU threads, making concurrency and parallelism relevant.

My program revolves mostly around single-table lightly/moderately joined queries on a small-to-moderate row count per call. Such tasks' latency is mostly bound to the performance of the i/o [93]. That I/O latency dominance means these tasks are 'I/O-bound' [287]. Only concurrency reduces the I/O latency [102, 159, 210].

Asyncio handles high concurrency better, provides much better cancellation and lighter task switching, and comes with fewer concerns about race conditions, but it is only compatible with a few Python web frameworks, and external libraries only support it in their asyncio-compliant versions, if any [98, 207, 309, 375]. But 'asyncio provides sub-optimal execution time when legacy libraries must block the code or the DB driver used is synchronous' [45, 301]. Asyncio alone is best for i/o-bound tasks [247]. If asyncio can be used, I will exclude Background tasks, for they do not provide a retry system in the event of task/system failure/reset [239].

Asyncio is compatible with parallelism [304]. Secondly, the DB driver I need supports asynchronous programming, notably with asyncio [250]. Also, Oracle DBs are compatible with Celery, requiring no more effort than other databases used with Celery, in case I ever change my asynchronous handling from an asyncio-based system to a Celery-based one [36]. Lastly, I have asyncio versions of Python libraries my program needs (this only regards i/0-bound tasks; CPU-bound tasks are discussed after):

- Oracle DB-Python communications (storing and retrieving info, whether the user answers the questionnaire, or new user projects, or user info): such communications require setting up a connection between Python and the Oracle DB, using either Python-OracleDB in thin mode or in thick mode (the latter requiring using Oracle Instant Client libraries) [258]. I found the "aioodbc" library for connecting to my Oracle DB from Python, using Oracle Instant Client ODBC as a substitute for the usual "pyodbc" library [103, 189, 197, 258, 293]. When not using Oracle Instant Client (i.e., when using the python-OracleDB driver in thin mode instead of thick mode), it is also possible to use asyncio [250].
- Client/service-like communications: Since my program is currently at the prototyping stage and only receives inbound HTTP requests, and queries only via drivers rather than making outbound HTTP calls to external services or internal microservices, I do not need to give the program the ability to act as an HTTP client for outbound calls. Therefore, I don't yet need an asyncio library

that provides that ability (e.g. aioHTTP), but I know that such libraries exist if I may add such a feature to my program at some point [155].

- Authorisation and permissions checks (e.g. user wants to create a new user project, system must check if user is authorised to do so):
- unique/sample of data assignment (e.g. when the program executes a calculation, either an algo or a formula, 1st it must assign the right values to each variable in formulas, or the right data sample to an algorithm) (noteworthy if that falls under data retrieval, please ignore this bullet point)
- mailing: aiosmtplib is a library implementation of smtplib, i.e. the Python library for sending emails, compatible with asyncio [56, 131].

Therefore, asyncio is compatible with my sequentiality requirements and the 95-99% I/O bound of all my programs' tasks, with the other tools of the program, and is a good fit if I ever add parallelism.

Also, as said earlier, the app would process user requests much faster in parallel because the requests/s exceed what a single thread can process in a second. In my case, I cannot tolerate growing processing queues. Therefore, parallelism is necessary. Uvicorn is the fastest WSGI/ASGI solution, creating multiple CPython processes for true parallelism, for an I/O-bound Python app (deduction I base on TechEmpower querying benchmarks). It also has anomaly detection and worker recreation features, and it is fine that it does not have workflow re-dispatching like Celery does, because the user can just re-send the request, it is not expected to really make the user experience bad, since it should not happen often (and if it does I troubleshoot) [198].

Consequences: I must find a config that allows me to use asyncio and Uvicorn.

4.2. ADR: Clean Architecture and DDD division and annotation of the code

Date (dd/mm/yyyy): 08/10/2025 (and some source additions later in March 2026)

Status: Accepted.

Context: I want my code to be organised as logically and clearly as possible, so that the general role of each part of the code is clear just by looking at a single or a few easy-to-find lines/words (whether folder names or others).

Options: I can use code tags, specific code folder names, write Python comments, and/or use specific colouring/highlighting on the code text.

Decision: I present and organise the contents of each DDD bounded context using Clean Architecture (CA) as the main sections and DDD for the subsections within the CA domain layer, as well as in-object-name and in-function-name DDD and CA tags.

Rationale:

Code text colouring/highlighting is not a sufficient solution alone, because the program's number of bounded contexts already exceeds or nears the number of easily distinguishable colours and because there are other dimensions to account for (is this code element a parameter, an object name, what is its workflow role beyond its Python syntax nature, etc.). Also, I want to organise and annotate the code so that any developer can easily understand it and use their own colour-coding system, since multiple developers/software engineers will work on it.

In-element-name tags and folder names are straightforward and reliable. When you look in any folder, you can see from the tags in the name of any element which file it is stored in, its workflow role, and its bounded context of belonging.

Like DDD, the CA framework best helps developers achieve low coupling and high cohesion (which all programs aim for) and modularity (which I need to obtain separate reusable and project-specific code) in large, complex financial programs (which perfectly matches FinApp1.0's description) [160, 402]. CA helps conceptualise and organise the code and other software elements that the core invariant code needs for a deployment, therefore facilitating clean code design [84, 138, 402]. CA also supports isolation testing of the different code objects [340]. CA's most invariant code layer (dubbed *Enterprise Business Rules* or *Entities*) corresponds to DDD's Tactical design elements, i.e. the DDD elements created to materialise the DDD bounded contexts [67, 170]. Then, CA's Application Business Rules, Interface Adapters, Frameworks and Drivers use these Enterprise Business Rules in a service deployment context. Since my FastAPI C4 container's components (besides the frontend) are its bounded contexts, and in my case the 3 most inner CA layers contain the Python code equivalent to the bounded context code (the outermost layer is frameworks and drivers configuration code, which is supporting all my bounded contexts but not specific to one, each bounded context's code should gather all CA code specific to it, organised between the 3 most inner code CA layers each code block belongs to, with the *Entities* layer's code elements classified using DDD [154].

The bounded contexts thus contain the following elements (this is simply a summary of DDD and CA components), and I assign each element a bounded-context tag, a CA/DDD tag, and a more specific in-name role indication. E.g., if something is an output of a bounded context, it will contain the bounded context tag and the word 'output'. E.g., if something is an app business rule, it will contain the bounded context tag, a "UseCase" tag, and an indication of the more specific role of that use case. And I also add a FinApp1.0 tag when the element is specific to this app and not reusable in other services:

- ENTREPRISE BUSINESS RULES (also dubbed the Entities, but that is not to be confused with the Entities concept from DDD). This is the CA category I subdivide using DDD's Tactical design elements, namely [38]:
    - Models: Models are aggregates (which are made of multiple DDD entities and DDD value objects, but with at least one DDD entity, which acts as root) or entities and value objects not part of any aggregates, and they must contain rich behaviour [194]. An entity is an object whose attributes are mutable and whose ID is immutable (e.g. a user project instance object with a changing 'status' attribute, a 'student' object with an email address which might change), therefore defined by its ID, while a value object is an object whose attributes are immutable and whose ID is immutable, therefore defined by its attributes (e.g., an address is a value object, because it is unique not because of its ID, but because there are no similar address in the world) [398]. The rules they must follow are as follows. First, 'Entity relations should not hold a reference to another entity class; it can only keep the ID of another entity. This entails that an aggregate can only depend on another one by citing the other aggregate root's ID as a foreign key' [194]. Secondly, 'An aggregate describes a cluster or group of entities and value objects that can be treated as a cohesive unit' [376]. Thirdly, 'to maintain separation of aggregates and keep clear boundaries between them, it is a good practice in a DDD domain model to disallow direct navigation between aggregates' [376]. "Aggregates should remain unaware of each other; they are independent units. They serve as a facade to hide the internal workings and specific business logic from anything outside their boundaries. However, it's acceptable for any object inside the aggregate to hold a reference, specifically the unique ID, to the root of another aggregate" [135].
    - The domain events: 'Domain events capture moments, representing something that has occurred within the domain in the past, and which influences the rest of the system' [318].
    - The DDD repositories (aka concrete repos' interfaces): DDD repositories (aka concrete repos' interfaces) abstract data persistence. "DDD repositories (aka concrete repo's interfaces) separate data access logic from the domain model. They provide a consistent interface for querying and storing domain objects. DDD repositories (aka concrete repo interfaces) hide the specifics of how data is retrieved or stored. They encapsulate the translation between domain objects and underlying data storage methods, such as databases or external services" [7]. This means that they contain the abstract representation of concrete methods living in the interface adapters layer, because the interface adapters layer is outer to the layer calling the methods (Application Business Rules layer), inducing that calling the methods in the Application Business Rules layer requires an abstraction at the use case layer level or the entities layer to ensure we respect the dependency inversion principle (and in practice, putting such abstraction in the domain layer is preferred here,

because I have domain services which might one day be written and which will call these interfaces, so they cannot sit a in an outer layer than the domain) [116].

  - The domain services: “In DDD, a Domain Service is a stateless, operation-centric class that encapsulates domain logic or operations that don't naturally fit within the boundaries of an Entity or a Value Object. These services are all about performing actions or providing domain-specific functionality that doesn't belong to a single entity” [6].
  - The factories: Objects that have the single responsibility of creating other objects [52].
- APPLICATION BUSINESS RULES, which use ENTREPRISE BUSINESS RULES elements and tools to act on specific observations
- INTERFACE ADAPTERS (also called Anti-Corruption Layer in DDD [188]): these are “adapters that convert data from the format most convenient for the Application Business Rules and entities, to the format most convenient for some external agency such as the Database or the Web” [222]. Interface adapters include:
  - Controllers: Controllers receive an HTTP request, hand it to a use case, and (in some systems) return a response. a controller contains: 1) Endpoint definitions: the route, the HTTP method, the path parameters and query parameters; 2) Request parsing and validation: in FastAPI this is largely done by Pydantic input models declared on the endpoint signature, but the controller is where you decide which model to use; 3)Dependency injection declarations: in FastAPI, Depends() calls that inject DDD repositories (aka concrete repo’s interfaces), services, or handlers into the endpoint function; 4) Calling the appropriate use case or handler: one line, passing the parsed input; 5) Mapping the result to a response model: taking what the use case returned and passing it through the Pydantic response model [290].
  - Presenters: “the Presenters repackage the OutputData into viewable form as the ViewModel”, which, in the context of FastAPI, is either a Python dataclass or a Pydantic Model [290].
  - Gateways: the bridge/translator between the core code app and an external service it talks to [157].

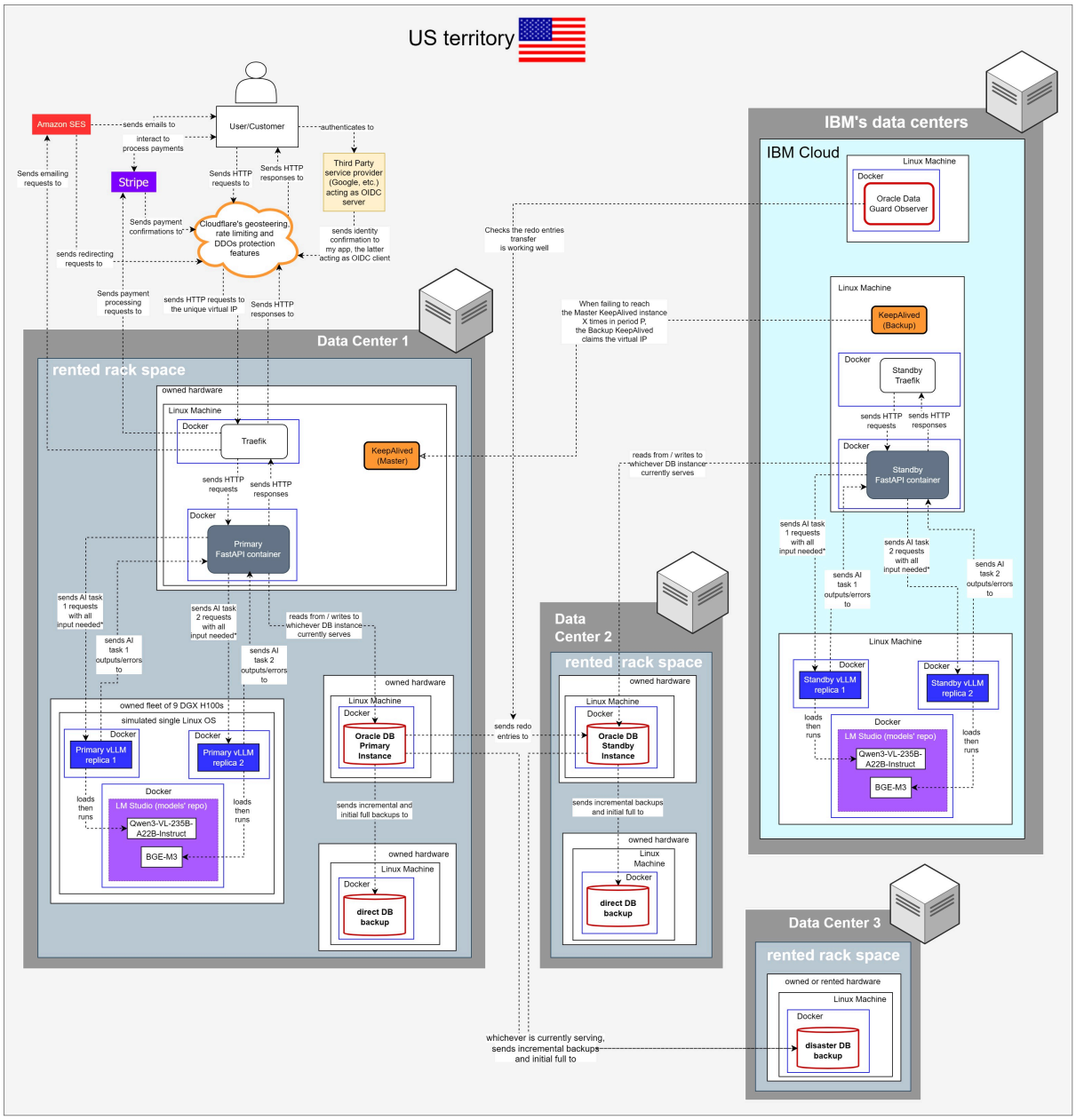
Note that Clean Architecture also contains a fourth layer called FRAMEWORKS & DRIVERS, but these are the elements such as FastAPI and Python-OracleDB driver, which are made of code, but which are not like the previous CA layers’ elements, in so that they are not part of bounded contexts, so they are not following a per-bounded-context tag system unlike elements in the previous layers, although they live in distinguishable files too [142].

Consequences: Now that we know the elements’ classification, I must determine what Pythonic form they will take and how they must work together to obtain high cohesion and low coupling. However, these are codebase concerns, therefore only mentioned again in Section 6.

5. C4 Deployment Context: the US deployment case

5.1. Diagram

Please zoom in to see the details.



* the FastAPI container serving always first queries the primary AI container, but if this one is not responding, it redirects its requests to the standby AI container instance.

Note: each ‘owned hardware’ element is distinct, meaning that multiple owned hardware elements in the same data centre are separate physical machines, communicating via wires. Consequently, the Linux machines on each of these hardware are different Linux machines.

5.2. ADR: OS

Date (dd/mm/yyyy): 01/04/2026.

Status: Accepted.

Context: My program must run on an OS (operating system). I need to choose which OS the program will run on.

Options: There are various providers, but the main ones are Mac OS, Windows, and Linux. And each has its different OS versions.

Decision: I choose a Linux OS.

Rationale:

Linux performs better on given hardware than alternatives. It is also very stable, secure, and open-source, unlike the alternatives, and exists in specific versions for Oracle DBs [206, 358].

The DB's block size must be equal or a multiple of the OS's block size [296]. Since my DB block size is 8kb, that gives 8kb, 4kb, 2kb, 1kb as OS block size options (keep in mind 8kb, 4kb, etc., are just shorter names for the exact size, which is always a multiple of 2).

Consequences: Components chosen must be compatible with Linux.

5.3. ADR: using Python-Oracle DB driver in Thick Mode

Date (dd/mm/yyyy): 10/11/2025

Status: Accepted.

Context: A domain code and a DB communicating need a clear interface to do so [203].

Options: The only one I found for Python app-Oracle DB communication is the Python-OracleDB driver [44]. It comes in two modes: Thin and Thick. Thick mode simply refers to using specific Oracle Client Python libraries that provide additional features [275].

Decision: I choose Thick Mode.

Rationale: It seems that Thick Mode adapts better to changing deployments (due to load-balancing or network-encryption features), which might come in handy for me. Thus, I choose Thick Mode, but note that this requires installing Oracle Instant Client [275].

Consequences: The driver lives within the FastAPI container, thus part of it in the diagrams.

5.4. ADR: user input format validation solution

Date (dd/mm/yyyy): 20/08/2026

Status: Accepted.

Context: To protect user accounts against attacks, I want to enforce a minimum password strength requirement that includes password strength analysis and blocking the saving of passwords that are not strong enough. Also, when users answer questionnaires or fill out name bars, etc., it usually must adhere to a given format (e.g., if the program asks for a number, the user must provide a number, not words).

Options: I can use either a front-end solution, or in-app solution, or DB solution.

Decision: besides the security features mentioned in Section 1.5., I choose front-end HTML validation for input constraining, and DB validation for final verification.

Rationale:

First, for password control, this solution provides the same eventual result as the others. But avoids a trip to the app. To enforce password constraints without a strength meter (the bars that show how strong a password is), an HTML frontend page suffices [227, 228].

For the answer bars format control, the challenge is that the next question is unpredictable from a program point of view. But since the program relies on sub-workflows and that in each sub-workflow, given types or one given type of answers are expected (e.g. sub-workflow X expects only numeric answers), there is still some general logical structure on which to base user input format control (e.g. when the code knows it is in sub-workflow X, it uses the numeric answer frontend template, which contains a "only numeric answer" HTML check upon whatever user writes in the input frontend cell).

The unpredictability is absent from the other user-input parts of the usage cycle (namely, payment, user registration, customer info entry, user project creation, or name edit). So these can use the frontend solution just like for the password creation/modification validation. I suspect Stripe to intervene in payment info filing checks [352].

The FastAPI container must eventually run its own check as well, because client-side validation in the mentioned solution can be bypassed [228]. This FastAPI-level validation, for all fields, including the password creation field, is done with Pydantic because it can validate data efficiently, including user input types, min-max lengths, and specific admin-defined character patterns [33, 122, 298–300, 364].

Consequences: This choice induces I require no additional C4 component for user input validation.

5.5. ADR: endpoint caching?

Date (dd/mm/yyyy): 20/08/2026

Status: Accepted.

Context: I try to optimise HTTP request-response latency. I came across endpoint caching as an optimisation method. Endpoint caching means caching the HTTP response [225]. It makes the response faster because the response is already prepared and waiting to be delivered, without needing to run FastAPI logic. I wonder if I should use it.

Options: I can use endpoint caching or not.

Decision: I choose to use no endpoint caching.

Rationale:

Endpoint caching means placing an endpoint and its response in a cache in front of FastAPI, so FastAPI does not need to generate the response when an HTTP request hits that endpoint. 90%+ of the requests are user- or user-project-specific. Each of these requests triggers a unique response both within and across user projects (unless a user cancels a step and redoes it identically, or unless two user projects are very similar). The remainder of the requests trigger similar responses across within/usage cycles because they are requests to display the website homepage or other universal non-user-specific pages. These latter HTTP responses (they are few in type, but of course, they are called often) could reach the user faster with a caching system, because caching them gains time on delivering them. But this would increase latency for all other requests because they lose a bit of time going through the HTTP caching system (acting as the first request processor). Considering this is a latency increase for 90+% of HTTP requests, using endpoint caching would increase program latency.

Consequences: I do not need a system in front of FastAPI to cache HTTP responses (e.g., Redis).

5.6. ADR: system failure management

Date (dd/mm/yyyy): 20/08/2026

Status: Accepted.

Context: When the system running the program shuts down or significantly slows down, I want users to continue enjoying the service despite the failure. That requires a specific backup plan.

Options: The backup can be a cold-start or an active backup program instance.

Decision: I opt for a cold-start backup of standby container instance with the standby DB being an Oracle Data Guard standby DB, and to that I add a disaster backup storage for the DB data.

Rationale:

Keeping multiple program instances active is costlier than just one active instance because of the baseline costs of running additional cloud VMs, or additional racks, and because it duplicates the instance-wide activities (health checkups, cybersecurity features) [147]. Moreover, restarting an entire program instance when only a single primary container does not work, would also be overkill, and due to the chosen hosting solutions (see Section 5.8.), would induce an energy-and-cost-inefficient usage of the available capacity (because the primary containers instances are rack hosted all year long, meaning as long as a primary container instance runs, starting to use its on-demand capacity cloud-hosted standby instance would increase cloud costs with no benefits).

For the DB specifically, a backup implies two instances of the DB. Also, for failures not requiring a switch to the standby DB, I want the primary and standby DB instances to each have a main backup locally on a specific disk, and a disaster backup in another physical location (in another data centre). Thus, in total, 3 backups, not 4. The standby DB, when serving (meaning the primary instance stopped working and the standby takes over), needs the same capacity as the serving primary DB instance, since it takes over all the work of the latter. And since I want high availability, with at most some minutes of downtime, the standby DB must swiftly start serving, thus promptly adopt the state the primary DB instance was in when it went down. We will see later that due to the very large size of the DB and write volume, writing everything from the backup to the standby would be too time-consuming, and probably as long as or longer than the time needed to troubleshoot the primary server and relaunch it. Therefore, the solution is to keep a standby copy doing the writes as the write orders arrive. In Oracle, this system is known as Oracle Data Guard, available in the Enterprise Editions of Oracle DB [259].

The great advantage of this solution is that, as long as the primary DB instance is working fine and its server is running, the primary DB instance can serve. This means that either the primary or the standby FastAPI container, whichever is currently serving, can use the primary DB instance directly. Alternatively, if there is an issue with the primary DB instance, the standby instance serves, no matter which FastAPI container instance currently serves [259].

Note: since, with this Data Guard feature, the standby DB instance constantly performs data writes or backups just like the primary instance, it is active. But it becomes the direct recipient of FastAPI data requests only upon a cold startup, hence called ‘dormant’ when the primary DB instance is serving.

Consequences: I must determine what tools are used to ensure this cold startup when needed. But I discuss this in the next ADR because it is also linked to load-balancing decisions.

5.7. ADR: mechanism used to redirect HTTP requests to the standby FastAPI instance

Date (dd/mm/yyyy): 20/08/2026

Status: Accepted.

Context: Users send requests to the FastAPI container, and I want to use a cold-startup version of the program that takes over in the event of a failure of the primary program instance. I need to determine which method/tool to use for this requests' redirection.

Options: I can use a load balancer or not, DNS/VRRP systems.

Decision: To redirect HTTP requests to the standby FastAPI instance, I do not use a load balancer for HTTP requests unless FastAPI requires processing capacity of multiple machines. Instead, I use a Keepalived failover mechanism. Due to capacity requirements observed in the next ADR, KeepAlived is the final choice.

Rationale:

Load balancers dispatch user requests to the right recipient [15]. User requests go only to my FastAPI container, which is a monolith. My FastAPI container seems to have no limits on the number of Uvicorn workers or on allocated capacity, according to online sources, and the program can run entirely on one machine. Also, Docker containers I will use know no limits in size [50, 109]. And we saw in Section 1.5 that Cloudflare geo-steers requests, which is the first 'load balancing' step. Thus, the program needs only one serving FastAPI instance unless FastAPI consumes the processing capacity of multiple machines. Therefore, besides for redirecting HTTP requests to the standby FastAPI instance, my program does not need HTTP request load balancer unless FastAPI requires processing capacity of multiple machines.

I use a standby program instance serving only upon cold start. That does not justify a load balancer redirecting requests to the backup, since I can reconfigure routing via KeepAlived, which is open-source, runs on Linux machines (good since that's the OS I selected), and uses the Virtual Router Redundancy Protocol (VRRP) for routing [196].

The DNS maps a domain name to an IP address, ensuring that the internet interface users use to access an online service points to the program that runs it [79]. Changing the IP address linked to a given domain in the DNS takes about the time to access the space to do that change, plus the time before the domain name links to the new IP address, and that can be very quick, a matter of a few minutes [16, 61]. This setup works but also requires a small website, in addition to the DNS, to display a fallback message to users, such as "the service is temporarily down for a few minutes". And it also leaves

some users in the dark until the new DNS update. So I prefer simply using VRRP, as it allows the domain to be linked to a virtual IP that can be paired with a new program in an instant [62].

Note that Cloudflare includes a cold backup failover mechanism via its load-balancing functionality [73]. And since geo-steering, which I use from Cloudflare, is part of Load Balancing, I already pay for load balancing. However, the fees for Cloudflare load balancing increase with the number of origins (i.e., in my case, the backend instances) and the number of requests [411]. If failover 'logic' is in Cloudflare, this implies Cloudflare must know the standby origin and the primary origin for each country. But if I use Keepalived, Cloudflare only needs to know one origin per country: Cloudflare always performs the same task (this request goes to this virtual IP) and never changes the virtual IP destinations, and KeepAlived, which acts as a watcher, sends dummy test requests to check reachability of the primary app instance, and if reachability fails X times, assigns the VIP to the backup deployment. So this reduces the Cloudflare bill. And since KeepAlived is free and performs failover just as well, it is a great option for reducing costs.

Consequences: KeepAlived must be in the C4 deployment diagram and I must discuss how I deploy it. I discuss in Section 1.2. which technology requests KeepAlived to change the recipient FastAPI instance.

5.8. ADR: hosting and isolation

Date (dd/mm/yyyy): 02/08/2026

Status: Accepted.

Context: I must determine where my program's components physically and logically live.

Options

- Multiple physical locations or a single one, abroad or in the US;
- Renting/owning hardware and machines;
- Cloud solutions: Oracle Cloud. AWS Cloud. Microsoft Azure. Google Cloud, etc ;[4]
- Serverless, serverful;
- Containerisation solutions (Docker, Kubernetes, etc.), virtualisation solutions.

[4] Note that FastAPICloud does not integrate Oracle DBs yet [123], but if it did, it could be a viable solution.

Decisions

- I deploy the primary instance of the DB (and its direct backup) and of the FastAPI and AI containers on my hardware on rented rack space in one data centre, each container on a different machine plus a machine specifically for the direct DB backup;
- I deploy the standby instances of the FastAPI and AI containers, KeepAlived and the Data Guard observer on Cloud. I need on cloud VM for the FastAPI container, one for the AI container, and one for the KeepAlived and observer.
- I deploy the standby DB instance and its direct backup on my hardware on rented rack space in a different data centre than the primary program instance's, and I again put the DB and its direct backup each on their own machine;
- I deploy the disaster DB backup on my hardware (if possible, otherwise on rented hardware of the data centre directly for security reasons) on rented rack space in another data centre than the primary program instance's and the standby program instance's.
- I containerise components which are on the same machine separately using Docker, except for the FastAPI container, which I containerise as a whole.

Rationale

I shortlist options in logical order based on deployment requirements. Although the domain is reusable, I assume it is only used to sell the FinApp1.0 service, as I want to know whether the service would be profitable when used this way (see "Consequences" subsection of this ADR).

Solutions retained based on legal and industry requirements

My software expects public and private clients. Private and public clientele require that I store all their usage cycle data in sovereign storage, i.e., local physical storage operated by local citizens [83, 136, 389]. Clients and security requirements (since similar for both) do not require the physical and logical separation of data between the country's public and private clientele [83]. Thus, I need the US-based deployment of all components so that data does not exit the US territory when stored or transiting between the user and the DB instances.

The deployment tools must comply with SOC 1 and SOC 2 (for the US market) and ISO 27001, all of which are compliance certifications that clients will demand [184]. The deployment tools must also comply with ISO 42001, as I, as system owner, will use an AI model fleet and monitor/fine-tune its nature and use (which is all within the scope of that ISO) [185]. The deployment tools must also comply with local and regional regulations. Notably, any potential cloud part of the US deployment requires FedRAMP's authorisation because of the nature of the customer/user project data my program uses [289].

Thus, I disqualify deployment options that do not comply with the above. The non-compliant options are those using service providers that do not comply, and deploying on my machines with loose security and transparency standards. However, this does not rule out a general method of deployment such as Cloud or on-prem etc., because each general option can comply.

Solutions retained based on installation/access requirements

I cannot install/access all the systems and containers how I want.

Stripe, Amazon SES, and third-party authentication are external services. Thus, I do not install them; my program accesses them via the web.

FastAPI is never a SaaS, as it is a web framework, not an app itself. Since I use it to build this app, the app itself is neither a SaaS I purchase. Therefore, the FastAPI container is only deployable on IaaS or PaaS Cloud solutions [407], or on capacity accessed via another network (vps, ssh), or on hardware I own/rent. KeepAlived must be installed on the same machine as the machine instance it wants to designate as the virtual IP where to direct HTTP requests to [143]. Therefore, I will deploy KeepAlived instances the way I deploy the Traefik/FastAPI container instances.

The AI container can be SaaS, since some remote AI inference providers (companies running models for people to access via API calls) commercialise BGE-M3 and Qwen3-VL-235B-A22B-Instruct inference [47, 166, 168], and I can deploy it on PaaS, IaaS, or on hardware I own/rent.

As evidenced later when discussing prices, my Oracle DB instances are deployable on IaaS and PaaS Cloud solutions, on capacity accessed via another network (vps, ssh), or on hardware I own/rent, or accessible as a SaaS.

I can containerise all of the elements [106, 248, 278, 315].

I plan for peak usage, not for scaling up later. Thus, the criteria for selecting among these solutions are reliability (performance and security), user traffic patterns (how many requests/s, when), capacity limits, processing speed, debuggability, the level of control I need, and price. Let’s discuss them all.

Solutions retained based on backup plan and app’s statelessness

The app is stateless (see Section 2.2.1), HTTP request-driven and monolithic. I can deploy such app either via (a) server(s) that I configure and/or maintain (serverful deployment), or that I don’t configure nor maintain (serverless deployment, also known as FaaS) [202, 238], which is what the deployment options retained so far enable. My chosen failover system (cold-start backup) can exist in either type of deployment retained so far as well, on the condition that the backup elements live separately from the primary instance, so that if the primary instance’s hardware goes down/is hacked, the standby elements (as well as the monitoring elements, i.e., the data Guard observer and

KeepAlived) are not impacted, can run, and can enforce the cold-start of the backup as well as the redirection of HTTP requests.

Next, I must determine the solutions eligible based on capacity requirements. For that, I must first estimate containers' (C4 containers) capacity consumption (next subsection). Note, although potential virtualisation/containerisation will consume capacity too, I expect these layers' capacity requirements to be negligible compared to the rest of the program. Thus, I discuss my virtualisation/containerisation choices later.

Capacity consumption estimates by program container

As explained in the introduction, I don't know the eventual capacity requirements due to potential code evolutions and lack of self-benchmarking. When I lack an exact metric to calculate capacity, I use an arbitrary value, or derive it from third-party projects or technology knowledge and a forecast I made of the number of concurrent usage cycles assuming an as-monolith-single-commercial-usage deployment. Note I discuss only the results of the forecast here.

In the calculations, I also often round up results to the next higher decimal place to accentuate number inflation. Also, I give each component 30% headroom capacity, the capacity listed for each container should be all the program needs including OS's capacity consumption.

I will often use the terms "serving"/"non-serving or "dormant". 'Serving' designates an element of a program when it actively delivers the service to the user, whether as the sole or co-deliverer of that service (e.g. a DB performing tasks co-delivers a service with the FastAPI app and other services). Oppositely, a "dormant" or "non-serving" element is part of the architecture, can do some constant work of its own (e.g. a DB mirror instance which also does some writes), but which does not directly provide the service to the users (in this example, the primary instance of the DB is serving, and the mirror copy is non-serving).

Capacity planning is iterative; adjusting the performance or latencies of a given component changes the capacity needs for components communicating with the former, because the latter then wait longer or shorter to work, and thus holding out request/response/retry info for longer/shorter, thus changing RAM, VRAM capacity at the very least, but can also change CPU or GPU usage or disk storage, to respect a general latency goal for example. I however ignore the relationships between DB capacity and AI container capacity due to time constraints.

Expected service demand

I here discus, based on my forecast, the duration of a usage cycle, times of usage, the number of HTTP requests expected, and the number of concurrent users and new usage cycles per day.

Duration of a usage cycle

A usage cycle is the entire set of requests made by the same customer (a customer is a logical group and domain value object of one or more users sending HTTP requests as part of the same usage cycle) that are performed to benefit from all the app's features necessary to receive the service 1x in its integrity. Based on my forecast, **a usage cycle lasts on average 1 workday**. This is largely above what most cases last, but some cases will be at least or bigger than this, and I prefer to inflate than estimate too low/tight.

Times of usage and impact on capacity calculation

Users mostly work 9-to-5 or 8-to-6. In the case of the US, if we have a single place where the primary instance of the program resides, and we set the program's time to that time zone's, because of the US time zone landscape (7 hours of difference from most east to most west), the program does not experience high traffic only from 9-5 where it is located, but more from 6 to 8 pm of 5 am to 9 pm for example (if located in GMT -7). But because US time zones imply that there are multiple hours each day all US clients work at the same time, I prefer considering capacity needed as if all clients used the service at the exact time, thus assuming an 8-hours peak activity every workday.

Also, I expect usage cycles to evenly spread out over the work week, and definitely spread out within each month and year. Only exceptionally will a week's worth of traffic comes the same day. Thus, to plan usage-cycle-related data storage for more than a week, it is best to use the daily average and to plan processing capacity, it is better to use the request count on a day worth a week of traffic.

HTTP requests per second per million inhabitants

Based on my forecast, we have an average of 76.48 concurrent users per million inhabitants if the load is evenly distributed throughout the week.

90+% of the activity is financial case info fetching, and I know the average number of financial case info to fetch per usage cycle (which I slightly inflated so capacity is large enough). This info-fetching requires asking the user questions only when an AI task 2 process fails or if task 1 is impossible/the user did not upload case documents. Therefore, if the number of processes P1 decreases per HTTP request, the number of HTTP requests per usage cycle increases. I thus had to select the number of requests per usage cycle and, consequently, spread the expected number of processes (indexed by the number of use-case loops) across these requests. For 90+% of the info fetching job, the AI container should generally find all it needs, thereby requiring few user interventions, and thus more "*ask Ai task 2's model to find this info*" loops per HTTP request. But inflating the number of requests per user project gives capacity headroom for the variable API's work (intercepting HTTP requests/sending HTTP responses, depending on user-input documents' quality). Thus, I inflate the number of

requests/s to on average 230.4 requests per day per user (the number of requests for 1 usage cycle, which usually lasts one workday max), thus 17,620.856 requests a day/million inhabitants.

Divided by the number of seconds in an 8-hour workday, we find an average of 17,620.856 ÷ 28800= **0.61184 HTTP requests/s/million inhabitants**.

Earlier, we saw that the maximum number of concurrent users is the average number times 5. Thus, if all concurrent users for one week start a usage cycle on the same day, the maximum number of requests is 0.61184×5=3.0592 HTTP requests/s/million inhabitants. On off days and on all nights, it drops near 0.

Concurrent users, and new usage cycles per day per million inhabitants

In my forecast, there are 76.48 new usage cycles per million inhabitants per workday.

Since usage cycles last on average 1 day, there will likely be **76.48 concurrent usage cycles, and the same number of users**[5] **during working hours on weekdays**, assuming the pool of users starts their usage cycles at a constant daily frequency during the week. But that's not the maximum. Indeed, users work only on one usage cycle at a time. They usually complete at least one usage cycle per week due to the imperatives of their work schedule. The above number of HTTP requests assumes the week's pool of users' usage cycles start evenly spaced throughout the workweek. Since they can start their minimum 1 usage cycle on any of the 5 days in the workweek, they could all start and complete 1 usage cycle at the same time of the week. That would amount to a maximum of 76.48 × 5 = 382.4 concurrent usage cycles and the same number of concurrent users for a million inhabitants on the same day (thus, an equal number of connections). However, the traffic is expected to be spread out over the 5 days of the week rather equally.

Users and usage cycles drop near 0 outside work hours.

Needed capacity of the DB, its backup space and a standby db

I must determine the peak CPU usage, peak read I/O and write I/O, peak connection counts, and storage growth rate to estimate the capacity my DB needs [191, 365]. I discuss all that can help determining them, then I determine them, and then I size the DB.

One can estimate some sizing data can only from historical data in the Automatic Workload Repository (AWR) Report for a DB. I don't have my DB yet running, thus no AWR Report. Thus, I asked an Oracle DBA I know for one of his DBs' AWR Reports and compared them with data I already have about my DB's activity to estimate missing values, such as redo writes and physical writes/sec. Obviously, this is an approximate comparison because the DB can size redo entries

[5] I use the term user instead of customer, because whether or not a customer includes one or multiple users, only one user of the group is intervening for one usage cycle at a time, so I can use the term user here interchangeably

differently across projects, resulting in different redo and physical write frequencies. But it is still informative in some aspects. I look at his DB's two 19-Jul-26 05:00-19-Jul-26 06:00 instances. Why two and not just one? Because some data I need for my comparisons (number of transactions) is not divided per instance in the report but is cluster-wide (because group 0 and group 1 LGWRs are not mapped to a single instance, but to a single worker, which can work for either instance) [260].

Since I use Oracle data Guard, the DB capacity I calculate includes the capacity for the serving DB, at least two identical backup data storage spaces per DB (and the storage process to these spaces), and a standby DB instance. And Data Guard requires the standby DB instance to have at all times, the capacity to perform the writes based on the archived redo logs the primary DB sends, and the capacity to feed its own direct backup storage [259].

Disk type

I use SSD storage for the in serving mode DB and the standby for its speed advantages and environmental resilience (notably resistance to temperature changes and extremes) over HDDs. For the backups, as long as I have sufficient write bandwidth, I can opt for HDDs because, although slower, they are much cheaper than SSDs [90, 96, 338].

Optimisation

My queries contain one or more of the following: equality lookups, range lookups, prefix matching, sorting operations using ORDER BYs, JOIN operations between large tables on indexed columns, filters on multiple fields. All these operations execute faster and with fewer IOPS when using b-tree indexing (a sorting method). Since I can use that indexing in Oracle and need to optimise query speed, I will use it [137, 261].

Also, sometimes the database stops using indexes when it evaluates that using the indexing (with the subsequent random read triggered) is more bytes-costly than a direct sequential read, assuming the program runs on an HDD (on which, even when the indexed random read is way faster than the direct sequential read and if the program runs on an SSD) [145, 314]. To ensure the DB does not lose speed by entering what it sees as cost-efficient direct reads (which are eventually much longer), I write my queries using FORCE_INDEX and PREFER_INDEX, or dbms_SQLdiag_internal.i_create_SQL_patch [262, 351]. Random reads consume RAM. Thus, the more random reads I want, the more RAM I need [285]. Also, I need to enforce a KEEP pool (to retain important small indexes/blocks that large queries could push out of the cache otherwise), and optimise clusters [285]. Note that I index the tables upon which the users' requests only trigger reads. This is because, in the tables that user requests trigger writes to, indexes can significantly slow performance, since each write incurs indexing overhead, whereas reads do not [372].

I also optimise the size of my cache, including the redo log buffer cache, so it is not a bottleneck.

Finally, I pick the best database block size. Block size tuning can introduce problems with design adaptation when applied to an already working program and DB, but seems to have a positive impact when set up from the start [363]. There are 5 options available in Oracle: 2k, 4k, 8k, 16k, and 32k bytes [263]. Bigger blocks mean more data is read in fewer I/O operations (1 block read is one i/o), and that's highly favourable for read-heavy programs [312]. On the other hand, since each block gets 2 transaction slots, and the higher the number of slots, the more writes can happen concurrently, and the smaller the blocks, the more blocks there are (if all other things being equal), smaller blocks are best for write-heavy DBs [312]. Therefore, dividing the DB into 2 tablespaces: one for write-heavy tables with one block size, and one for read-heavy tables with another block size, could increase performance. Some of my tables are really only queried by the program, so this could work, and Oracle allows such a double-sizing system [374]. But I could not find definitive sources stating exactly what problems it would bring (maybe double maintenance issues, etc.). Therefore, in doubt, I abstain from a double system.

My tables have a row size of 1kb or less. This eliminates the need to choose a high block size just so that one row can fit in a single block, because a 1kb row will always fit even in the smallest block. And I use optimised indexing on the read-heavy tables to ensure efficient random reads. These two factors push for a small block size, but not so small that my usual number of rows pulled (4+) requires too many blocks [264]. The in serving mode DB expects 1.32078125 loops of write transactions per use case loop, and 4.945775 read loops per use case loop, for the main use case (see Appendix 1), whose executions represent 80% of the program's use case executions. The writes are usually 90% altering, inserting, or deleting 1 row, but the rest of the time they easily go over 4 rows. Also, since the DB's block size must be equal to or a multiple of the OS's block size [296], picking something like an 8,192b block size for the DB gives more OS options and gives enough space to write 6 or 7 rows (considering there is always some overhead). Therefore, in theory, 8192b, the recommended block size, is ideal. I might revise after subsequent tests.

Nature of queries, number of writes and number of 'user write' commits

In my program, queries are lightly joined with a small-to-moderate row count per call. The row count is one user project's or one sub-workflow instance's worth of instances/inputs/outputs ; my usual get query searches all rows from a given table, whose value in column c matches the value in columns d and e from another table whose id=x. And a "logical" (in the sense of what the use case ordered) write writes one row at a time, taking ~7–10 values from other tables, dates, and (for the inserts) generating a unique integer PK. A read can query 100 blocks and usually return 10 rows or fewer.

Reads and writes either load data into the cache (also known as a buffer) and read/write in the cache, or bypass the cache and read/write directly on/from disk [335]. In my case, all of the DB's working set is eligible/likely to get loaded in cache for reads and writes (as much as cache size allows), because

data blocks of the working set are accessed a lot of times and usually represent not many (less than 200) rows, and so the DB deems it more resource efficient to load it all in cache [335]. Even when a DB is fully eligible to be loaded into its cache, the DB will sometimes perform reads and writes directly to/from disk, bypassing the cache, when it deems it more resource-efficient [43]. One cannot constrain the DB to never bypass the cache. But there are ways to incite the DB to always do so. The main way is to keep a cache large enough to host the entire working set. An additional way is smart indexing. With modern SSDs, if indexing eliminates many rows and selects only 100 or fewer, the DB considers it more resource-efficient to load the remaining rows into cache. But if, despite the indexing, the eventual reduced selection is tens/hundreds of thousands of rows, the DB bypasses the cache to find info it wants [145]. Now, this indexing strategy is even more performant if we avoid large table scans in the SQL text itself: asking for something small that can follow the index path will avoid them. Finally, I can use the *cache* attribute for tables which I want to be always loaded in cache. Also, it's important to avoid direct-write SQL.

I will very likely manage to avoid direct reads and writes. Direct reads/writes comprise some direct plain and some temp reads/writes [3]. The temp ones result from using temp space due to insufficient RAM to perform certain operations (especially sorts, hash joins, ORRDER BY clauses, GROUP BY clauses, and temp index creation) [3, 276, 396]. I don't have hash joins, because these are valuable on unindexed data, whereas I use indexing [141]. I don't have temporary indexes because I already have fixed indexes. And the other operations can be memory-intensive, but I aim to have enough memory to host them. So I avoid temp direct reads/writes. And I avoid the other direct reads/writes with the optimisations evoked earlier in this paragraph. But later, we will see that for certain sizing purposes, we cannot completely rule out direct reads and writes. **For the remainder of this ADR, I specify cache reads/writes as the total number of reads and writes in my program, assuming I can almost always avoid cache bypassing; for unavoidable bypasses, I will remind the reader of potential bypasses**.

My app's workflow is 90% centred around the execution of a given use case dubbed main use case, which contains, on average, 1.32078125 loops of transactions (transaction in the Oracle sense of the term, i.e., DML [265]) (when a user commits one or a series of transactions at once) per use case loop (see Appendix 1). The use case has 20 loops per HTTP request,[6] and each transaction loop contains a maximum of 100 1-row writes. Thus, there are 1.32×20 = 26.4 separate transaction loops per HTTP request, at least. Now, since 26.4 does not include the transaction loops of other use cases, which could execute aside the main use case, and which have around 15 transaction loops altogether max per HTTP request (1.5 transaction loop/use case loop × 10 use case loops) (here we can use 10 loops max, because it does not amount to the shortening of the logic executed), we have **around 39.6**

[6] This is arbitrary, based on the fact that each usage cycle expects a steady workload that I spread across an arbitrary number of HTTP requests (see earlier in the HTTP requests count subsection).

transactions loops per HTTP request on average, which I round up to 40. Each transaction loop contains a maximum of 100 1-row writes. So that raises our **total number of 1-row writes per HTTP request** to 40 × 100 **= 4,000**. Knowing we have a maximum of 3.0592 HTTP requests/s/mill. inhab., we should have a maximum of 3.0592×4,000 = **12,236.8 one-row writes/s/mill. inhab.** (**734,208/min**).

Because there is only one commit per use case loop, and on average 30 use case loops per HTTP request (with the main use case + satellite ones, or just the satellite ones), we have around 30 user commits per HTTP request, reaching a maximum of 3.0592×30= **91.776 user commits/s/mill. inhab**.

Peak datafile reads

In Oracle, reads include logical reads (reads of cached blocks) and physical reads (reads of non-cached blocks). Logical reads are the DB block gets (read from the buffer) and the consistent gets (read from a process private memory) [3].

Accessing data from the buffer cache requires, first, if not yet in cache, a physical read that loads the block into cache, then one DB block get for each index it uses to reach the block (if any), and then a DB block get for the block of interest [35, 42].

Based on the main use case's probabilistic tree (Appendix 1), the main use case triggers, on average, 4.945775 datafile read loops[7] per use case loop[8], thus around 4.95×20 use case loops=99 datafile read loops per HTTP request for the main use case. If we add around 30 datafile reads at least for the remainder of the response process beyond that main use case, we have around 130 read loops per HTTP request, which I round up to 150 to be safe. Times our 3.0592 max HTTP requests per sec per million inhabitants, we have a maximum of 458.88 datafile read loops/s/million inhabitants. Each loop comprises searches for on average 25 rows. We know these rows are not necessarily in data blocks containing only them, because they are written wherever there is space in their target table blocks at the time of redaction. But some will likely be grouped, so I imagine we would have around 12.5 data blocks per read loop. This makes for a total of 458.88 ×12.5= 5,736 end data blocks (as in 'not the index blocks') targeted by SELECTs/s/million inhabitants. To that, we add that the DB has, on average, 3 indexes to go through before reaching its data block of interest, and knowing that all I considered as reads are DB block gets, I would not have any consistent gets, we obtain 5,736+ 3×5,736= **22,944 total number of logical reads/s/million inhab**.

I add the physical reads, which load the end and index blocks into the buffer cache. Their number is rather low, because most data is either loaded once for everyone (when the table stores rules), or once for a user project but only in small quantities, because usually the resumed or the started user project

[7] e.g., "for each …, read…", but sometimes there is no "for each", just "read..", usually at the beginning of the use case loop.
[8] The execution of a use case before it loops back to a previous step of that same use case

has few rows to (re)load, then does inserts which make for new rows. So it is likely 20 rows per each of the max 382.4 concurrent usage cycles per day, plus 10,000 rows for each of the 35 rules tables, all that divided by the number of seconds in a workday (28,800), divided by the potential number of these rows found in the same data block (because the read loads the block in cache, not just the row) equalling (20×382.4+35×10,000) ÷ 28,800 ÷ 2= **6.20917 physical reads/s/million inhabitants**.

My block size is 8192b, with <1kb rows. Oracle leaves 20% of a block empty for elements other than rows [266]. We thus have 6.20917×8,192b= **50.9kB/s/mill. inhab. of physical read size, and 22,944×8,192b= 187.957248 MB/s/mill. inhab. of logical read size.**

The US deployment, with 350 million inhabitants, **thus expects** 187.957248 MB×350= **65.7850368 GB/s of logical reads for** 22,944×350= **8,030,400 logical reads/s, and 50.9kB×350= 17.815 MB/s of physical read size (which is thus the disk read throughput) for 6.20917×350=2,173.2095 physical reads/s.**

Peak data file random writes (physical writes)

DML transactions can persist physically either directly to disk, or via the buffer cache [3]. In the latter case, DML changes are made to copies of database blocks in the **buffer cache, making them 'dirty blocks'** [319]. DBWn later physically persists dirty blocks to disk, either one or more per physical write, within the limits of what the OS allows [35].

In my case, I want physical writes to never bypass the buffer cache. So all my writes are what we find under the AWR Report under "physical writes from cache" [3].

An online source reports that a DB had a physical write throughput of 141.7 MB/sec, with 18,135.2 blocks/sec and 15,096.9 I/O write requests per second [132]. The number of block changes/sec is 120.13% of the number of i/o write requests/sec (15,096.9×100 ÷ 18,135.2= 83.26). This suggests that some physical write requests led to more than one block change, which is consistent with the fact that a single physical write can persist one or more dirty blocks. Also, with the probable 8192b block size in that source, we get 18,135.2×8192b = 148.564MB, which is almost the execution size of 141.7MB/s of writes. This suggests that throughput is the number of block changes of a given size the program can sustain in a given period. So the physical write throughput is DB's block changes/s × block size. This substantially eases physical write size prediction, because no matter how many blocks persist per physical write, the size of writes is the size of the blocks they persist.

In my DB, logical writes are 1-row each (this is due to FastAPI's work), but each write transaction loop affects the same buffer cache blocks (as long as the original inserts of these target rows could populate blocks that had multiple rows available), not one block per row of the transaction loop. Due to block overhead and the 8kb data block size I selected, my blocks contain 6 or 7 rows of data (I use 6 rows to avoid under-sizing). The DB triggers physical writes when a certain percentage/number of

buffers (a buffer is the cache container of one block) are dirty [35]. Inserted data rows are placed into data blocks with available space. Thus, a single insert of X rows into blocks of Y rows will require at least X/Y blocks to be populated, but it could require more blocks when filling some blocks that were already partially filled from previous transactions. My writes per request do not thus necessarily affect the entirety of each block it touches, but the entire daily activity; the writes are condensed into blocks fully modified on each row or near that. My blocks are each 6 rows. Thus, the **maximum number of physical to-write blocks/s/million inhab.**= 1-row transactions per second ÷ 6 = 12,236.8 1-row writes/s/million inhabitants ÷ 6 **=2,039.5**, for a total of 8,192b×2,039.5**=16.707584MB physical write size/s/mill. inhab**. per DB, **and that is without the archlog writes (which I discuss later)**.

So how much physical write IOPS is that? As noted earlier, the DB might group to-be-written dirty blocks into a single physical write I/O [69, 320]. The criterion(a) for grouping is undocumented. Thus, I must assume that, **at most, for my DB, the number of physical writes equals the number of blocks to be written**. This is probably an overestimation, but it 'balances' to an extent the potential underestimation of the number of blocks to write.

For **the US deployment, that amounts to 2,039.5 × 350 = 713,825 physical write blocks/s, for a physical write size of 16.373 GB/s, for a max of 713,825 physical writes/s (physical write iops), and that is without the archlog writes (which I discuss later)**.

Peak redo write size

Redo writes are the physical persistence, by the LGWR, of the redo log entries found in the redo log buffer, to the online redo log files (which are called online files although they are not online but on disk). The LGWR writes redos at least every 3 seconds. The number of redo writes the LGWR performs per second is called the redo write size and is determined by the machine's capacity and whether there is space in the online redo log files to write redo entries [3, 25].

We have a maximum of 734,208 one-row writes/min/mill. inhab. hitting the redo buffer (see precedent earlier). A 1-row transaction usually changes around 4+ blocks (the target table block, the undo block, the block of the transaction table for the undo segment, and the index leaf block at the very end of the index tree branch), and all that fits into 1 redo entry: so in my case, one 1-row write equals roughly one redo entry [1, 208]. Redo entries are not fixed in size [1]. But I noticed that for simple-to-medium writes in my contact's report and online sources, a 1-row transaction usually requires a 500B+ redo entry. So I estimate it at 600 bytes just to be large. So I would have around 600 B ×734,208= **440.5248 MB of redo entries/min/mill. inhab. (7.34208 MB/s/mill. inhab.)**. I want a DB backup space per DB instance on a separate local disk, and a disaster backup in a specialised data centre. Each backup receives archlogs from the in serving mode DB at all times, as well as full and incremental block backups. When the primary instance of the DB is in serving mode, the Data Guard standby DB receives redo entries in real time. And for each deployment, I also have a disaster-recovery backup

space that receives writes. So my number of concurrent writes is 4 for the primary DB, and 2 or 3 for the standby DB, which raises the total needed redo write bandwidth of the in serving mode LGWR to 7.34208 MB/s×4 **= 29.36832 MB/s/million inhabitants for the primary DB, and for the Data Guard, when it takes over (since it might also write to the backup)** 7.34208 MB/s × 3**=22.02624 Mb/s/mill. inhab. For the US deployment**, that's respectively 29.36832 MB×350**=10.278912 GB/s** for the primary DB, and 22.02624 MB ×350= **7.709184 GB/s of redo size for the standby DB (that's when it runs as the main; it's two thirds of that when it is in standby mode)**.

Online redo log files, their size, and redo archive space size

Let's first discuss how transaction logging and consistency work, to understand why redo log files and redo archive space matter.

First, as noted in the previous subsection, the disk contains 'online redo log files', which store redo entries from the log buffer. Ideally, I choose to archive redo log files because the DB system can check in the archives the earlier write schedule and if this schedule fully completed. In a DB with log archiving, a redo log file fills up, then when full, its contents are archived and the writes it notifies are written, and only then the file is emptied and available for reuse (for new redo writes). Therefore, a DB with redo log archiving must have at least 2 online redo log files [267].

Also, a log group is one instance of the same redo log process (the writing of given redo entries). In my case, I have primary DB and a standby DB, on different disks. Each must contain the same online redo log files, since I want the backup to take over immediately in the event of a failure. Each DB must contain at least two redo log files. Also, it seems that groups are not designed for splitting logs between the serving and dormant Db instances: the serving mode DB has its own groups and members, and the standby or backup has its groups and members. Thus, I use a minimum of two log groups, with each group having at least 2 members on the in serving mode DB's disks, and the same config on the standby DB [267, 268, 311].

Thus, on each disk, I must set the size of the online redo log files, their number, and how many redo files the archive space holds at a time.

The redo archive space stores archived redo log files until they become obsolete. The company using the DB determines what obsolescence means [267]. In my case, "obsolete" means the backup space has received the full DB snapshot. Since I want a snapshot every 1 hour, I need enough archive space to store one hour's worth of redo archive. Thus, ARCn and DBWR must together persist the physical writes and redo log file 'R1' as fast as LGWR fills the online redo log file, which precedes R1 in the LGWR queue.

Archiving (ARCn) and physical writes (DBWR) occur simultaneously [269]. In my case, they both write to the same disk (the in-serving mode DB's disk and the backup DB's disk). One disk storage

expects 16.707584 MB of physical write size/s/mill. inhab., for 7.34208 MB/s/mill. inhab of redo log entries. So ARCn and DBWR together take 24.049664 MB/s/mill. inhab. of disk write throughput per storage destination, while LGWR (which also writes to disk, since online redo log files are on disk) uses 7.34208 MB/s/mill. inhab. of write throughput per storage destination. **For my US deployment**, that requires **each storage space to have a write throughput of 8.4173824 GB/s for ARCn + DBWR, and 2.569728 GB/s for LGWR, therefore 8.4173824 GB + 2.569728 GB = 10.99GB total write throughput, which we can round to 11 GB/s)**. SSDs write up to 12GB/s [181]. Therefore, I will need to split each DB instance onto multiple disks (using a process called ASM stripping: one DB, multiple storage disks logically grouped as an ASM disk [413]). But that also means I will meet the write speed.

Changing the redo log files' size does not affect this throughput requirement, because a single write does not become heavier/lighter as the redo log file size grows/shrinks. But what size to pick? A checkpoint occurs when ARCn finishes archiving a redo log file. "A checkpoint can momentarily reduce runtime performance" [270, 271]. Thus, more archiving processes create more checkpoints, which slows overall performance. However, with smaller redo log files, the files stay less time in the archive, making for a smaller archive overall. Considering that on restart for recovery, the DB checks archived redo logs to the write schedule completed, the bigger the archive the slower the restart. In other words, smaller redo log files can mean faster DB restart. Thus, Oracle recommends a log file size that allows a log switch at most every 20 minutes, with a maximum of a few GB and a minimum of 4 MB [272].

If my DB switched log files every 20 minutes with a redo write size of 7.34208 MB/s/mill. inhab., one redo log file would be 7.34208 MB×60seconds×20minutes = 8.810496 GB/mill. inhab, so 8.810496 GB × 350 = 3.0836736 TB for my US deployment. That is in the "few GB" recommendation. Therefore, I set the redo log file size with a little overhead to **4GB**. And to have some LGW free space in case writing and archiving slows down for any reason compared to how fast LGWR fills up redo log files, I select **8 online redo log files per DB**; 4 log groups of 2 members, all part of the in-serving mode DB, and 4 groups of 2 members each in the standby DB (that's a total of 8×4GB= **32GB per DB**). I want a full DB snapshot backed up to my backup storage every hour. Therefore, I need to archive an hour's worth of redo entries. 20 minutes' worth of redo entries for my US deployment is 3.0836736 TB. Thus, I need **redo archive space of** 3.0836736 ×3= **9.2510208 TB per DB instance**.

DB Physical disk storage space size

For the DB, the disk (in my case, multiple disks, as mentioned in the previous subsection on ASM stripping) stores tables and DB work elements. Let's discuss the total disk size needed for the in serving mode DB, and note that the backup DB's disk is of the same size.

I have 71 business tables. 11 are of fixed size, 24 are of evolutive size but are the same size no matter the deployment and the number of users (mainly some law-related tables, but they do not change in size drastically over time; they just require a very large sizing), and 35 are of evolutive size depending on the number of users.

Each of the first 35 tables expects 400,000 rows of 500 bytes each. That's 35×400,000×500 bytes = 7GB of raw data, and that's per deployment, not per million inhabitants. With 20% of the table size worth of data block overhead [266], and 100% of the table size in indexes (very likely in my case, because they are read-heavy tables, except when admin commits), and at least 20% of the undo tablespace, which totals **16.8 GB per deployment**.

27 other tables have each up to a thousand < 1 KB rows on average per user project I want to hold info of at a time. 1 other table has 5 <1kb rows per user project whose info I hold at a time. 5 other tables will have up to 20 1kb rows per user project whose info I hold at a time. 1 other table has up to 50 <1kb rows per user project whose info I hold at a time. The number of user projects whose info I hold at a time is the sum of the uncompleted user projects from the last 3 months (arbitrarily chosen duration) and the completed ones from the last 7 workdays. The completed ones represent a workweek's worth of user projects, which is equal to the maximum number of usage cycles in a workweek, i.e., 76.48 new usage cycles per day, times 7 (since a usage cycle takes on average 1 workday to complete, so those completed from D-8 until D-1 are those finished in the last 7 workdays), so around 535.36. In 3 months, at most around 2,000 user projects would be uncompleted per million inhabitants (an arbitrary estimate). So these tables store info for 2,500 user projects. Together they thus represent 2,500×(27×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)= 67.8875GB per million inhabitants. With 20% of the table size worth of block overhead [266], and 40% of the table size in indexes (fewer indexes here because these are write-heavy tables, and, as explained in the 'Optimisation' subsection earlier, indexes in write-heavy tables slow down performance), and at least 20% of the undo tablespace, which makes for **122.2GB per million inhabitants**.

1 and last other table has 1 <1kb row per customer I want to hold info for at a time, and another with 10 <1kb rows per customer I want to hold info for at a time. For a 1-million-inhabitant population, we expect at most 76.48 (new usage cycles per million inhabitants per workday; one new usage cycle can mean no new customer) times 261 (number of workdays per year) customers per year, i.e., 261×76.48 = 19,961.28 customers max per year. This seems large enough to hold and should stay rather steady over time, because the customer pool does not grow and is expected to use the service every year. So I retain for an equivalent of 25,000 customers. That amounts to 25,000×(10+1)×1kb = 275MB per million inhabitants. With 20% of the table size worth of block overhead [266], and 40% of the table size in indexes (fewer indexes here because these are write-heavy tables, and, as explained in the

‘Optimisation’ subsection, indexes in write-heavy tables slow down performance), and at least 20% of the undo tablespace, which totals **495 MB per million inhabitants**.

Finally, we add space for the index blocks, which store information about the indexes to be enforced on a given table.

Each index block has a fixed size, just like the other data blocks (in my case, 8,192b). But the number of index blocks is fixed by three factors: the number of indexes, the number of rows per table, and the data block size. An index block stores rows, each containing an index key, a row ID, and the related value, which is the subject of that index. There are as many index blocks as needed to store the value (that matters for a given index) of a given row. An 8,192-byte block uses 80% of its capacity to store this index info, so 6553.6 bytes. Each index entry is likely around 50 bytes max. So each index block contains around 131 index entries. My tables usually have a maximum of 3 indexes. So for 131 rows of a table, one can count 3 index blocks. I have 75 tables. Among these, 35 tables will contain 400,000 rows each. So these 35 tables require (400,000×35) ÷ 131×3= **320,611 index block-equivalents per deployment (320,611×8,192bytes=2.62645 GB/deployment)**. Then, 34 other tables require 2,500 user projects. Together, they thus represent 2,500×(27×1,000 + 1×5 + 5×20 + 1×50) ÷ 131×3= **1,554,676 index blocks/million inhab (12.7359058GB/million inhab)**. Then the last 2 other tables expect 25,000×(10+1) ÷ 131×3= **6,298 index blocks/million. inhab. (51.6MB/million inhab.)**.

For the US deployment, that’s a total of 16.8 GB + 122.2 GB ×350 + 495 MB × 350 + 2.62645 GB + 12.7359058 GB ×350 + 51.6 MB ×350 = 47.4383035 TB disk space for the table blocks and index blocks.

To that, we add capacity for the online redo log files (**32 GB per DB for the US deployment) and their archives held in both the in serving mode and backup DB’s disks until deemed obsolete (9.2510208** TB **of space per DB for the US deployment)**.

There is also size needed for temp, measured by the size and number of physical reads to and writes to the direct temporary tablespace [3]. Sorting, joining, aggregation, temporary index creation, or other operations which require more memory than available, trigger the creation of temporary tablespaces (or the use of the default created one) to store intermediate results [276, 396]. As indicated earlier, I want my DB to have enough RAM to run all its current-session work in memory. And thus temp would be non-existent. However, I want to reserve some disk space for temp in case direct reads and writes occur anyway (due to a cache issue or otherwise). Let’s see how I estimate it.

My contact’s DB had 45 physical writes to direct temporary buffers out of 49 physical writes direct (see Appendix 2): that’s 91.8% of the direct writes. It made 45 direct physical writes out of the 10,518 it had to make during the session of interest: that’s less than 0.43% of direct physical writes. The least executed (2 SQL texts) were executed fewer than 49 times together, and there was only one other SQL write text. Writes did not represent a minority of the scripts; the former is logically not present

because that DB (just like mine) had almost no physical reads. The size of these buffers was 8,192 bytes each, because their size equals the size of the data blocks (that's what Oracle internal AI services told my contact), and the data blocks were 8,192b (see Appendix 2). So for 2.92/s, my contact's DB had a temp of 45×8192b=368.64kB. But we don't know what 45 temp. buffers meant in terms of how many blocks were written. Since I have 2,039.5 blocks physically written/s/mill. inhab., I need a temp of (2,039.5 ÷ 2.92) ×368.64kB= 257.47989 MB/mill. inhab., thus, for my US deployment, 257.47989 MB×350= 90.118 GB. This is honestly a bit small, because my contact's DB had so few direct writes. I'd rather oversize storage and reduce it later. And we need to take into account potential overflow of read operations. So I choose **2 TB of temp space for the US deployment**.

Putting it all together, for the US deployment, the primary DB and the standby DB (because it must have all when it takes over) each require (47.4383035 TB disk space for the tables and index blocks + 2TB for temp +8×4GB for redo log files + 9.2510208 TB for their archive) × 1.3 overhead = a little under 78 TB (let's round up to **78 TB**) **of disk space.**

For backup storing, I choose to use Oracle Recovery Appliance, which, unlike conventional RMAN (which keeps full DB backups, incremental backups, and arch logs), keeps 1 full backup, only incremental backups (backups of only what changed), and arch logs. I choose it because my DB is very big and Recovery Appliance's approach (namely 'incremental-forever') can reduce storage needs and also speed up recoveries [41]. I determine the storage size for that backup solution as follows.

First, each backup storage must store the unique full DB backup. This full DB backup is the backup of the DB at whatever retention limit I choose (e.g. with a 15-day retention, the full is the DB as it was 15 days ago). So this full equals the size of the DB on that day, i.e., max 78 TB.

Secondly, I want each backup storage to hold 1 incremental backup per day for the recovery period. Sizing for the incremental backups requires knowing each incremental backup's size and the number of backups held. I choose a 30-day retention policy. Activity is almost null on weekends. And I add 2 workdays' worth of admin changes per 30 days. Therefore, I need space to hold the equivalent of 29 ÷ 7×5 + 2 = around 23 incremental backups. The physical write size per day is not the size of an incremental backup, because these writes for the same user project but for spaced use case executions often alter the same data blocks, and the incremental backups do not record all the changes, just the state of the changed blocks at the incremental backup time. So the right indicator is the compound daily edited blocks size in DB tables which received user project and user-specific info during the day. Earlier, user project and user info specific DB tables' size was calculated per million inhabitants as 2,500×(27×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)×1.8 + 25,000×(10+1)×1kb×1.8, but 2,500 and 25,000 were user or user project counts for longer period than a day, while in practice, for a day, these two counts come down to 76.48. I add a little overhead for the indexes (worth 51.6

MB/million inhab) since they might change frequently. So the total incremental backup size for the US deployment is 76.48×(27×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)×1.8×350 + 76.48×(10+1)×1kb×1.8×350 + 51.6 MB×350 = 1.32698308 TB. Therefore, the compound incremental backups' pool per backup storage is 1.32698308 TB × 23 days = 30.5206108 TB. This confirms I need less backup storage than with conventional Oracle backups, because 31 TB nearly half the size of one full backup without the overhead (which was around 62 TB before I added 30% overhead, leading to a total of 78 TB), while conventional Oracle backups would store 4 of these full backups for my 30-day retention.

Thirdly and finally, each backup retains the day's arch logs for each of the 30 days. We know the DB expects 440.524MB of redo entries/min/mill. inhab (evoked earlier in the ADR), so 4.86338496 TB for the 30-day period (440.524MB × 60 minutes per hour × 8 hours × 23 workdays of load).

So each of the 3 backup storage systems needs 4.86338496 TB+ 60 TB + 30.5206108TB **= roughly 95.4 TB of disk storage space, but I plan for 125 TB to give each about 30% headroom**.

DB Peak Storage bandwidth

'Storage bandwidth is how much data can be read from / written to a disk on a given period' [68]. We know for the primary DB we have 11 GB/s of write throughput per storage space (evoked earlier), plus 17.815 MB/s of physical read size/s. Therefore, **each of the five storage spaces for the components mentioned (the primary DB, the standby DB, the standby DB's direct backup, the primary DB's direct backup, and the disaster backup) requires 11 GB/s of write bandwidth. The standby DB, when in serving mode, and the primary DB each require 17.815 MB/s of disk read bandwidth. Also, the standby DB, even in standby mode, loads blocks from disk into its cache; since it writes to this cache first, it requires 17.815 MB/s of disk read bandwidth.**

DB Peak memory

The RAM for the DB contains everything that must live in the live memory for use.

First in-memory element: the redo log buffer and what it contains. The redo log buffer is the intermediate storage of redo log records before the LGWR writes them to the online redo log files (they are called online, although they are on disk) [260]. The LGWR writes the redo logs in the log buffer to the redo log files at least every 3 seconds [25]. So 3 seconds' worth of redo logs is what the redo log buffer must be sized for. In my case, 3 seconds' worth of redo logs equals 7.34208 MB/s × 3 = 61.6725 MB/s. But if we size the redo log buffer to that size, it would always be nearly full, and if DB processes behind that buffer checkpoint slow down, the buffer could overflow. So I size the **redo log buffer** a bit bigger, at **1GB per million inhabitants**.

Secondly, I want my reads to be in-cache, and writes to first hit the cached blocks. There are several things needed for that.

First the cache needs to host all blocks and associated structures to serve the active users. We saw earlier what all tables weigh per million inhabitants in the physical disk. If we load all tables which do not change in size **per deployment**, that requires **16.8GB of cache**. To that we add blocks storing info specific to the max 382.4 simultaneously active usage cycles/user projects. On the physical disk, you find 34 tables concerned by this rate (see earlier in the ADR). But in-cache, we likely need only 25 of those at a time. As noted in the disk storage subsection, the total weight of these 34 tables per million inhabitants is 2,500×(27×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)×1.8. The in-parentheses 27, 1, 5, 1 represent the tables, and the 5 and 1 are always loaded in cache, and 2,500 is a fixed number (the number of held user projects). Thus, the calculation for the cache capacity for these 34 tables is 382.4 ×(18×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)×1.8= **12.5GB of cache per million inhabitants**. To that, we add cache for the one table that scales with the number of customers, with 10 < 1kb rows per customer, and one with one 1kb row per customer. Since we have a maximum of 382.4 users active at a time, these tables require 382.4 × (10+1) × 1 kb = **4.2064 MB of cache per million inhabitants**.

To that, we add RAM capacity for loading their indexes. This is calculated as **2.62645GB/deployment and [**382.4 ×(18×1,000 + 1×5 + 5×20 + 1×50) + 382.4×(10+1)] ÷ 131×3×8,192bytes + 51.6 MB = **1.35481808GB/million inhab**.

To have all my reads and writes in cache, I don't need additional RAM capacity to perform logical reads and buffer writes, since they apparently incur no RAM cost [35]. But for operations that bypass the SGA cache and work in the PGA, there is a RAM cost [35]. My physical writes and reads do not bypass the SGA cache. Thus, I add an arbitrary RAM capacity of 20GB for exceptional cases.

For the US deployment with 350 million inhabitants, this amounts to 1GB × 350 + 4.2064 MB × 350 + 16.8 GB + 12.5 GB × 350 + 2.62645 GB + 1.35481808 GB × 350 + 20GB = **a little less than 5.25 terabytes of RAM for the primary DB when serving.**

The backups do not need that RAM, since they store hard copies of files.

When serving, the standby DB will need as much RAM capacity as the primary DB. When dormant, it needs as much as well, because it always persists transactions to the cache first, just like the primary DB [294].

DB CPU

Logical reads, direct reads, direct writes, buffer writes, physical writes of the dirty blocks, ARCn, and redo log writes are the Oracle activities that consume CPU.

For a given number of operations, DB CPU usage can vary greatly depending on the complexity of the operations (e.g., aggregations, sorting), since CPU time increases with the number of SQL executions [356].

The first way to estimate DB CPU usage is to look at the CPU Time (the total time it worked asynchronously) and the total CPU enqueue time (the time CPU would have worked if running synchronously each process sent to it) in AWR Reports of comparable DB loads, and check the used CPU configuration.

A second way is to look at ranges of CPU time a given DB task takes, based on load simulations tools, and pick a CPU time within each range based on the level of complexity of my tasks, then multiply each picked CPU time by the amount of that DB task my DB does [24, p.5, 108]. SLOB is one load simulation tool. But it does not mimic my DB's activity: instead of running my code and my tables, it is a simulator of activity for a given schema size, block size, concurrency level, and quantities of given types of SELECTs and UPDATEs that SLOB itself provides [70, 71]. I suspect INSERTs and DELETEs have a different CPU usage than those two statements. Since my DB has all 4 types of statements, I need a tool to simulate CPU usage for a DB that runs them. Such tools are Hammerora or Swingbench [384].

A third way is to actually run my own SLOB (or Swingbench, etc.) test, specifying the parameters I want it to use (the number of query types and schema/table sizes) [133]. Tools like Swingbench require specific configuration, which can take time and some Java knowledge to run my own queries on my own tables [82].

A fourth way is to run my queries atomically in Oracle and review the CPU time for each run in the AWR report [323]. That way I obtain the CPU time per query, and can determine the average CPU time per main use case loop based on my probabilistic tree of occurrences of these queries in one main use case loop, then apply the same value to loops of satellite use cases, then aggregate CPU time of all loops of one HTTP request, then multiply that by the number of requests/s. This 4th solution applies only to a running app with populated tables. My DB tables are not populated, which eliminates option 4.

A fifth way is to approximate (with AI or otherwise) the average types of reads, writes, and redo writes my program executes, check their CPU requirements and multiply these by the number of occurrences.

I don't want to spend too much time setting up other benchmarking tools, since the current deployment is more of a training exercise than anything else. Finally, I did not find benchmarks with very comparable queries. Therefore, I choose the fifth option. I now execute that method.

Each write to the cache creates/alters one row with ~7–10 values pulled from other tables, some dates, and a unique integer PK. So each takes around 2 µs. Each logical read searches all rows from a given table, whose value in column c matches the value in columns d and e from another table whose id=x. that costs 10 µs max. Each DBWR block write, LGWR write, block archiving, and physical read takes on average 5 µs. In my US deployment, the DB expects 8,030,400 logical reads/s, 2,173.2095

physical reads/s, 713,825 physical writes IOPs (same number of blocks to write), 4,282,880 one-row writes to the cache per second (12,236.8/s/mill. inhab. × 350), as many LGWR writes, and 713,825 archived blocks per second. So in total, we would have 8,030,400×10µs + 2,173.2095×5µs + 4,282,880 ×2µs + 4,282,880 ×5µs + 713,825 × 5µs + 713,825 × 5µs = 117.433276 seconds of compound (i.e., if everything was dealt in a single thread) DB CPU time/s. If I use 256 double-threaded-core CPUs (such as AMD EPYC 9996 with its 256 cores and 512 threads [21]), the US deployment would need 117.433276 ÷ 512= approximately 0.22936186718 seconds to handle the per-second workload of the DB. CPU does not need headroom. Thus, the primary DB only needs a quarter (0.23 is close to ¼) of 1 AMD EPYC 9996 CPU's capacity. Therefore, **my primary instance of the DB for the US deployment requires the CPU capacity equivalent to 1 AMD EPYC 9556 CPU (64 cores)** [22]. This processor 's1k+ GB/s bandwidth (see AMD's website) should suffice.

In serving mode, the standby DB requires just as much CPU as the primary DB instance. In standby mode, it performs the same activities, except for logical reads and for LGWR writes, but it does write the redo entries to its own files, thus I include redo entries' writes. Therefore, instead of 117.433276 seconds of compound DB CPU time/s, it expects in standby mode 2,173.2095×5µs + 4,282,880 ×2µs + 4,282,880 ×5µs + 713,825 × 5µs + 713,825 × 5µs = 37.129276 seconds of compound DB CPU time/s (3.16282159663 times less than when in serving mode). Thus, **the standby DB in standby mode needs** only something 3.16282159663 times smaller than the AMD EPYC 9556, e.g., **the CPU capacity equivalent to 1 AMD EPYC 9256 with its 24 double-thread cores (since there is not one processor exactly 20 times smaller)** [21, 23].

Needed capacity for the AI container

My program relies on 2 models, vLLM as inference engine (see Section 3.2.4) and LM Studio as models' parameters repository (see Section 3.2.4). A model's context is information specific to the given request (in our case, the documents and files the AI models work on, plus the request). The parameters are the AI model's logic capacity and internal knowledge acquired over time/training. Tokens are the units of data that the models process [115]. The engine requires space (ideally in a GPU's VRAM) to load models, processing power (CPU and GPU usage) to execute logic, and the repository needs disk storage to store the parameters [4]. To calculate so, I must take the following three metrics interdependencies and other things into account.

First, the number of AI machines equals the total engine workload (VRAM, GPU and CPU usage) plus context and parameters storage, plus storage for the empty installations of vLLM and LM Studio, all divided by the maximum capacity of a machine. LM Studio's empty installation storage is negligible but vLLM's is not (see later). A vLLM replica can run model replicas only on the machine where it resides [324]. Also, a model replica processes as much as its provided capacity can. That capacity is limited by what a machine can provide. Thus, since I have two models I need at least two

vLLM replicas, which can live on the same machine or on separate ones, but I might need more if two machines is not enough to process the workload. Thus, the storage for the empty installations of vLLM equals the per-vLLM-replica empty installation's size, times the number of machines' worth of AI activity and if that number is 1, then times two since I would have two vLLM replicas on that single machine. This shows the number of AI machines and the storage for the empty vLLM installations are interdependent metrics.

Secondly, to reduce latency and load all parameters (see later), I provision multiple GPUs per model replica. A vLLm replica running a model on multiple GPUs loads on VRAM a single copy of the model's parameters [237]. Thus, the fleet's VRAM dedicated to loading the parameters equals the parameters' weight times the number of model replicas. Since the number of replicas is tied to the number of machines, itself tied to the fleet's VRAM dedicated to loading the parameters, the two metrics are interdependent as well.

I deduce that to calculate the total capacity despite these two metrics' interdependencies, I must calculate first everything as if all could live on a single machine with the minimum number of replicas needed, then check if all that actually fits onto a single machine, and if not, I split it onto multiple machines knowing that each new machine means a part of that machine needs capacity for the storage and manipulation of the parameters and the vLLM empty installations.

Thirdly, each process's context loads on VRAM, constituting the context's VRAM. Thus, total context VRAM equals the size of each task's context times the number of simultaneous task processes. The latter number depends on the execution speed of the processes and how many processes I want to execute in a given time. Speed of execution seems tied to GPU performance (which is the processing speed of the GPU, independently of the percentage of occupied VRAM) and GPU/CPU capacity, as some benchmarks show speed improvements when the number of GPUs/CPUs increases [406]. But I could not identify a linear relationship between the number of GPUs/CPUs per replica and the tokens/s speed. However, even without this data, I cannot calculate context VRAM before calculating GPU needs, because GPUs can come with VRAM and the total GPU count must provide enough VRAM for the total concurrent process, and that total depends on the GPU number itself because the latter dictates processing speed. Therefore, I must calculate GPU and CPU and context VRAM needs together.

The penultimate thing to take into account is that only the request input gets tokenised, not the model parameters [391]. GPU/CPU usage is largely indexed on the tokens as we will see later. Thus, I can calculate GPU and CPU needs of the entire fleet ignoring the number of parameters loading events and inference engine replicas.

Finally, I also deploy a standby AI container instance. When serving, it requires the same capacity as the primary instance. When dormant, it needs only parameters and installations' storage capacity.

Parameters disk storage needs

One or multiple vLLM inference engine (see Section 3.2.4) replicas can each load the same instance of parameters and context from the same disk [392]. Thus, the total parameters/context storage capacity needed equals a single copy of the parameters/context.

My models are a BGE-m3 and a Qwen3-VL-235B-A22B-Instruct. Their parameters weigh a little differently depending on the language used to interact with them, but since my code contains human wording in English, I know that **BGE-M3** (which I choose because it is the bge model supporting the most input tokens at once, which I require for the input token load discussed later in this subsection) has 569M **parameters**, totalling 2.27GB [48] per model installation (**3GB** with headroom). For Qwen3-VL-235B-A22B-Instruct, at the highest precision level I found (fp16), 1 parameter takes 2 bytes [180, 286]. So it would seem that for Qwen3-VL-235B-A22B-Instruct, because it has 235 billion parameters and I choose the FP16 precision, **the parameters of my Qwen3-VL-235B-A22B-Instruct weigh** 235 billion × 2 bytes = 470 GB per model installation (**611 GB** with headroom).

per-inference-engine-instance Disk storage needs

For one bare install of vLLM, the needed storage is 8 GB [66] (11 GB with overhead). Since I need at least two replicas, the minimum installation storage of my vLLM fleet is 22 GB.

AI container's context storage needs

On disk, an AI system stores model parameters and sometimes some context. The AI task 1 uses 10 MB worth of user-input documents[9] per usage cycle, but it does not store them to the disk, it only keeps them in memory for the time the task completes. Task 1 produces a JSON file as output, that likely weighs 2 MB per user project [100], and that must be kept to perform task 2 processes. Task 2 stores no context on disks. Thus, if storing context on disk, the US deployment expects 2 MB × 76.48 usages cycles max a day per million inhab. × 350 = 53.536 GB of stored AI context per workday. I must hold the extracts for at least a full workday because Task 2 processes for a single-user project occur during almost the entire 1-workday usage cycle, but some users might pause and resume a usage cycle many days or weeks later. Holding context in memory would be expensive considering its total size (53.536 GB). Therefore, I store it on disk. However, re-computing the extracts is expensive. Thus, to avoid storage overflow, and because often usage cycles should not extend a couple days even with interruptions, but to avoid over-recomputing I decide to retain only the JSON extracts for 15 workdays and add 30% headroom. Therefore, **for the US deployment, the AI container requires** 15 days ×53.536 GB×1.3 **= 1.04 TB of context disk storage**.

[9] This is because the concerned documents are financial reports in their non-public form (just the numbers, not the detailed public explanation), and from looking at different ones online, this seems to be the trend size if we group all potential years of coverage + the size of the documents [151, 400].

Per-model-replica parameters' VRAM

Models ideally load all their context and parameters onto the V-RAM [397]. Based on parameters' weight indicated earlier, the model's parameters for task 1 require 611 GB of VRAM per model instance, and the model's parameters for task 2 require 3 GB of VRAM space per model instance. As evoked in the introduction of the AI container capacity planning, a model replica loads in vram only a single copy of the parameters. Sources mention that the Qwen model only has 22B active parameters, but I did not find evidence that this means that for a single inference engine replica handling multiple requests (e.g. task 1 requests), the model is limited to 22B on VRAM. This makes some sense in my opinion, because if the engine receives 9 task 1 requests, and we know that each task 1 request comes with a different set of documents to perform the task on, then, at a single time, the engine might need to use more than only 22B parameters, since each task potentially needs different parameters to deal with their documents. Thus, I rather assume that the entire 235B parameters load on VRAM. This means **the per-task-1-model-replica parameters VRAM is 611 GB, and the per-task-2-model-replica parameters VRAM is 3 GB.**

Task 1 context VRAM, and CPU and GPU usage

As mentioned in the introduction of this AI fleet capacity planning subsection, the following is the result of iterative adjustments between VRAM and GPU estimates.

Context VRAM includes VRAM for loading context tokens and the context elements before they are tokenised (in our case, FastAPI requests, including some JSON and other files).

AI Task 1, i.e., the full info extraction and JSON file conversion, must have a context VRAM big enough for each potential concurrent task 1 request. I set that the software expects maximum 20 task 1 requests/s for this deployment.[10] The token input per task 1 request is on average 600,000 tokens, with maximum 60,000 tokens of output (input of an average of 10MB of PDF containing texts and images, made of 95+% of financial reports and 5-% of legal info, output of an average of 2mb JSON worth of {infoname:infovalue} elements [179]). A Megatron Bridge model training system's benchmark shows that the token throughput of the Qwen model I use, when provided 256 H100 GPUs, is 1,575 tok/s/GPU [240]. This matches sources saying that the more GPUs one adds for a task, the more inter-GPU communication, which makes the tok/s/GPU relatively low, quite close to the tok/s of 8-GPU Qwen models' runs [406]. Since my GPU count will be high too, I can trust this tok/s/GPU amount. With my 660k tokens per task 1, and a 20 task 1 process concurrency, and arbitrarily deciding that a

[10] Since there is only one task 1 request per usage cycle, and each usage cycle lasts only a day usually, and there are 76.48 usage cycles/day/million inhab., the program expects 76.48 task 1 requests/day/million inhab. AI work is intensive, especially task 1. Thus, I can force a queue for the task 1 requests so that in the AI deployment there is never more than 10 task 1 requests hitting FastAPI per second. This should be fine because users do not exactly trigger a task 1 AI request at the same time, and with 76.48 × 350 usage cycles a day, that makes for all task 1 requests to arrive in around 22-23 minutes compound, and they should not all arrive in the same 22-23 minutes time window.

task 1 process should last maximum 120 seconds excluding time waiting in queue,[11] Therefore, **the US deployment's task 1 workload requires** 660,000 tokens per task × 20 tasks ÷ 1,575 tok/s/GPU ÷ 120 seconds processing time per task 1 once the process starts = **a little under 70 H100 GPUs** to complete the 20 task 1 processes per seconds in 120 seconds max.

If each task 1 process lasts 120 seconds maximum (less for smaller files, e.g. 10 seconds minimum), and the GPU capacity can only deal with 20 maximum at a time (rule I've set), this means task 1 concurrency is 20 and up to 20 task 1 processes can finish at the same time. Each process uses 10 MB's worth of user input documents, which the VRAM must load. It must also load the thin FastAPI request text itself. Each finishing process outputs a 2 MB maximum JSON file, which lives in VRAM before being sent to disk storage. Therefore, taking into account that I want the next 10 requests to start only after the 2 MB JSON files got stored on disk, if only 10 task 1 requests get processed at a time, the compound non-token task 1 context in the US deployment consumes (10 MB × 20 + 200 bytes of request text besides documents × 20 + 2MB×20) × 1.3 for headroom = 312.0052 MB of VRAM. To that, we must add token VRAM load, since the input gets tokenised and outputs come first in the form of tokens too, this entire token load forming a cache called the KV Cache, loading on VRAM [225]. The model for task 1 is Qwen3-VL-235B-A22B-Instruct. It requires 192,512 bytes/token of KV cache (KV-cache memory per token is 2 × layers × KV heads × head dimension × bytes per element, and in the context of this model at FP 16 accuracy, which is the highest one I found available, this equals 2 × 94 × 4 × 128 × 2 = 192,512 bytes/token), and this seems to encompass both input and output tokens [169, 305, 307]. Each task 1 has a token load of 660,000 tokens. There are maximum 20 concurrent task 1 processes for the US deployment (20. Thus, counting some overhead, task 1 tokens require maximum 660,000 × 20 × 192,512 bytes × 1.3= a little less than 3,304 GB of VRAM. This means **the total VRAM load of the task 1 context workload is around 3,304 GB**. With 70 H100 GPUs, I have 70 × 80 GB of VRAM = 5,600 GB of VRAM. This is largely big enough for the entire workload, meaning **the 70 GPUs provide enough task 1 context VRAM**.

Nvidia recommends a minimum of 6 CPU cores (of either NVIDIA Grace, Intel Xeon scalable processor or AMD Turin) for each GPU (GPU, not GPU core) for inference [241, 242]. An Nvidia Grace chip provides 144 cores [243]. Thus, **task 1 requires** 70 GPUs × 6 ÷ 144 **= a little less than 3 Nvidia Grace CPUs for this deployment**.

---

[11] This deployment expects a total of 76.48 × 350 = 26,768 task 1 requests a day. If they constantly come in the same 22-23 minutes time window at a rate close to 20/s, and 20 are dealt with at a time in 120 seconds, the total (meaning the pending tasks plus the ones currently processed) concurrency is maximum 2,400, the maximum waiting time for a request to obtain a response is 2,400 ÷ 20 = 120 seconds for the process to start + the 120 seconds of the process running = 240 seconds. That is more than totally acceptable for this process, which is a one-time process per usage cycle, and also because the task 1 requests will only rarely come in the same 22-23 minutes time window at 20/s rate, and almost never in any shorter time window nor at higher rate, and also because it makes sense for GPU needs compared to VRAM needs, as the entire subsection here will show.

Task 2 context VRAM, and CPU and GPU usage

There are 67.3024 Task 2 requests/s/million inhabitants (the average number of requests per second per million inhabitants × number of use case loops requiring info extraction × number of info extracted per loop = 3.0592×22×1).[12] For the US deployment that's 67.3024 × 350= 23555.84 task 2 requests/s. These requests come at that rate steadily throughout the workday, meaning I need the load processed at the rate at which it arrives, thus completing 23,555.84 task 2 processes/s.

The model processing task 2 requests, Bge-m3, sustains a 90,000 tok/s throughput on an Nvidia H100 PCIe GPU, no matter the input/output (prefill/decode) ratio [230]. This means task 2 processes require 23555.84 ÷ 90,000 = 0.26173155555 H100 GPUs to be dealt with in a second. I decide to dedicate at least 1 entire H100 GPU to this workload to process it in around a quarter of a second, which allows to, considering network latencies, complete the full AI task 2 request/response cycle under a second.

Each task 2 request induces the model loads only the section of interest from the target JSON file (see Section 2.2.3.). A section weighs at most 4 kB.[13] Task 2 documents and answers get tokenised. Each task 2 token output (also called decode throughput [362]) is 1 float value or a couple of words worth of tokens (so around 20 tokens max). Each task 2's token input (also called prefill throughput [8]) is a max 4kb JSON file (800 words max, 1,100 tokens max) plus the text request (100 tokens max). That is a total of 1,220 tokens/request. Thus, a single task 2 request completes in 1220 ÷ 90,000 tok/s = 0.014 seconds (14 milliseconds). Knowing the workload can require the GPU to process 90,000 tokens/s, there are up to 90,000 ÷ 1,220 = 73 concurrent task 2 processes in the US deployment. Thus, assuming an additional 2 MB total per million inhabitants of context for table data and request info that FastAPI sends as request, the compound non-token task 2 context in the US deployment is [73 × 4kB + (2MB × 350)] × 1.3 = 910.3796 MB. To that, we need to add the VRAM for the KV cache load. The model for task 2, BGE-M3, is based on a XLM-RoBERTa-large architecture [246]. It has 1,024 hidden dimensions [47]. It requires 196,608 bytes/token of KV cache (and in the context of this model at FP 32 accuracy, this equals 2×24 layers×16 KV heads ×64 head dimensions×4 bytes per element in fp32 config = 196,608 bytes/token), and this likely encompasses both input and output tokens [46, 169]. The processing batch must be big enough to execute the concurrent task 2 processes (up to 73). Therefore, the number of concurrent tokens for task 2 processes is 73 × 1,220 = 89,060. That is a little under the 90,000 tok/s speed of the single GPU because it cannot do half of a 74th task, knowing 74 tasks would exceed the 90,000. Therefore, task 2 tokens require up to 89,060 × 196,608 bytes × 1.3 = 22.762881 GB of VRAM, thus fitting on the 80 GB of VRAM of the H100 GPU I provide for the task 2 workload.

---

[12] 20 is the number of loops in the use case representing 90+% of the use case workflow, and I add 2 loop for the other tasks which can run in the same HTTP request-response run.

[13] This is estimated based on comparison to sizes of JSON documents found online [100].

Knowing the per-GPU CPU ratio guidelines and the number of cores per Nvidia Grace CPU evoked earlier , × **task 1 requires** 1 GPU × 6 ÷ 144 **= a little less than 0.042 percent of 1 Nvidia Grace CPU for this deployment**.

Now that we know the total of GPUs and CPUs and VRAM the context consumes, let's see how many machines I must deploy to benefit of that capacity.

Number of machines needed and final capacity count

An Nvidia rack system with H100 GPUs comes with maximum 8 GPUs per system unit, with 2 122-cores XEON CPUs which work as well in my case, and a single of these systems has enough disk storage to hard-store the parameters as well as the entire 1.04 TB of to-hard-store context data (as discussed earlier in this ADR) and an installation of 2 vLLM replicas [244, 245]. These 8-GPU-2-CPU systems, called NVIDIA DGX H100, can talk to each other and act a single logical unit, requiring only a single load of the parameters (and due to their size, i.e., 611 GB plus 3 GB, the unoccupied VRAM out of the 5,600 or so GB of VRAM is largely sufficient since the context only takes up to 3,304 GB of the VRAM) [51, 337, 395]. And we discussed earlier that there are no fixed limit of capacity provision per replica with these two models other than how much capacity I can physically configure. Therefore, I can simply deploy a single replica of each model on the NVIDIA DGX H100-based config, and the parameters load only once for the entire system. Therefore, **the AI container is actually only one replica of each of the two models, and all is running on exactly 71 ÷ 8 = 9 NVIDIA DGX H100s when serving. When dormant, it just needs around 620 GB of storage, neither GPU, CPU usage nor VRAM.**

Now we can calculate the needed capacity for the FastAPI container. Later, I will also discuss exactly where I eventually hard-store task 1 output data (see the subsection on optimising processing speed).

Needed capacity of the FastAPI container

The FastAPI container (note when I say FastAPI container I mean the FastAPI web framework structure, and what runs behind it, i.e. Starlette, Uvicorn, and UV [124]) contains my clean architecture code and executes as follows: ASGI server (Uvicorn / Hypercorn), then FastAPI router, then Dependency resolution (Dependency Injection), then Validation and parsing with Pydantic, then Type conversion, then Response serialisation, then sending the response [101]). Uvicorn workers execute this work [125]. The FastAPI documentation says that Uvicorn workers are 'processes' and that 'processes consume RAM and CPU' [125]. Thus the processes I just described, which my app uses, consume CPU. And FastAPI does not send any hard storage; the info on the async processes runs in the async event loops, where Uvicorn workers' coroutines queue, so these loops and processes have a live presence in memory [118]. Therefore, I must calculate the CPU and RAM usage of the FastAPI container. I also need to gauge the system workspace and disk storage required to install FastAPI and store the installation, code, and structure of the current FastAPI project.

Note that I also deploy a standby instance of FastAPI; when serving, it needs the same capacity as the primary instance, and when dormant it only needs storage capacity to store the FastAPI installation.

Number of FastAPI instances per deployment

There is no maximum number of workers per FastAPI container instance [382]. Thus, I can set up a single FastAPI container and provide it with everything it needs to handle the expected workload. However, I know I need a primary and a standby instance.

Disk storage

There are multiple smaller FastAPI versions than the exhaustive one, but while their installations weigh less than the exhaustive one, each loses some functionality that seems useful in my case [385]. Thus, I stick to the classic install, which, without any project but with Starlette, Pydantic, and UV, is only a few MBs (as I observe when creating a FastAPI project via PyCharm). And the code and files I build for the project are a couple of MB as well.[14] The FastAPI installation, especially when packaged as a container image, can easily reach 1+ GB [385]. Thus, I plan at least **4GB of disk storage for an entire FastAPI c4 component**. And since I only need one instance of that component, that is all the disk storage space I will need for the US deployment.

FastAPI container's CPU

Here I use an approximate value because the only correct metric would be the one of my eventual code up and running. And this is fine because, as mentioned in the introduction, this deployment crafting is best taken as a practice exercise, and dummy values are provided when lacking to complete the entire exercise.

My FastAPI container expects 3.0592×350= 1,070.72 HTTP requests/s for the US deployment, and this likely implies 1,070.72×10×0.04= around 428.288 of concurrency because we estimate roughly that the HTTP request-response latency would be around 10s[15] max on average, and 96% of the latency is on the components external to the FastAPI container (DB, AI container, Stripe, OAuth). According to the last two TechEmpower benchmarks, one FastAPI container instance, when the code is well written and the surrounding architecture is well built, can process around 65,723 single-row *select-then-JSON-serialisation* requests per second at 512 concurrency, on a single Xeon 6330 box with 28 cores (56 threads) [91, 182, 343, 367]. This could go down a little when coming from 214.144

[14] This is based on an example of a FastAPI project with nearly 88,000 lines of code and thousands of files weighing in total 3 MB [99, p. 79].

[15] Remember that the duration of a request-response cycle varies depending on how much info the program finds before it needs to consult the user again. For the present project, although the main use case takes around 20-50 use case loops executing under a second, I set the average cycle time to 10 seconds to take into account the minority requests which lead to a AI task 1 request can take a minute or more to complete a task 1, thus raising the average latency. This means that 10 seconds is likely an over-estimation, but it's better to plan too large than too small.

concurrent users: thus I estimate 60,000 such requests. I use the smaller of the two, i.e., the number from the 256 concurrency, as it inflates my estimation a little, and as I prefer oversizing. Now my FastAPI processes triggered by a single HTTP request weigh around 250 times the work of a single-row select-then-JSON-serialisation request. I base this on the 30 use case loops/HTTP request mentioned earlier, with 39.6 transaction loops and the 150 read loops/HTTP request, plus the rest of the activity the FastAPI will perform besides creating these transaction commits, and since the benchmark metric includes DB latency time, I figured the added complexity of my project would make for my FastAPI's CPU time would include also that DB latency time this benchmark included. Thus, in one second, for the same setup, my FastAPI container instance would process 60,000 ÷ 250 = around 240 HTTP requests per second. Thus, my app's traffic is 1,070.72 ÷ 240 = around 4.4613 times what this Xeon processor can process; the US deployment requires a 28 × 4.4613 = around 124.9164 cores + 30% headroom = around 163 (or above) cores processor for performance/core equivalent to the Xeon 6330's. Therefore, to be large, **the FastAPI container for the US deployment needs the CPU equivalent of 2 Xeon 6980E+ processors, each with 264 cores** [183].

FastAPI container's RAM

I use an approximate value for the same reasons as for the CPU usage.

From my understanding, FastAPI uses RAM to store the live info of the Python application, the runtime, the currently active connections/tasks and their state/data (this last part notably because of the asyncio system, which makes the FastAPI layer await certain external actors to return something to resume a given Uvicorn process).

I know the serving FastAPI instance handles 1,070.72 requests per second in this US deployment, and the program takes up to 10s to return an answer on average. Due to choices I discuss later in this ADR, the FastAPI instance sends the all the input of the AI task requests, including documents for AI task 1, and sections of JSON extracts for AI task 2. It also receives the AI response results, including the extracts from task 1. And there are a maximum of 2,400 concurrent task 1 AI pending responses in this deployment (see earlier in this ADR), and 235.5584 concurrent task 2 processes. As evoked earlier, some AI task 1 requests (including their inputs) will queue, waiting for the current processes to end. I can put that queue in different places. One option is on RAM in the FastAPI container. In this case, each task 1 request's 10MB of files and 10 kB of info must stay in RAM until processed, while up to 20 task 1 outputs concurrently hit the FastAPI container. Therefore, for task 1-bound HTTP requests, and assuming state info for each awaiting FastAPI co-routine takes 10kB of RAM minimum to which we can add AI task document transfer and AI response overhead, **the FastAPI container instance when serving requires** (1,070.72 HTTP requests/s×10kB×10 seconds + 2,400 concurrent AI task 1 requests×10MB + 20 AI task 1 responses× 2MB + 235.5584 concurrent AI task 2 requests × 4kB + 235.5584 concurrent AI task 2 responses × 100b)×1.3 of overhead = **32 GB of RAM**.

Solutions retained based on capacity requirements

Based on peak capacity estimates we just read, we know that the US deployment has capacity requirements equivalent to (this means the given products I choose can be different but their capacity must be equal to) the following:

- The primary instance of the DB requires 5.25 TB of RAM and 1 AMD EPYC 9556 CPU and 78 TB of disk storage with 11 GB/s of write throughput, 17.815 MB/s of read throughput.
- The standby instance of the DB, both in serving mode and in standby mode, requires just as much capacity as the primary DB, with the only difference being that, for CPU usage, the standby DB in standby mode only needs the equivalent of an AMD EPYC 9256 CPU.
- Each of the 3 backup storages (one co-local with the primary DB, one co-local with the standby DB, one disaster backup storage in another data centre) requires 125 TB of disk space, and 11 GBps of storage bandwidth.
- The primary instance of the AI container in this deployment requires 9 NVIDIA DGX H100s.
- The standby AI container requires as much capacity when in serving mode as the primary instance; otherwise, it requires just the space to store installations and parameters (620 GB).
- The primary instance of the FastAPI container requires 2 Xeon 6980E+ processors, 32 GB of RAM and 1.04 TB of disk storage.
- The standby FastAPI container when in serving mode requires as much as the primary instance; when dormant, it requires only the 4GB of disk storage.
- KeepAlived is very thin, so negligible.
- Since I sized all the components with 30% headroom for each, and since the sizing was made based on benchmarks which run on a given system eventually, the capacity listed in the 3 previous bullet points should be all I need.

As evidenced later in the price analysis subsection, the biggest unit eligible for a tight co-local deployment (same data centre), i.e., the primary program instance, when running directly on bare metal or on virtual machines, requires capacity spread across multiple chassis. This does not eliminate any of the hosting options retained so far, because whether as a supercomputer or as multiple network-connected computers, each deployment option still shortlisted can provision the capacity (I give some examples later). I now tackle the next determining selection criterion.

Solutions retained based on the level of control I need

Relying on Decentralized/peer-to-peer compute marketplaces (DePIN), such as vast.AI and Akash Network, is risky because they have no SLA layer, are often not compliant with my needed compliance requirements, and are likely more costly because they function based on bids for hardware usage, and thus this solution makes hardware usage for my app uncertain based on potential co-

locators' bids [5, 107, 313, 377]. And the same uncertainty applies to spot cloud (access to cloud resources as long as a more privileged customer 'steals' the capacity). So I eliminate these resource-access solutions from my list.

Also, I could choose either private or public cloud prices, but private cloud requires a lot of set-up and managing work while public cloud is managed and maintained and updated by the provider (e.g. AWS) [17], and high security. For that, I will not use private cloud.

Finally, I read some articles about Cloud lock-in, i.e., the idea that it is often difficult or costly to switch cloud providers. In my case, the migration aspect is not impossible; the hard part is migrating the DB data because it is large and keeps changing, while the backup can write to my new target cloud deployment. Now the real issue is the payback of switching. The only thing I actually must migrate as such is the DB data, around 340-350 TB, and such migration costs from a few thousands to 20,000 dollars. But if you count 50 thousand dollars or more for the labour cost of the operation (because this requires some people to supervise and plan), then payback is only profitable after some years, and that's if the previous provider did not lower its pricing in the meantime. But to me, cloud lock-in is not a good reason to eliminate cloud as a deployment option. I think so because bare metal also has same-effect lock-ins (if I want to buy components which consume less electricity, or switch to more performant c4 components, that switch implies similar costs because in bare metal element or/and in labour costs for the migration supervision), and because Cloud can be a more financially profitable option in some cases.

Solutions retained based on processing speed and performance

My system needs to be fast because users complete 200+ requests per day, and long waiting times directly impact customer satisfaction and retention. The underlying capacity is the same across hosting options, but the HTTP request-response latency varies because each option has different network latency costs. A network latency difference between one hosting config and another ranges around 1 microsecond per stream all other things being equal, plus the bandwidth difference: for example, if communication used to go through X cables, and now it goes through X other cables, each less performant cable has a 1 microsecond higher stream cost, and that's X more microseconds of latency, all other things being equal, let alone bandwidth limitations. Thus, do network latency differences between deployment options impact my program?

I calculated the required capacity based solely on the processing requirements. Therefore, the capacity I calculated suffices: adding more will not make a deployment with higher I/O latencies faster. My FastAPI container, the primary instance of the DB can fit on one machine. That would be optimal for communication between these components, because the number of communications between them is gigantic (see capacity estimates), but it could compromise security and efficiency. Thus, I keep them

on separate machines, and having them in the same data centre, connected by direct cables, should keep the communications fast enough.

Another machine can host the backup of the primary DB instance, and a wired Ethernet link that provides 50GB/s of network bandwidth sustains the write load. Network fabric is suitable too [177]. Non-wired communication to the backup DB is not really suitable because not as effective or as reliable as wired communication, and laser transmission (FSO) is overkill (more technology to deploy, more environmental constraints). Wired communications do not require servers to be adjacent, as data centres can wire-link two servers even when in opposite corners of the same facility [26]. Communication between Cloud VMs of the main providers can have similar network bandwidth, but it requires multiple parallel flows to be provisioned at all times, which providers do not guarantee – I deduce that from sources saying bandwidth can decrease when the network gets busy [18, 148]. Because my backups are asynchronous periodic backups, some small additional latency is fine. But I would then need to give the primary DB machine more storage so it does not overflow in ARClogs when sending backup data to the backup gets slower. The same logic applies to communications between the primary DB/the standby DB's backup, and to the disaster backup (which is on another machine in another data centre).

One question remains: what do FastAPI and the AI C4 containers need to communicate besides the vLLM instances acting as AI APIs, and FastAPI acting as API? Do they need a message broker or load balancer of some sort? The FastAPI container can host the task request queue. Also, task 1 requests go to the single vLLM replica for task 1, task 2 requests go to the single vLLM replica for task 2. Also, each AI task request is a step of the synchronous user project workflow. Thus, it does not need to survive beyond the HTTP request's lifetime. Consequently, the FastAPI-AI communications do not require a broker. The latter would be useful only to make the AI call's lifetime independent from the HTTP request's. Therefore, I decide that the FastAPI container manages the task requests input and outputs. That avoids installing an app outside the FastAPI container to load/store/maintain documents, which would add latency or configuration hassle. My choice induces that the 4.871776TB compound context storage is within the FastAPI container.

Note LM Studio stores the model parameters, not the context. Therefore, the LM studio instance can live separately from the FastAPI machine.

Finally, note the standby program instance can be a little slower than the primary, since its use is exceptional. This includes accepting a cold startup of the elements, as evoked earlier. But for the primary instance, a cold startup would significantly degrade the user experience. Especially so concerning the AI models, because cold-starting a model can take several minutes. One way around this slow model cold-start issue is to keep the model's parameters loaded in memory, thus only

dynamically scaling the rest of the needed serving capacity when there is context traffic. Note that this does not disqualify cloud deployment since that memory capacity can remain active on cloud too.

Solutions retained based on the level of isolation I want, and containerisation

I choose to use containerisation with Docker, and no virtualisation (unless the retained deployment solution involves virtualisation by default) for the following reasons.

First, containerisation isolates the dependencies required to run a given component from the rest of the program. This eases debugging, migrations, and duplication (the latter, notably, for developing new versions and integrating them smoothly into the main workflow), and there is no size/capacity limit, because containerisation isolates the element's environment and dependencies, not the CPU/RAM/disks storage it consumes [317, 111]; this helps keep a component as a single unit, if capacity-per-machine allows so, thus avoid load balancing of the requests to that component. Note, with each added layer of containerisation or virtualisation, latency time increases [85]. But often this increase is negligible and the isolation advantages outweigh it.

Common containerisation options are Kubernetes and Docker. Kubernetes and Docker both perform automatic health checkups and restarts of images which run abnormally, which is a feature I need [50, 201]. Thus, I do not lose that feature by choosing either. Kubernetes' advantages over Docker are to manage microservices across machines and with multiple images per microservice, as well as managing a large number of containers [19]. However, I deploy everything as co-locally as I can (see earlier in the processing speed subsection), and the app is a modular monolith. And the capacity estimates evidence each instance has 7 C4 components/container to containerise (the DB, its backup, the FastAPI C4 container, the by-FastAPI reverse proxy, the two vLLM replicas and the single model repository). Thus, Kubernetes' advantages over Docker are useless to my program, while Docker is easier to manage, set up, and lighter/cheaper. I thus favour Docker over Kubernetes.

Secondly, portability issues can exist despite containerisation, but Docker prevents them. Indeed, in some configurations and with certain container providers, in-DB data written after the container image's installation might be stored in a way forcing to migrate the OS with my program when I migrate the latter to another server. Luckily, with Docker, I can persist in-DB data in a Docker volume, which does not delete when the container restarts or is deleted (unlike when writing data in the container's writable layer) [110], and which I can migrate alone, without the machine's OS [373].

In my case, I expect non-frequent migrations. Reinstalling the Docker engine and the needed OS[16] on the migration destination plus migrating my few Docker containers (7 per program instance) and

[16] because though I do not need to migrate the OS, I still must always run Docker on an OS [111].

volumes is very quick, thus not that time-expensive over time. Plus, adding a VM layer adds a small latency and VM infrastructure costs of multiple gigabytes of storage, while containers are much thinner isolation technologies [57]. Therefore, unless the retained deployment solution involves virtualisation by default (e.g., using the Cloud), I do not use a VM.

Note that I will not containerise KeepAlived, because I did not find evidence it should be done.

In conclusion, this usage of containerisation (possible of virtualisation too) does not eliminate any of the deployment options shortlisted so far. Now let's eliminate the remaining deployment options based on the last remaining criterion: price.

Solutions retained based on price

Deployment prices comprise prices to deploy the capacity itself, and prices of necessary memberships/usage fees that one must pay depending on the deployment option. In my case, services I pay are Stripe, and Oracle Database (OAuth2.0 protocol is free to use, and as seen in the ADR on security decisions at system level, I do not pay any provider for the implementation of the identity management and security) [232, 353]. Stripe fees do not vary by deployment option, as fees are indexed on user payments [353]. But Oracle prices do vary, as discussed later. Therefore, to determine the cheapest of the remaining deployment options, I first discuss capacity prices besides Oracle fees, then I add Oracle DB fees into the equation.

Prices of capacity aside from membership fees

While the backup DB storage must always be running because it receives constant data from the in serving mode DB (since it copies massive amounts at an hour interval), the other elements (standby AI container, standby FastAPI, standby DB) need to run (and thus need capacity) only when the in serving mode program cannot run anymore. This makes Cloud deployment of the standby program components, besides the standby DB, interesting because the cloud can provide dynamically-scaling capacity costing high fees only when the standby program components (besides the DB) serve, which should be rare. Let's first compare prices per element by deployment option, and then I will compare overall solutions based on requirements such as logical location, together with price.

For the solutions where I own hardware (the first two in the following table), I must assume a hardware renewal rate of 5 years (10 years for the racks). Thus, the annual price of a hardware component is its purchase price divided by 5. Also, I pay maintenance for anything I own. Elements receive near-null traffic outside the 8-hour workday period, except for the DB, which runs incremental backups at a fixed night hour and other processes. I also assume at least 3 days per month (36 days per year) of activity in which the standby components are in serving mode, but that is not 36 days of usage less for the primary instance, because while the primary instance is down, it will undergo

internal/admin activity for troubleshooting purposes. Finally, as noted earlier, DB backups can be on HDDs as long as meeting write bandwidth needs does not cost more than the equivalent SSD capacity.

Note that for Cloud pricing, I use the prices of on-demand and local hardware solutions, since these are requirements (see earlier).

Lastly, remember hardware components cited so far are examples to gauge requirements and prices.

Tables: cost comparisons of hosting solutions for the largest system component (before Oracle solution purchase costs):

	Primary DB
Price on-premise/y	The primary instance of the DB requires 5.25 TB of RAM and 1 AMD EPYC 9556 CPU and 78 TB of disk storage with 11 GB/s of write throughput, 17.815 MB/s of read throughput. The costs CPU $8,000 [22] ÷ 5 years amortisation = $1,600. The RAM (5.25 TB) costs around $500, and we divide by 5. The SSD storage costs at least $20,000 ÷ 5=$4,000/y [186]. This set up would require a specialised 1U or 2U rackmount server chassis with direct-attached PCIe Gen 4 or Gen 5 NVMe backplanes and robust multi-lane support, costing $3,000 to $5,000 (so $800/y) [348, 408]. To that, we can add $25,000 of employee work, external maintenance and energy needs for the on-prem infrastructure per year. A thousand dollars for wiring. 20,000 for security procedures. Total: 20,000 $52,500/y, ignoring building rental prices.
Price on my hardware in rented rack space/y	The same prices for the components since I own them, plus $100/month for housing [86, 195] (likely more for security purposes. I probably need 2U ($700/month), and $3,000/y of component maintenance. The rent includes network wiring costs. I add $20,000/y of security procedures. The total is 20,000 + 3,000 + 1,600 + 800 + 4,000 + 100 + 700 × 12= $38,000/y. **This is the cheapest option.**
Price on rented hardware and rented rack space/y	I can imagine it costs a bit more than the above option, since renting the components is expected to cost as much as or a little more than owning them, because rental providers must be profitable and incur the same purchase and maintenance fees. However, renters buy items in larger quantities, so they get larger discounts. So I still checked to be sure. However, it seems that the CPU plus a fraction of the RAM (between 1 and 3 TB, out of the 5.25 TB needed) and storage I need is already as expensive as the housing solution (around $2,000-3,000/month ×12 months) [279].
Price on Cloud/y	An AWS U7i-6tb (provides what I need plus 2.5 times additional CPU need), solution costs $62.79/h [81]. Capacity of the DB is rather down on weekend days, and outside

	work hours users do not much solicit the service, but the DB has off-workhours activity due to backups, and we can count within week hours an hour or 2 for the thin weekend incremental backups. So we can count around 16 hours of activity per workday. This solution would then cost \$62.79/h ×16 hours ×261 days also compensates for the ignored thin incremental backups of the weekend days) = above \$262,000/y.

	Standby DB (considering 36 days/y of in serving mode activity, but the DB is still working even when not serving)
Price on-premise/y	\$52,500/y, because downtimes are mostly unpredictable, thus requiring always to dispose of the "as if in serving mode" capacity. This ignores building rental prices.
Price on my hardware in rented rack space/y	\$38,000/y., for the same reason. **This is the cheapest option.**
Price on rented hardware and rented rack space/y	Same as for the primary DB, so costing over the two above.
Price on Cloud/y	Unlike with the primary DB, CPU usage is much lower when the DB instance is on standby mode. But RAM stays as high as for the primary DB instance, and that drives cloud prices over the two first options'.

	the 3 DB backup storages
Price on-premise/y	We need (125 TB of disk space, and 11GBps of storage bandwidth) × 3. 10 Exos M HDDs of 24TB and 6GB/s per backup provide exactly that. This costs a total of 10× \$1,100.22 [49] ÷ 5 years of use = \$2,200/y. To that, we add at least \$10,000 in maintenance and energy costs per backup per year: only two are on-prem, since the disaster backup is in a different dedicated location, but I count for its maintenance too as 10,000. Also, the rack install medium for each backup, i.e., a JBOD enclosure with 3.5-inch bays [11], costs max \$100/y each. But then we add quite security fees (could easily rise to 60 thousand dollars in total). We can add 20,000 dollars for building space renting since it requires 3 small size locations. Total: 10,000×3 + 2,200×3 + 100×3 + 50,000 + 20,000 = \$ 116,900/y,
Price on my	\$2,200/y + \$100/y + \$700/month for 2U rack space/month (since this is all supposedly

hardware in rented rack space/y	needing a 2U rack space), and there is probably still $3,000 of component maintenance per year per backup. I can add $20,000 each per year for additional security procedures. Total: ($2,200 + $100+ $700×12 + $3,000+ 20,000)×3= $101,100/y. **This is the cheapest option.**
Price on rented hardware and rented rack space/y	Rental prices are $29.96/month for each of my 10 drives [88]. That's 29.96×12×10×3 backups= $10,785.6/y. To that we add $700×12 for the rack space renting per backup, and $117.66/mo per JBOD [89]. And still we must add 2,000 of maintenance per year per backup. I can add $20,000 each per year for additional security procedures. Total: $10,785.6 + ($700×12 + $117.66+ 2,000+20,000)×3= $102,339/y.
Price on Cloud/y	With pricing around $0.1/GB/month [149] (and since we have 125 TB per DB backup, which is 125,000×1GB), so the total would be $0.1×125,000×12 months × 3= $125,000/y.

	Primary AI container instance
Price on-premise/y	The primary instance of the AI container in this deployment requires 9 NVIDIA DGX H100s when serving. This costs me $269,000 per DGX h100 system [65] × 9 ÷ 5 years = $484,200/y. To that we can add a few thousands for network cabling. Looking at the components' energy consumption and the price of electricity in the US, and knowing a single system consumes 10.2 Kw/H of electricity, we add at least 92 Kw/H for the entire AI container, which costs about $16.87 per hour [295] × at least 8 hours a day × 21.5 days (the workdays basically) × 12 months= $34 819.68/y. We can count at least as much for heat cooling and sound-proofing. We can count $20,000/y of maintenance of the on-prem infrastructure, and $20,000 for security. The total is $596,839.36/y, ignoring facility rent prices.
Price on my hardware in rented rack space/y	This solution costs the same components' prices minus electricity bills plus at least half of the maintenance bill + rack space renting bill. Rack space renting facilities usually charge 196 dollars/kwh/month, and this is paid even if the activity goes below that on multiple days in the month: it prices on what the rack would usually consume when functioning [87]. My AI container requires around 92 Kw/H . So that's a total of 92 kwh × $196 × 12 months =$216,384/y for rack space renting. Putting it all together, we get 484,200 + 216,384 + 10,000 + 20,000 = $730,584/y. I ignored the small additional costs for the HDD storage space. But because the on-

	prem pricing so far ignore the renting prices, **this is likely the cheapest option.**
Price on rented hardware and rented rack space/y	We already know the colocation price: 216,384/y. Then we have the components' rent prices. I did not find definite prices, but I expect it to be pricier than the previous options.
Price on Cloud/y	For on-demand (automatically scaling) cloud capacity, one must pay around $55.04/h for a DGX h100's capacity [161]. I have 9 of such systems in my config. Therefore, the fleet on cloud would cost me around $55.04×9 × 8 hours a day×261 days= $1.03M/y.

	Standby AI container (considering 36 days/y of in serving mode activity)
Price on-premise/y	As much as for the primary AI container instance, since I must own/rent year-round because the primary instance's downtime is mostly unpredictable (except for the one the company plans for certain manipulations).
Price on my hardware in rented rack space/y	As much as for the primary AI container instance, since I must own/rent year round because down times of the primary instance are mostly unpredictable (besides the one the company plans for certain manipulations).
Price on rented hardware and rented rack space/y	As much as for the primary AI container instance, since I must own/rent year-round because downtimes of the primary instance are mostly unpredictable (besides the one the company plans for certain manipulations).
Price on Cloud/y	For reminder, during the whole year except 36 days, it needs 620GB of storage and an OS, and for 36 days each year it needs the capacity of 9 NVIDIA DGX H100s. Therefore, the fleet on cloud would cost me around $55.04×9 × 8 hours a day×36 days + 1,070 of yearly storage for the dormant fleet [20] = $143,734/y. **This is the cheapest option by far.**

I do not calculate prices for Docker and FastAPI instances, as the Docker is likely thin[17] and each FastAPI instance follows the deployment solution of the system it serves, so it sits close to it.

[17] The Docker overhead primary instance and the standby instance both require 5 GB of disk storage, 1GBps of disk bandwidth, the equivalent of an AMD EPYC 9016 of 8 cores, and 5 GB of RAM.

So far, I voluntarily neglected a part of the Oracle DB deployment dimension, which is that purchasing the solution and deploying it are tied concerns from a financial point of view (as I am about to discuss).

Membership fees per deployment option:

I use an Oracle DB Enterprise Edition solution, notably for its security features and its Data Guard feature (as discussed in earlier ADRs). The options to purchase that service, are a Traditional License purchase for on-prem/self-deployment (which includes cloud deployments on other clouds than Oracle's), and *BYOL* (Bring Your Own License) and *License Included* solutions running on Oracle Cloud (OCl) infrastructure. This makes Oracle DB deployable on non-cloud solutions and PaaS, IaaS Cloud solutions, and accessible as a SaaS. If I choose an Oracle Cloud solution, it would be 'Oracle Base DB Service – Enterprise Edition', because it seems to match my needs, providing security across multiple physically distinctly stored DB instances, which work together (which is my case since I use Data Guard), and because it runs on Oracle's own Cloud infrastructure, unlike the other Cloud solution for Enterprise Edition, namely Cloud@Customer [39, 273].

Prices for the three purchasing options of that DB Edition are as follows [257, 274]:

Purchase Option	Purchase indexing metric	Enterprise Edition
Traditional License Purchase (only one metric to choose)	Named User Plus (NUP)	$950/user one time, plus $209/user/year (the latter being support fees)
	Processor , also called ECPU (what Oracle calculates as such based on equivalence tables [256])	$47,500/processor, plus $10,450/processor/year (the latter being support fees)
OCl BYOL (both rows apply together)	ECPU per hour	$0.0484
	Gigabyte storage capacity per month	$0.12
OCl License Included(both rows apply together)	ECPU per hour	$0.1075
	Gigabyte storage capacity per month	$0.12

So how much would each option really cost in my case? Let's start with the Traditional License purchase option for the primary DB instance and its direct backup storage. An NUP is any human user and device authorised to connect and manipulate data in the Oracle DB [163, 257]. In my case, this number should not exceed a hundred. But Oracle fixes the number of NUP at minimum 25 per

processor [205, p.42]. The number of processors is an Oracle-defined metric, calculated as the actual number of cores of the given processor model used, times an Oracle rate factor [256]. Oracle did not yet define how much Oracle processor equivalents AMD 9006 series (the one I used to measure the DB's needed capacity) equals in Oracle processor terms. However, the previous AMD generations had a 0.5 rate factor per CPU for this equivalence calculation [256]. I chose the most recent processor capacity example to index my DB's capacity on, but an equivalent 9005 series processor (in cores number) could probably do the job too, and the 64 cores 9005 and 9006 series processors have close characteristics [22, 24]. Therefore, I assume a 0.5 per CPU rate factor. Thus, we have 64 cores × 0.5 = 32 Processor Licenses for the DB primary instance (same for the standby instance when in serving mode). Thus, the primary DB instance's pricing is based on 32 × 25 NUP per processor = 800 NUPs. Thus, with the NUP indexing, the primary DB instance as a Traditional License purchase would cost a one-time $950 × 800 = $760,000, plus $209 ×800 = $167,200/year. With the processor indexing, the primary DB instance as a Traditional License purchase would then cost $47,500 × 32 = $1,520,000 one time, plus $10,450 × 32 = $334,400/year. This shows that, if I purchase a Traditional License, I should do so at the price based on NUP count. But this would just be the purchasing cost, neither the deployment cost of running the DB instance nor the deployment cost of running the direct backup storage.

The OCl option with BYOL requires paying the License of $760,000 (since I do not yet possess one) and the same yearly support fees of $167,200/y, plus at least $0.0484 per ECPU per hour, and since I have 32 ECPUs, and 261 × 16 hours (see hour count mentioned earlier in the price calculation section of the ADR), and knowing the primary DB instance requires 78,000 GB of inner storage plus 125,000 GB of direct backup storage space, this purchasing option would cost $0.0484 × 32 × 261 × 16 + 167,200 + $0.12 per GB of storage per month × 12 months × (125,000 + 78,000) = $465,988/y, plus an upfront cost of $760,000, but these $465,988 per year include purchasing and deployment expenses.

The License Included purchase + deployment would cost $0.1075 × 32 × 261 × 16 + 167,200 + $0.12 per GB of storage per month × 12 months × (125,000 + 78,000) = $473,885/y.

Now let's weigh these DB options' prices against each other. The cheapest eligible deployment of the primary DB instance, namely on my hardware in rented rack space, costs around $38,000/y, and the direct backup's similar deployment costs around 33k per year (around 100k for the three backups divided by three). With the yearly Oracle fees for a Traditional License purchase of $167,200/y (I do not amortise the upfront license cost within the yearly fees because a license is a long-term investment that can benefit other ventures), the total cost of Traditional self-deployment is 238,200/y. This is half of the yearly cost of the other Oracle purchasing options, and allows me 1) to keep the entire rack in the same physical cluster (as discussed earlier, connected by cables within the same rack pile or the same data centre) in a way so that the primary AI container instance, which is the priciest and most

decisive element, can be deployed in the cheapest way too. Therefore, the deployment of the primary program instance my hardware on rented rack space is still the most profitable by far.

We still have to calculate the most profitable option for the standby DB and its direct backup storage, and for the disaster DB backup. Let's start with the DB standby instance and its direct backup storage. For all year (minus 36 days), this instance needs 5.25 TB of RAM, 78 TB of disk storage with 11GBps of write throughput, 17.815 MB/s of read throughput, and the equivalent of a AMD EPYC 9256 CPU (24 cores). For 36 days a year, it needs the serving capacity. The Traditional License purchase would cost me just as much as the primary instance's, because I would buy one time the License for the serving capacity, and each year I also pay the same Oracle Support costs. The OCl option with BYOL requires paying the License of $760,000 (since I do not yet possess one) and the same yearly support fees of $167,200/y, plus $0.0484 per ECPU per hour, and $0.12 per GB of storage per month. That 24-cores processor capacity equals 12 ECPUs (24 × the 0.5 core factor). Thus, we have the same calculation as for the primary DB instance except with a 8 ECPUs config for all year minus 36 days, and then for 36 days the same config as the primary DB instance. That gives us $0.0484 × 12 × (261 – 36) × 16 + $0.0484 × 32 × 36 × 16 + 167,200 + $0.12 × 12 × (125,000 + 78,000) = around $462,503/y, plus the upfront one-time License purchase of $760,000. The License Included purchase + deployment would cost $0.1075 × 12 × (261 – 36) × 16 + $0.1075 × 32 × 36 × 16 + 167,200 + $0.12 × 12 × (125,000 + 78,000) = around $466,145/y with no upfront license costs. Here again, the traditional license purchase paired with on-rented-rack-on-my-won-hardware deployment is the most profitable option, because, I can then deploy it with the cheapest eligible option evoked earlier (namely on my rack on rented rack space, since it must be in a different physical location than the primary DB instance as evoked earlier) and that most profitable option would cost me $238,200/y, which is significantly under the around-$460k/y price for the other options.

Finally, deploying the disaster backup costs 125,000×12×0.12= $180,000/y with a License included Oracle Cloud or a BYOL Oracle Cloud deployment. This costs significantly over the expected near $33k/y deployment price of that backup if deployed on rented or owned rack in rented rack space (see earlier tables).

Therefore, the most profitable of the still-shortlisted deployment options is the same before and after considering the Oracle solution fees.

Consequences

First, I must fine-tune the number of Uvicorn workers manually, because otherwise the automatically set up number might and cause traffic issues [193].

Since some components are ideally deployed on dynamically-scaling Cloud, I must also choose such solution out of the available ones.

Also, since the user-input documents that the AI container uses are stored and managed by FastAPI, it should be a component within the FastAPI container in the c4 model diagrams, since a container must include all its responsibilities [54].

Also, each inference engine (1 for Qwen, one for BGE) needs its own separate vLLM engine [329]. And they can be containerised, and should be containerised, even though my AI container relies on more than one physical machine [393].

Also, now we know there is a single serving FastAPi instance at a time. This has consequences related to TLS termination. Nowadays, browser HTTP request encryption is TLS-based. “TLS termination is the process of decrypting TLS-encrypted HTTPS traffic (the browser encrypted [80]) at a specific point in the network, usually a load balancer or reverse proxy, before passing the request to backend servers” [146]. In my program, Cloudflare receives TLS-encrypted HTTP requests first (see Section1.5). For security purposes, it re-encrypts the requests before sending them to the FastAPI container. Because there is only a single serving FastAPI instance (see Section 5.8), and because Cloudflare and KeepAlived already direct signals the right way (see Sections 1.5, 5.7), there is no real need of load balancing between Cloudflare and FastAPI. Therefore, the FastAPI container must decrypt signals with a reverse proxy [126, 171]. Thus, I must choose that reverse proxy (see Section 5.11).

Finally, considering a yearly profit before expenses of $67,455,360 for the US (see Appendix 3), and yearly salaries expenses (around 10M dollars) + at least 3 million a year in legal and patent and marketing fees + deployment costs per year (38,000 × 2 + 101,000 + 596,839 + 143,734 +20,000 of Docker and FastAPI costs and KeepAlived deployment costs (max for the standby FastAPI instance on cloud)) + 167,200 × 2 for the Oracle fees per year (besides upfront purchase of the license) + Cloudflare and Stripe fees (note KeepAlived is open-source) which should not exceed the few millions + unforeseen costs (let’s say 2 millions) + yearly taxes (minimum 21% of income tax in the US plus the additional average 6.57% of tax on revenue in the US [219, 297], plus at least 10% of other taxes), the business expects around [67,455,360 - (10,000,000 + 3,000,000 + 38,000 × 2 + 101,000 + 596,839 + 143,734 +20,000 + 167,200 × 2 + 3,000,000 + 2,000,000)] × [1 - 0.21 - 0.0657] = (67,455,360 – 19,271,973) × 0.7243 = $34,899,227.2041/y of net profit for the forecasted user traffic.

Because the margin of error of the AI models obliges the user to proof-read the values the software used in its calculations (this info is all summarised in a user project summary file users can download), inducing end customers will probably still prefer to consult a human financial service provider. The latter would likely use the digital service to simplify its work for the complex cases, therefore making the total number of usage cycles close to at least 50% of what I forecasted. This would halve both the revenue and expenses (besides legal and marketing costs and taxes and salaries),

thus guaranteeing at least (67,455,360 ÷ 2 – (19,271,973÷2 + 5,000,000 + 1,500,000)) × 0.7243 = $12,741,663.6021 of annual net profit.

5.9. ADR: cloud provider

Date (dd/mm/yyyy): 05/09/2026

Status: Accepted.

Context: Some of my components are best deployed on dynamically-scaling cloud capacity (see previous ADR). I must choose a provider of such solution to deploy these on Cloud.

Options: I can use a private or public cloud, and if public cloud, there are multiple providers and 3 main types of solution to choose between (IaaS, PaaS, SaaS) which are all dynamically scalable [229, 407], 223, 228, 320].

Decision: I choose to rely on IBM Cloud.

Rationale:

People can create their own cloud, but it brings no scale or security (it is rather less safe) benefits over third-party providers' clouds (public Cloud).

Also, for cloud hosting, there are spot and on-demand options. Spot capacity is usually much cheaper than on-demand, but there is a trade-off: the cloud provider can reclaim those instances with little notice, and specific instance types may not always be available immediately [214]. So I would need on-demand cloud whenever I use it.

Google, IBM, Microsoft and AWS solutions seem to have equivalent advantages and provide all the features I require (inference endpoints number-based scaling from zero to peak number of inference points needed, security, standards, endpoint get both GPU and CPU capacity). I also read about other less known providers like RunPod, which also provide zero-to-peak elasticity based on scaling the number of inference endpoints providing both GPU and CPU capacity for the models and their engine, and providing the most common transparency and data protection standards for rather cheaper than the other solutions [325–328].

I do not need a global reach since the deployment is with country-based capacity just for the clients in the country. The price remains rather low no matter the provider since the backup AI container component runs only on average 36 days or less per year. So price should not be the main decisive factor. Security is much more important since the parsed info includes financial reports and contracts. And surrounding services (DNS, etc.) are not a strong requirement either since for networking and load balancing for failover I already found my opens source solutions. I cannot investigate it too deeply due to time constraints but I choose IBM Cloud for its finance industry high compliance

features, its FedRamp compliance [174], which matter a lot in my case, and because I can run specifically Nvidia GPUs and CPUs, which alleviates the need for CPU mediation [172, 175], and because I can run Linux VMs on IBM Cloud [388] and because they have data centres in the US, which respects my geographical storage requirements [95].

5.10. ADR: by-FastAPI/in-FastAPI TLS decryptor

Date (dd/mm/yyyy): 19/08/2026

Status: Accepted.

Context: I must choose the reverse proxy to use in my FastAPI container (see Section 5.8.'s *Consequences* subsection).

Options: Nginx or Traefik [126, 171].

Decision: I pick Traefik.

Rationale: Traefik supports enough HTTP requests for my deployment (it supports 45,000/s; I have a max of 1,070.72/s in my deployment), and is easier to set up than Nginx [9, 192]. I found no source stating that Traefik does not support my service-side Jinja2 rendering solution; thus, it seems compatible [9].

Consequences: Traefik shows as a component distinct from the app's C4 container, since it sits in front of the FastAPI app [126].

6. Codebase specifications

6.1. CA elements' classification by code form

Python code consists of functions and/or objects [30]. I here detail which I use and when, knowing that the goal is to achieve the lowest coupling and highest cohesion [140]:

CA / DDD Element	Python construct	Key reason
DDD Entity	class	has identity and mutable state [357, 360].
DDD Value Object	@dataclass(frozen=True) or Pydantic model	The element's attributes define it [23, part 1.1]. Element is immutable (immutability is not common to all data classes but can be set) [302].
DDD Aggregate Root	class (entity + enforcement)	Since, in DDD, the aggregate root controls its members, but these members can change over time: this is best represented as a class, because it makes the aggregate root an entity, and its members the internal mutable attributes [403].
Domain Service	class (stateless, injected deps)	a domain service has no state of its own, but it depends on repos or other services, which act as interfaces between domain and DB to ensure dependency injection [321]: - UserRegistrationService depends on UserRepo to check user info - user projectService depends on OracleUser projectRepo to load and save the User project aggregate , and of EventBus to save events - UniqueUserSessionPolicy depends on OracleUserSessionRepo to check whether an active session already exists for that user - UniqueUser projectRuntimePolicy depends on OracleUser projectRepo to check project status (if already opened somewhere) To write such dependencies cleanly, one must make the Domain service a class with the given other service/repo as a constructor injection [321].

CA / DDD Element	Python construct	Key reason
		Classes also give you named, testable units, which is also the right practice, since test-driven and test-driven is good practice [345].
Application Service / Use Case	class with a single public method,	Making the element a class makes it independently injectable, testable, and named. If no dependencies, a function works. Classes are for state-focused workflows, while functions suit better the action-focused code elements [360]. Moreover, “, testing is simpler when you use functions. Functions that don't modify the global state can be tested in isolation, making it easier to verify the correctness of each component. This leads to more reliable, testable code without the overhead of additional abstractions” [360]. However, my Application Business Rules all comprise multiple named, injectable, mockable dependencies preferably swapped in tests (e.g., testing the branch where get_input_id() returns nothing, vs. the branch where it returns something). This sort of use case requires controlling these dependencies, and since I will use dependency injection to ensure clean repo method calls via an interface, this justifies sticking to writing Application Business Rules as classes [360].
DDD repos	Protocol class or ABC class, or Duck typing	First, we know this must be an abstraction a method’s signature attaches to. In Python, this is expressed as a protocol or ABC class or with Duck typing because Python's type system has no other construct to attach method signatures to [23, part 1.2, 61].
Repo Implementation (Gateway)	class implementing Protocol (Its methods are the same signatures as the Protocol)	The element has state [360].
Domain Event	@dataclass(frozen=True) subclassing base or pydantic model	has No behaviour, no mutation [23, part 1.1].

CA / DDD Element	Python construct	Key reason
DomainEvent base class	class	metadata enforcement + serialization hook= its __init_subclass__ or to_dict() logic may need to be non-trivial [341].
Controller / Route	FastAPI APIRouter class + async def functions	Controllers control incoming HTTP requests and redirect them to the right orchestration code. Doing so, it depends on that recipient. In FastAPI, controller logic can live in a function, and dependencies stay attached to an APIRouter object. The advantage is that this single object can 'keep' the dependencies of multiple routers, so it is slightly configuration-time saving, without losing any other advantages. But the controller in my case must be async, because it waits on the I/O taking place with the use case querying the DB or the AI models (except for some minor HTTP requests), and that asynchronicity lives at a function, so that all satisfies async needs and routing configuration ease [10, 127, 303].
Presenter / Response Model	Pydantic BaseModel	That Pydantic model acts as final type and shared information validator before the information sends to the UI, and it also translates the response info from its FastAPI-friendly form to its UI-friendly form (serialisation in other words) [128].
Factory	class with static method, or plain def	My factories are: PaymentGatewayFactory. PaymentGatewayFactory relies on a Stripe Gateway. A Stripe gateway needs at minimum a Stripe API key and probably a webhook secret, which will come from either a settings object, environment variables, or a secrets manager [354]. This means the factory depends on other elements to configure its own role. StripeGateway will need an HTTP client to call Stripe's API. In a properly testable system, that HTTP client is injected rather than instantiated inside the gateway, so the factory receives it and passes it through. That is an injected dependency. Considering these dependencies of the factory, the latter shall be a class.

6.2. Design patterns and SOLID Principles

Computer science authors (most famously the ‘Gang of Four’) wrote ‘design patterns’, i.e., ‘ready-to-use’ solutions to solve a problem with cleanly designed code [27, 130, 401]. These patterns notably align with other well-known, more recent object-oriented programming principles, such as the SOLID principles [140]. I used these general design rules as follows.

First, in FinApp1.0 code, although I did not really plan it at first, there is no inheritance, only composition. And I want to be careful to keep it that way as I fine-tune the code. Why? Inheritance is defining objects (e.g. a Dog object) as a type of a more abstract object (e.g., an Animal object). Composition is defining objects (e.g. Dog object) by its attributes (e.g. Leg object). Both methods introduce coupling, but inheritance does so at compile time, whereas composition does so at runtime. The coupling composition introduces is looser than the coupling inheritance introduces, because an objects the composed object uses (e.g. Legs) is less susceptible to change, and if I want to change the composed object (Dog) I can do it without affecting the ‘components’ (Legs), while a superclass (animal) is more likely to change and changing it has effects on all its subclasses (dog, cat, snake, etc) [371].

Also, Application Business Rules must call a repo method via an interface. This dependency injection avoids that the app's business-rules layer directly depends on an outer layer’s element, and instead depends on an abstraction in the core layer of the CA (the interface), thereby enforcing the dependency inversion principle [340].

Also, a use case must call another use case via an anti-corruption layer (ACL), and neither is a child class of the other, nor is either a child class of a common parent (whether a common 3rd class or separate parents). The use case call to another use call acts as a way of ending/pausing the workflow in one context and starting/resuming workflow in another context (in my case, this means that use case of bounded context 1 either created bounded context 2's elements by calling bounded context 2 repo method interfaces create_instance create_input, or looked for a certain instance id of bounded context 2, and then simply calls use case of bounded context 2 to execute for instance_id = bounded context 2's newly created instance's id). This is a way to ensure delegation/orchestration of work between objects while respecting the principle that modules of a modular monolith (here, my bounded contexts) should not directly depend on each other to ensure low coupling.

I avoid global variables because they increase coupling [28, 310].

Also, the HTTP request-response system is based on, either one of the subtypes of the MVC (Model-View-Controller) pattern [60, 134]. It works as follows (and this is what I adopt):

- “1. The user interacts with the view.
- 2. The view creates a request (object) which is passed to the controller.

- 3. The controller converts the request into a request model and passes it to the use case interactor through its input port (the controller returns nothing to the presenter it only triggers a use case, and so even if the use case does nothing more than calling a repo method interface, the flow must go through the use case first).
- 4. The use case interactor processes the request model and creates a response model which is passed through the output port to the presenter.
- 5. The presenter converts the response model to view model which is then passed to the view.
- 6. The user sees the result of his interaction in the view" [290].

In Clean Architecture, the request-response system comprises a Use Case Output Port, a Use Case Input Port, a Use Case Interactor, a Response Model, a Request Model, found under the Application Business Rules layer, as well as the Controller and the Presenter, the latter two found under the interface adapters layer [222, 290]. The output port is 'the API that the app sends output through, for this particular use case' [60]. In Python, the input port is usually the execute() method signature [199]. The output port is a presenter. The interactor is the use case itself [290]. The request model is the object the controller constructs from the HTTP request and passes into execute(). It is a plain domain-friendly object, not a FastAPI request object. The Response model is the plain object the use case returns – not a FastAPI response, just a domain result. That response model is often separated from the request model. However, in a FastAPI project, having the controller both call the use case and format the HTTP response (without a separate presenter class) is a legitimate and widely used simplification [291]. So I use that simplification.

In terms of where the repo method abstraction should live, I found that if the repo method exists to enforce or evaluate a domain rule, the abstraction must be in ENTREPRISE BUSINESS RULES, because it must be in the same layer or an inner layer than the element consulted (here the domain rule in the most inner layer), and if the repo method exists to fetch data so the use case can decide what to do, the abstraction can be APPLICATION BUSINESS RULES [330, 404]. But note that it also makes sense to group all abstractions in the core layer, since it is easier to find them with my by-layer folder structure in my FastAPI app. So I group all repo method abstractions in the enterprise business rules layer.

Note the abstraction topic can be confusing. One might wonder, why in clean architecture, saying "a L2 component (app use case) who wants to use a L3 (interface adapters) repo method, must be first dependent on a abstraction in L1 (enterprise business rules)" is clean, because it makes it look like, eventually this L1 element must know of the L3 repo method to use that L3 repo method. But in fact, the L1 element does not know of the L3 element: only the L3 element is aware of contracts happening within the inner layers. And having this L3 element does not make the code more capacity-consuming than having no L3, because the L3 element is not like an active watcher in that sense: it only runs at runtime of the other elements.

Lastly, “every API call costs network latency (50–200ms minimum), connection overhead, TLS handshake (first request), serialisation/deserialisation, database roundtrip, and authentication/authorization check” [280]. So ideally, I need to reduce the number of API calls. One way to do so is to collapse more processes into fewer endpoints: for example, if 4 endpoints are used to access a given page instead of 6, the user has to make 4 clicks to access the page instead of 6, reducing API call costs [280]. Additionally, fewer API endpoints mean less maintenance, while the backend produces the same results [280, 378]. So I keep the number of endpoints to a minimum.

6.3. UV as dependency manager

The project needs a Python dependency manager to ensure quality of migrations and potential access by collaborators [34]. Multiple managers exist: pip + pip-tools, Poetry, Pipenv, PDM, UV, Conda. I choose UV (Astral) for the following reasons:

- Significantly faster resolution and installation than pip/Poetry,
- uv.lock provides deterministic, cross-platform lockfile,
- Native virtual environment management,
- Compatible with pyproject.toml (PEP 517/518 standard),
- simplest and most advanced solution [32, 34].
- I use Docker containers (see deployment ADRs) and uv is compatible with that technology, and lives within the FastAPI container [37].

6.4. DB queries code

My program must contain queries to communicate with the database. FastAPI allows querying tables either using raw SQL (in code or in stored procedures), an ORM (ORM models can be combined with Pydantic models, but not necessarily), or a SQL query builder like the non-ORM SQLAlchemy [29].

I choose to write my queries as raw SQL. Also, the SQL text for each query must include placeholders, and the Python code that sends the query must specify the value the placeholder takes. Here is why.

The software relies on static queries [173]. My queries are mostly process-state management queries (ensuring workflow execution), with a minority of entity-state management queries (managing the lifecycle of objects such as users, user projects, etc.) [29, 31]. My queries are almost all very simple insert, update, and delete commands. The statements contain lookups, joins, filtering, JSON construction, and existence tests. Often, these require additional relational queries because the relevant data is not in memory. The Insert and Update statements all share the same “methodology”: besides

the primary key of the new/to update row, they find in other tables the values to put in that new/to update row.

I use Pydantic models, but they differ from ORM object mappings, so I don't need ORM object mappings to use Pydantic [36]. It is possible, however, to combine data validation, serialisation, and object mapping for an ORM with SQLModel [31, 347]. But eventually, with an SQLModel ORM-validation combination, or with the ORM alone, I might have fewer query lines, but I would lose the speed of raw SQL or stored procedures [209]. I need to prioritise speed in the querying system because my program is so I/O-bound, and because I/O is not only the main inter-requests bottleneck but also occurs in large quantities even within a single HTTP request-response. Securitisation against SQL injection is a concern, but since my queries are rather straightforward, SQL injection security will be as well [29].

My queries do not contain looping mechanisms or other elements that can only exist in the SQL dialect of my DB; all can exist as ANSI SQL [94]. Keeping raw SQL in-code queries avoids DB-app navigation just to troubleshoot my code: all is in one place in the app.

FastAPI allows me to use an ORM and in-code raw SQL queries and stored procedures in the same project [129]. This means I can always switch to an ORM system later on if needed.

I use an Oracle DB. Oracle keeps parsed SQL statements and their execution plans in memory in the library cache, which is part of the shared pool (SGA). "When an SQL clause comes into the Oracle database, Oracle first checks whether it has been executed or not by looking at the library cache. If the query has been executed before, Oracle uses the previous execution plan without parsing this SQL. This process is called soft parse. If the query has not been executed before, Oracle will parse the executed SQL and save it to the Shared SQL Area in the library cache. This process is called a hard parse" [104]. If a query comes back in the DB with variable values concatenated inside it, it always forces a hard parse, whereas if the variables come separately, the DB only needs a soft parse. The soft parse is less cache- and hardware-intensive. Thus, to ensure shorter, lighter query execution, I must ensure that Oracle receives each request with the passed values sent separately rather than enclosed within it. To pass the value separately (this is called passing the value as a bind variable)**,** the SQL text of each query must contain placeholders, and the Python code sending that query must specify the value for each placeholder. This ensures the Python-DB driver understands these are two different elements, and sends them as such [104, 251].

For my program, a clean way to use raw SQL queries is with the '*cursor.execute'* method, which ensures that the query is executed with specific parameter values only once per call, which is what the use cases require (unlike cursor.executemany or others) [252].

Bibliography

[1] L. Ashdown A. Agrawal and Mark Doran. Managing the Redo Log. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/admin/managing-the-redo-log.html

[2] Khalid Abuhakmeh. 2025. OAuth 2.0 vs OpenID Connect: What Is the Difference? *Duende Software*. Retrieved September 9, 2026 from https://duendesoftware.com/learn/the-differences-between-oauth-2-0-and-openid-connect-and-why-they-matter

[3] Nipun Agarwal, Nigel Bayliss, Neerja Bhatt, Wilson Chan, Sumanta Chatterjee, Eugene Chong, Bruce Ernst, Thuvan Hoang, Mark Johnson, Jonathan Klein, Sushil Kumar, Tirthankar Lahiri, Bill Lee, Yunrui Li, Juan Loaiza, Rich Long, Catherine Luu, Neil MacNaughton, Gopal Mulagund, Subramanian Muralidhar, Ravi Murthy, Kant Patel, Mark Ramacher, Suvendu Ray, Debbie Steiner, Malai Stalin, Seema Sundara, Murali Thiyagarajan, Alex Tsukerman, Kothanda Umamageswaran, Steve Vivian, Eric Voss, Tak Wang, Steve Wertheimer, Andy Witkowski, Daniel Wong, Brian Wright, Qin Yu, Mohamed Ziauddin, and Mary Beth Roeser. Statistics Descriptions. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/21/refrn/statistics-descriptions-2.html?embed=1#GUID-2FBC1B7E-9123-41DD-8178-96176260A639

[4] Agents Hub. 2026. Bare Minimum Setup You Need to Run AI Agents Locally. *YouTube*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=77FU71SzaUM

[5] Akash. Akash Network - Decentralized Compute Marketplace. *Akash*. Retrieved September 6, 2026 from https://akash.network/

[6] Rubén Alapont. 2023. Domain Services and Factories in Domain-Driven Design. *DEV Community*. Retrieved September 4, 2026 from https://dev.to/ruben_alapont/domain-services-and-factories-in-domain-driven-design-55lo

[7] Asad Ali. 2025. Domain-Driven Design (DDD). *GeeksforGeeks*. Retrieved March 5, 2026 from https://www.geeksforgeeks.org/system-design/domain-driven-design-ddd/

[8] Jim Allen Wallace. 2026. Prefill vs Decode: LLM Inference Phases Explained. *Redis*. Retrieved September 6, 2026 from https://redis.io/blog/prefill-vs-decode/

[9] Open Source Alternatives. Traefik vs Nginx: Which Reverse Proxy Wins in 2026? *Open Source Alternatives*. Retrieved September 5, 2026 from https://www.opensourcealternatives.to/blog/traefik-vs-nginx

[10] Jeremy Alva. 2024. FastAPI Backend Architecture: Model-Controller-Service. *Medium*. Retrieved September 4, 2026 from https://medium.com/@jeremyalvax/fastapi-backend-architecture-model-controller-service-44e920567699

[11] Amazon. GODO 3.5 inch Dual Bay Hard Drive Enclosure,USB3.0 to 3.5" SATA HDDs External Enclosure,RAID 0/1/JBOD/PM Mode,Max 40TB RAID Hard Drive Docking Station,Aluminum: Amazon.co.uk: Computers & Accessories. *Amazon*. Retrieved September 6, 2026 from https://www.amazon.co.uk/GODO-3-5-Enclosure-SATA-External-Black/dp/B0CRH8TSJ7?th=1

[12] Amazon Web Services, Inc. OLTP vs OLAP - Difference Between Data Processing Systems. *Amazon Web Services, Inc.* Retrieved September 4, 2026 from https://aws.amazon.com/compare/the-difference-between-olap-and-oltp/

[13] Amazon Web Services, Inc. Cloud Email Sending Service - Amazon Simple Email Service. *Amazon Web Services, Inc.* Retrieved September 12, 2026 from https://aws.amazon.com/ses/

[14] Amazon Web Services, Inc. What is Embedding? - Embeddings in Machine Learning Explained - AWS. *Amazon Web Services, Inc.* Retrieved September 4, 2026 from https://aws.amazon.com/what-is/embeddings-in-machine-learning/

[15] Amazon Web Services, Inc. How Elastic Load Balancing works - Elastic Load Balancing. *AWS Documentation*. Retrieved September 5, 2026 from https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/how-elastic-load-balancing-works.html

[16] Amazon Web Services, Inc. What is Amazon Route 53? *AWS Documentation*. Retrieved September 5, 2026 from https://docs.aws.amazon.com/en_en/Route53/latest/DeveloperGuide/Welcome.html

[17] Amazon Web Services, Inc. Public Cloud vs Private Cloud - Difference Between Computing Environments - AWS. *Amazon Web Services, Inc.* Retrieved September 6, 2026 from https://aws.amazon.com/compare/the-difference-between-public-cloud-and-private-cloud/

[18] Amazon Web Services, Inc. Amazon EC2 instance network bandwidth - Amazon Elastic Compute Cloud. *Amazon Web Services, Inc.* Retrieved September 6, 2026 from

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-network-bandwidth.html

[19] Amazon Web Services, Inc. Kubernetes vs Docker - Difference Between Container Technologies. *Amazon Web Services, Inc.* Retrieved September 5, 2026 from https://aws.amazon.com/compare/the-difference-between-kubernetes-and-docker/

[20] Amazon Web Services, Inc. Comparing Amazon EBS volume types gp2 and gp3 - Amazon EMR. *Amazon Web Services, Inc.* Retrieved September 6, 2026 from https://docs.aws.amazon.com/emr/latest/ManagementGuide/emr-plan-storage-compare-volume-types.html

[21] AMD. AMD EPYC™ 9006 Server CPUs for AI-First Data Centers. *AMD*. Retrieved September 5, 2026 from https://www.amd.com/en/products/processors/server/epyc/9006-series.html

[22] AMD. AMD EPYC™ 9556. *AMD*. Retrieved September 22, 2026 from https://www.amd.com/en/products/processors/server/epyc/9006-series/amd-epyc-9556.html

[23] AMD. AMD EPYC™ 9256. *AMD*. Retrieved September 5, 2026 from https://www.amd.com/en/products/processors/server/epyc/9006-series/amd-epyc-9256.html

[24] AMD. AMD EPYC™ 9555. *AMD*. Retrieved September 22, 2026 from https://www.amd.com/en/products/processors/server/epyc/9005-series/amd-epyc-9555.html

[25] Rene Antunez. 2025. René Antúnez | What is an Online Redo Log? : A Back-To-Basics Guide. Retrieved September 5, 2026 from https://rene-ace.com/what-is-an-online-redo-log-a-back-to-basics-guide/

[26] Arista. 202AD. 7060X4 Series 100/200/400G Data Center Switches. Retrieved from https://www.arista.com/assets/data/pdf/Datasheets/7060X4-Datasheet.pdf

[27] ArjanCodes. 2021. The Strategy Pattern: Write BETTER PYTHON CODE Part 3. *YouTube*. Retrieved March 2, 2026 from https://www.youtube.com/watch?v=WQ8bNdxREHU

[28] ArjanCodes. 2023. What is Coupling and How Can You Avoid It? *YouTube*. Retrieved February 26, 2026 from https://www.youtube.com/shorts/XJGe6fP7Bco

[29] ArjanCodes. 2023. Raw SQL, SQL Query Builder, or ORM? *YouTube*. Retrieved February 19, 2026 from https://www.youtube.com/watch?v=x1fCJ7sUXCM

[30] ArjanCodes. 2023. Why Use Design Patterns When Python Has Functions? *YouTube*. Retrieved March 2, 2026 from https://www.youtube.com/watch?v=vzTrLpxPF54

[31] ArjanCodes. 2024. SQLModel + FastAPI: Say Goodbye to Repetitive Database Code. *YouTube*. Retrieved February 17, 2026 from https://www.youtube.com/watch?v=GONyd0CUrPc

[32] ArjanCodes. 2024. This Tool Replaces pip, Poetry, pyenv, and More (It's Fast). *YouTube*. Retrieved February 6, 2026 from https://www.youtube.com/watch?v=qh98qOND6MI

[33] ArjanCodes. 2025. Why Pydantic Is a Must for Data Validation. *YouTube*. Retrieved from https://youtube.com/shorts/H6SeIkCybts?si=zw4akAI6BfBtBfSl

[34] ArjanCodes. 2025. UV Just Got A Serious Upgrade. *YouTube*. Retrieved February 6, 2026 from https://www.youtube.com/watch?v=CJJzrtHeLGI

[35] Lance Ashdown, Donna Keesling, Tom Kyte, Drew Adams, Ashish Agrawal, Troy Anthony, Vikas Arora, Jagan Athraya, David Austin, Thomas Baby, Vladimir Barriere, Hermann Baer, Srinagesh Battula, Nigel Bayliss, Tammy Bednar, Virginia Beecher, Bjorn Bolltoft, Ashmita Bose, David Brower, Larry Carpenter, Lakshminaray Chidambaran, Deba Chatterjee, Shasank Chavan, Tim Chien, Gregg Christman, Bernard Clouse, Maria Colgan, Carol Colrain, Nelson Corcoran, Michael Coulter, Jonathan Creighton, Judith D'Addieco, Mark Dilman, Kurt Engeleiter, Bjørn Engsig, Marcus Fallon, Steve Fogel, Jonathan Giloni, Naveen Gopal, Bill Habeck, Min-Hank Ho, Lijie Heng, Bill Hodak, Yong Hu, Pat Huey, Praveen Kumar Tupati Jaganath, Sanket Jain, Prakash Jashnani, Caroline Johnston, Shantanu Joshi, Jesse Kamp, Vikram Kapoor, Feroz Khan, Jonathan Klein, Andre Kruglikov, Sachin Kulkarni, Surinder Kumar, Paul Lane, Adam Lee, Allison Lee, Jaebock Lee, Sue Lee, Teck Hua Lee, Yunrui Li, Ilya Listvinski, Bryn Llewellyn, Rich Long, Barb Lundhild, Neil Macnaughton, Vineet Marwah, Susan Mavris, Bob McGuirk, Joseph Meeks, Mughees Minhas, Sheila Moore, Valarie Moore, Gopal Mulagund, Charles Murray, Kevin Neel, Sue Pelski, Raymond Pfau, Gregory Pongracz, Vivek Raja, Ashish Ray, Bert Rich, Kathy Rich, Andy Rivenes, Scott Rotondo, Vivian Schupmann, Venkat Senaptai, Shrikanth Shankar, Prashanth Shanthaveerappa, Cathy Shea, Susan Shepard, Kam Shergill, Mike Skarpelos, Sachin Sonawane, James Spiller, Suresh Sridharan, Jim Stenoish, Janet Stern, Rich Strohm, Roy Swonger, Kamal Tbeileh, Juan Tellez, Ravi Thammaiah, Lawrence To, Tomohiro Ueda, Randy Urbano, Badhri Varanasi, Nick Wagner, Steve Wertheimer, Patrick Wheeler, Doug Williams, James Williams, Andrew Witkowski, Daniel Wong, Hailing Yu, and Mark Doran. Memory Architecture. *Oracle Help Center*. Retrieved September 5, 2026 from

https://docs.oracle.com/en/database/oracle/oracle-database/21/cncpt/memory-architecture.html

[36] Ask Solem & contributors. Configuration and defaults. *Celery 5.6.2 Documentation*. Retrieved January 30, 2026 from https://docs.celeryq.dev/en/main/userguide/configuration.html

[37] Astral. Using uv with FastAPI. *Astral*. Retrieved September 5, 2026 from https://docs.astral.sh/uv/guides/integration/fastapi/

[38] Salman Asu. 2024. comparing Domain-Driven Design (DDD) and Clean Architecture. *DEV Community*. Retrieved March 6, 2026 from https://dev.to/sallbro/comparing-domain-driven-design-ddd-and-clean-architecture-8d

[39] Michaël Aussems. 2026. Oracle launches Cloud@Customer for smaller local AI databases. *ITdaily*. Retrieved September 9, 2026 from https://itdaily.com/news/datacenter/oracle-base-database-cloud-customer/

[40] Auth0. JSON Web Token Introduction. *JSON Web Tokens*. Retrieved September 16, 2026 from https://jwt.io/introduction

[41] Andrew Babb, Donna Carver, Tim Chien, Sean Connolly, Donna Cooksey, Bill Fischer, Mahesh Girkar, Ray Guzman, Dah-Yoh Lim, Colin McGregor, Kant Patel, Chris Plakyda, Padmaja Potineni, Kathy Rich, Jony Safi, Toru Sasaki, Lawrence To, Randy Urbano, and Steven Wertheimer. Oracle Database - Release 23.1 - 1 Introduction to Recovery Appliance. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/engineered-systems/zero-data-loss-recovery-appliance/23.1/amagd/introduction-recovery-appliance.html?source=%3Aow%3Alp%3Acpo%3A%3A%3A%3ARC_WWSA210218P00008%3ALPD400146646

[42] Hermann Baer, Bjorn Bolltoft, Ali Cakmak, Sunil Chakkappen, Immanuel Chan, Deba Chatterjee, Chris Chiappa, Dinesh Das, Kurt Engeleiter, Leonidas Galanis, William Endress, Marcus Fallen, Bruce Golbus, Katsumi Inoue, Praveen Kumar Tupati Jaganath, Mark Jefferys, Shantanu Joshi, Adam Kociubes, Keith Laker, Allison Lee, Sue Lee, Cheng Li, David McDermid, Colin McGregor, Ajit Mylavarapu, Ted Persky, Lei Sheng, Ekrem Soylemez, Hong Su, Murali Thiyagarajah, Randy Urbano, Sahil Vazirani, Bharath Venkatakrishnan, Hailing Yu, John Zimmerman, Frederick Kush, and Lance Ashdown. Reading Execution Plans. *Oracle Help Center*. Retrieved September 5, 2026 from

https://docs.oracle.com/en/database/oracle/oracle-database/18/tgsql/reading-execution-plans.html

[43] Hermann Baer, Bjorn Bolltoft, Ali Cakmak, Sunil Chakkappen, Immanuel Chan, Deba Chatterjee, Chris Chiappa, Dinesh Das, Kurt Engeleiter, Leonidas Galanis, William Endress, Marcus Fallen, Bruce Golbus, Katsumi Inoue, Mark Jefferys, Shantanu Joshi, Adam Kociubes, Praveen Kumar Tupati Jaganath, Keith Laker, Allison Lee, Sue Lee, Cheng Li, Jiakun Li, David McDermid, Colin McGregor, Ajit Mylavarapu, Kantikiran Pasupuleti, Ted Persky, Palash Sharma, Lei Sheng, Ekrem Soylemez, Hong Su, Murali Thiyagarajah, Randy Urbano, Satya Valluri, Sahil Vazirani, Bharath Venkatakrishnan, Hailing Yu, Yuying Zhang, John Zimmerman, and Lance Ashdown. Query Optimizer Concepts. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/21/tgsql/query-optimizer-concepts.html

[44] Vijay Balebail. 2025. OracleDB Python Connections to Autonomous DB: Thick vs Thin Mode Explained. *Medium*. Retrieved September 5, 2026 from https://medium.com/@vbalebai/oracledb-python-connections-to-autonomous-db-thick-vs-thin-mode-explained-7e6146ff3001

[45] Michael Bayer. Asynchronous I/O (asyncio) — SQLAlchemy 2.0 Documentation. *SQLAlchemy*. Retrieved September 7, 2026 from https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html

[46] Beijing Academy of Artificial Intelligence. 2024. config.json · BAAI/bge-m3 at main. *Hugging Face*. Retrieved September 23, 2026 from https://huggingface.co/BAAI/bge-m3/blob/main/config.json

[47] Beijing Academy of Artificial Intelligence. 2026. BAAI/bge-m3. *Hugging Face*. Retrieved September 4, 2026 from https://huggingface.co/BAAI/bge-m3

[48] Beijing Academy of Artificial Intelligence. BGE-M3 — BGE documentation. *BGE*. Retrieved September 5, 2026 from https://bge-model.com/bge/bge_m3.html

[49] B&H. Seagate 24TB Exos X24 7200 rpm SATA III 6 Gb/s 3.5" Internal HDD (OEM). *B&H*. Retrieved September 6, 2026 from https://www.bhphotovideo.com/c/product/1806048-REG/seagate_st24000nm002h_24tb_exos_x24_3_5.html

[50] BinaryPatrick. 2025. Restart Unhealthy Docker Containers Automatically. *DEV Community*. Retrieved September 5, 2026 from https://dev.to/binarypatrick/restart-unhealthy-docker-containers-automatically-4c00

[51] Bristol Centre for Supercomputing Documentation. Distributed vLLM Inference. *Bristol Centre for Supercomputing Documentation*. Retrieved September 26, 2026 from https://docs.isambard.ac.uk/user-documentation/tutorials/distributed-inference/

[52] Philip Brown. 2014. What are Factories in Domain Driven Design? | Culttt. *Cult3*. Retrieved March 6, 2026 from https://culttt.com/2014/12/24/factories-domain-driven-design

[53] Simon Brown. c4 model. *C4 model*. Retrieved September 4, 2026 from https://c4model.com/

[54] Simon Brown. Container diagram. *C4 model*. Retrieved September 4, 2026 from https://c4model.com/diagrams/container

[55] Simon Brown. Component. *C4 model*. Retrieved September 4, 2026 from https://c4model.com/abstractions/component

[56] Jason Brownlee. 2023. Asyncio I/O Libraries. *Super Fast Python*. Retrieved February 5, 2026 from https://superfastpython.com/asyncio-io-libraries/

[57] Ian Buchanan. Containers vs Virtual Machines. *Atlassian*. Retrieved September 6, 2026 from https://www.atlassian.com/microservices/cloud-computing/containers-vs-vms

[58] bugfree.ai. JWT vs Session-Based Auth: Pros and Cons - System Design Interview Guide. *bugfree.ai*. Retrieved February 21, 2026 from https://www.bugfree.ai/knowledge-hub/jwt-vs-session-based-auth-pros-and-cons

[59] Kushal Byatnal. 2026. Agentic Document Extraction Guide March 2026. *Extend*. Retrieved September 4, 2026 from https://www.extend.ai/resources/agentic-document-extraction-complete-guide

[60] candied_orange. 2017. Answer to “Clean Architecture: Use case containing the presenter or returning data?” *Software Engineering Stack Exchange*. Retrieved September 5, 2026 from https://softwareengineering.stackexchange.com/a/357066

[61] CaptainDNS. 2026. DNS Propagation Time: How Long Does It Really Take? *CaptainDNS*. Retrieved September 5, 2026 from https://www.captaindns.com/en/blog/dns-propagation-time

[62] Alexandre Cassen. Quick Start. *KeepAlived*. Retrieved September 5, 2026 from https://www.keepalived.org/documentation/user-guide/quick-start/

[63] CatchPoint. Web API vs. REST API: Comparing RESTful and Non-RESTful Web APIs. *CatchPoint*. Retrieved February 19, 2026 from https://www.catchpoint.com/api-monitoring-tools/web-api-vs-rest-api

[64] Catio. 2026. Architecture Decision Records (ADRs): The 2026 Guide. *Catio*. Retrieved September 4, 2026 from https://www.catio.tech/blog/architecture-decision-record

[65] CCIR. 2026. Rent and MSRP: Five Generations of Posted Prices. *CCIR*. Retrieved September 27, 2026 from https://ccir.io/research/rent-and-msrp

[66] ch9hn. [Installation]: Reduce Image size when installing wheel with cuda 11.8 · Issue #4950 · vllm-project/vllm. *GitHub*. Retrieved September 24, 2026 from https://github.com/vllm-project/vllm/issues/4950

[67] Magomed Chatuev. 2020. DDD for microservices. *Medium*. Retrieved April 17, 2026 from https://medium.com/@chatuev/ddd-for-microservices-4778a363c071

[68] Joseph Chen. 2025. Understanding Hard Drive Bandwidth: Boost Performance Now. *ULINK DA Drive Analyzer*. Retrieved September 5, 2026 from https://ulink-da.com/hard-drive-bandwidth/

[69] Kevin Closson. 2006. DBWR Multiblock Writes? Yes, Indeed! *Kevin Closson's Blog: Platforms, Databases and Storage*. Retrieved September 5, 2026 from https://kevinclosson.net/2006/12/01/dbwr-multiblock-writes-yes-indeed/

[70] Kevin Closson. 2012. SLOB 2. Retrieved from https://kevinclosson.net/wp-content/uploads/2013/12/slob2_readme-2012-05-04.pdf

[71] Kevin Closson. 2014. SLOB Deployment – A Picture Tutorial. *Kevin Closson's Blog: Platforms, Databases and Storage*. Retrieved September 5, 2026 from https://kevinclosson.net/2014/08/04/slob-deployment-a-picture-tutorial/

[72] Cloudflare. 2026. Securely deliver applications with Cloudflare. *Cloudflare Docs*. Retrieved September 4, 2026 from https://developers.cloudflare.com/reference-architecture/design-guides/secure-application-delivery/

[73] Cloudflare. 2026. Quickstart. *Cloudflare Docs*. Retrieved September 5, 2026 from https://developers.cloudflare.com/load-balancing/get-started/quickstart/

[74] Cloudflare. 2026. Geo. *Cloudflare Docs*. Retrieved September 4, 2026 from https://developers.cloudflare.com/load-balancing/understand-basics/traffic-steering/steering-policies/geo-steering/

[75] Cloudflare. 2026. Rate limiting parameters. *Cloudflare Docs*. Retrieved September 4, 2026 from https://developers.cloudflare.com/waf/rate-limiting-rules/parameters/

[76] Cloudflare. Cloudflare Bot Management - Stop Bad Bots. *Cloudflare*. Retrieved September 4, 2026 from https://www.cloudflare.com/products/bot-mitigation/

[77] Cloudflare. Web Security Platform - WAF, DDoS Protection, Bot Management. *Cloudflare*. Retrieved September 4, 2026 from https://www.cloudflare.com/solutions/security/

[78] Cloudflare. Cloudflare Rate Limiting - API Abuse Prevention. *Cloudflare*. Retrieved September 4, 2026 from https://www.cloudflare.com/products/rate-limiting/

[79] Cloudflare. What is DNS? *Cloudflare*. Retrieved September 5, 2026 from https://www.cloudflare.com/learning/dns/what-is-dns/

[80] Cloudflare. What Happens in a TLS Handshake? | SSL Handshake. *Cloudflare*. Retrieved September 5, 2026 from https://www.cloudflare.com/learning/ssl/what-happens-in-a-tls-handshake/

[81] CloudOptimo. Compare u7i-8tb.112xlarge vs u7i-6tb.112xlarge. *CloudOptimo*. Retrieved September 27, 2026 from https://costcalc.cloudoptimo.com/aws-pricing-calculator/ec2/compare/u7i-8tb.112xlarge-vs-u7i-6tb.112xlarge

[82] clydiecom. 2013. Swingbench Tutorial for Oracle 12c Load Testing and Benchmarking. *YouTube*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=bovbie3kGGw

[83] CMS CyberGeek. 2023. Federal Risk and Authorization Management Program (FedRAMP). *CMS CyberGeek*. Retrieved September 5, 2026 from https://security.cms.gov/learn/fedramp

[84] CodeOpinion. 2025. Hexagonal vs Clean vs Onion Architecture... It Doesn’t Matter. *YouTube*. Retrieved February 26, 2026 from https://www.youtube.com/watch?v=Thzz5bvUI40

[85] Codespace. 2025. How Bare Metal is different from Cloud Computing and On Premise Servers. *YouTube*. Retrieved September 6, 2026 from https://www.youtube.com/shorts/5rlwc-CaStk

[86] ColoByTheU. 1U Colocation Pricing | $125/mo | No Contracts. *ColoByTheU*. Retrieved September 6, 2026 from https://colobytheu.us/pricing/

[87] Coloprice. The global colocation pricing & capacity index. *Coloprice*. Retrieved September 6, 2026 from https://coloprice.com/

[88] Connection. Seagate 24TB Exos X24 SAS 12Gb s 3.5" Internal Hard Drive (ST24000NM007H). *Connection*. Retrieved September 6, 2026 from https://www.connection.com/product/seagate-24tb-exos-x24-sas-12gb-s-3.5-internal-hard-drive/st24000nm007h/41738002

[89] Connection. QNAP TL-R2400PES-RP PCIe Interface 24-Bay JBOD Enclosure (TL-R2400PES-RP-US). *Connection*. Retrieved September 6, 2026 from https://www.connection.com/product/qnap-tl-r2400pes-rp-pcie-interface-24-bay-jbod-enclosure/tl-r2400pes-rp-us/41732624

[90] Thomas Coughlin. 2026. SSD Storage Capacity Prices Are Over 20 Times HDD Storage Capacity Prices. *Yahoo Finance*. Retrieved September 5, 2026 from https://uk.finance.yahoo.com/news/ssd-storage-capacity-prices-over-200117036.html

[91] Dikhyant Krishna Dalai. 2025. FastAPI Performance Bottlenecks: Why Middleware and ORMs Kill Throughput (and How to Fix Them). *Medium*. Retrieved September 6, 2026 from https://medium.com/@dikhyantkrishnadalai/fastapi-performance-bottlenecks-why-middleware-and-orms-kill-throughput-and-how-to-fix-them-a79924bfaebb

[92] Filip Danić. 2026. Do You Really Need Redis? When is an In-Memory Cache Enough? - Filip Danić. *Filip Danić's Website*. Retrieved September 4, 2026 from https://danicfilip.com/blog/do-you-really-need-redis/

[93] Erik Darling. 2021. I/O Bound vs CPU Bound Queries In SQL Server | Darling Data. *Darling Data*. Retrieved September 4, 2026 from https://erikdarling.com/i-o-bound-vs-cpu-bound-queries/

[94] Database Star. 2022. Stored Procedures are Better than Raw SQL. *YouTube*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=hTLW5fKbMCo

[95] Datacenters.com. IBM Cloud: United States Data Centers - Providers Map in United States. *Datacenters.com*. Retrieved September 15, 2026 from https://www.datacenters.com/providers/ibm-cloud/locations/united-states

[96] Dataprise. 2023. SSDs vs. Spinning Disks: Which Storage Solution is Right for You? *Dataprise*. Retrieved September 5, 2026 from https://www.dataprise.com/resources/blog/ssds-vs-spinning-disks/

[97] Davidism. 2025. jinja/docs/intro.rst at main · pallets/jinja. Retrieved September 4, 2026 from https://github.com/pallets/jinja/blob/main/docs/intro.rst

[98] A. Jesse Jiryu Davis. 2022. Why Should Async Get All The Love?: Advanced Control Flow With Threads. *Empty Square*. Retrieved February 2, 2026 from https://emptysqua.re/blog/why-should-async-get-all-the-love/

[99] Parth Rajesh Dedhia. 2025. Batch processing at Scale: How PostgreSQL outperformed BigQuery for us. *Data Science Collective*. Retrieved September 4, 2026 from https://medium.com/data-science-collective/batch-processing-at-scale-how-postgresql-outperformed-bigquery-for-us-34b00c572039

[100] Demos. JSON dummy data. *Demos*. Retrieved September 5, 2026 from https://microsoftedge.github.io/Demos/json-dummy-data/

[101] Agilista Desenvolvedor. 2026. FastAPI in Production: From the Default Framework to the Edge of Performance and a Fair Comparison…. *Medium*. Retrieved September 6, 2026 from https://medium.com/@agilista.desenvolvedor/fastapi-in-production-from-the-default-framework-to-the-edge-of-performance-and-a-fair-comparison-7d00524379b3

[102] Design Gurus. Which type of concurrency is best for CPU-bound programs? *Design Gurus*. Retrieved February 1, 2026 from https://www.designgurus.io/answers/detail/which-type-of-concurrency-is-best-for-cpu-bound-programs

[103] Devart. 2025. Connect to Oracle using Python. *Devart*. Retrieved February 5, 2026 from https://docs.devart.com/odbc-driver-for-oracle/using-from-programming-languages/use-the-driver-from-python.html

[104] Mehmet Salih Deveci. 2021. Hard Parse vs. Soft Parse and Bind Variable in Oracle Database. *IT Tutorial*. Retrieved September 7, 2026 from https://ittutorial.org/hard-parse-vs-soft-parse-and-bind-variable-in-oracle-database/

[105] Nawaz Dhandala. 2025. How to Implement Rate Limiting in FastAPI Without External Services. *OneUptime*. Retrieved September 4, 2026 from https://oneuptime.com/blog/post/2025-01-06-fastapi-rate-limiting/view

[106] Nawaz Dhandala. 2026. How to Run Keepalived in Docker for Virtual IP Failover. *OneUptime*. Retrieved September 12, 2026 from https://oneuptime.com/blog/post/2026-02-08-how-to-run-keepalived-in-docker-for-virtual-ip-failover/view

[107] Cem Dilmegani. 2026. GPU Marketplace: Vast.ai vs Shadeform vs Prime Intellect. *AIMultiple*. Retrieved September 6, 2026 from https://aimultiple.com/gpu-marketplace

[108] DNS Made Easy. 2025. REST APIs Explained: How They Work and Why They're Essential. *DNS Made Easy*. Retrieved February 21, 2026 from https://dnsmadeeasy.com/resources/rest-apis-explained-how-they-work-and-why-theyre-essential

[109] Docker. Resource constraints. *Docker Documentation*. Retrieved September 5, 2026 from https://docs.docker.com/engine/containers/resource_constraints/

[110] Docker. Use containerized databases. *Docker Documentation*. Retrieved September 6, 2026 from https://docs.docker.com/guides/databases/

[111] Docker. Use containers to Build, Share and Run your applications. *Docker*. Retrieved September 8, 2026 from https://www.docker.com/resources/what-container/

[112] Sudha Duraiswamy, Michael Hwa, Sudha Iyer, Supriya Kalyanasundaram, Lakshmi Kethana, Peter Knaggs, Andrew Koyfman, Dah-Yoh Lim, Adam Lee, Adam Lindsey, Rahil Mir, Gopal Mulagund, Andy Philips, Preetam Ramakrishna, Philip Thornton, Peter Wahl, Lixia Yuan, Paul Youn, and Patricia Huey. Managing the Keystore and the Master Encryption Key. *Oracle Help Center*. Retrieved September 4, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/21/asoag/managing-keystore-and-tde-master-encryption-key.html

[113] Roman Dykyi. 2025. Understanding Validation Levels. *Medium*. Retrieved September 4, 2026 from https://medium.com/@dykyi.roman/understanding-validation-levels-74d0adecda5e

[114] EDB Team. 2025. Oracle vs. PostgreSQL: Key Differences and Best Use Cases. *EDB*. Retrieved September 4, 2026 from https://enterprisedb.com/blog/oracle-vs-postgresql-comparison

[115] Erichain. 2024. A Deep Dive into AI Large Models: Parameters, Tokens, Context Windows, Context Length, and…. *Medium*. Retrieved September 4, 2026 from https://medium.com/@jiarunhuang/a-deep-dive-into-ai-large-models-parameters-tokens-context-windows-context-length-and-8187fb393acb

[116] Vahid Faraji. 2026. Where Should Repository Interfaces Live in Clean Architecture? *Medium*. Retrieved September 4, 2026 from https://vahid-faraji-dev.medium.com/where-should-repository-interfaces-live-in-clean-architecture-7c12f74a4d7b

[117] FastAPI. Benchmarks - FastAPI. *FastAPI*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/benchmarks/#benchmarks-and-speed

[118] FastAPI. Concurrency and async / await. *FastAPI*. Retrieved February 1, 2026 from https://fastapi.tiangolo.com/async/

[119] FastAPI. Python Types Intro. *FastAPI*. Retrieved February 18, 2026 from https://fastapi.tiangolo.com/python-types/

[120] FastAPI. Frontend - FastAPI. *FastAPI*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/tutorial/frontend/

[121] FastAPI. Templating - Jinja2Templates - FastAPI. *FastAPI*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/reference/templating/

[122] FastAPI. Request Body. *FastAPI*. Retrieved February 4, 2026 from https://fastapi.tiangolo.com/tutorial/body/

[123] FastAPI. Third-Party Integrations. *FastAPI Cloud Docs*. Retrieved September 5, 2026 from https://fastapicloud.com/docs/integrations/third-party-integrations/

[124] FastAPI. FastAPI - FastAPI. *FastAPI*. Retrieved September 6, 2026 from https://fastapi.tiangolo.com/

[125] FastAPI. Deployments Concepts - FastAPI. *FastAPI*. Retrieved September 6, 2026 from https://fastapi.tiangolo.com/deployment/concepts/

[126] FastAPI. Behind a Proxy. *FastAPI*. Retrieved September 5, 2026 from https://fastapi.tiangolo.com/advanced/behind-a-proxy/

[127] FastAPI. APIRouter class - FastAPI. *FastAPI*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/reference/apirouter/

[128] FastAPI. Response Model - Return Type. *FastAPI*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/tutorial/response-model/

[129] FastAPI. SQL (Relational) Databases - FastAPI. *FastAPI*. Retrieved September 5, 2026 from https://fastapi.tiangolo.com/tutorial/sql-databases/

[130] Patricio ferraggi. 2020. Do you need Design Patterns in Functional Programming? *DEV Community*. Retrieved March 6, 2026 from https://dev.to/patferraggi/do-you-need-design-patterns-in-functional-programming-370c

[131] John Fincher. 2026. Sending Emails With Python – Real Python. *Real Python*. Retrieved September 4, 2026 from https://realpython.com/python-send-email/

[132] flashdba. 2014. Oracle AWR Reports: Understanding I/O Statistics. *flashdba*. Retrieved September 5, 2026 from https://flashdba.com/2014/02/26/oracle-awr-reports-understanding-io-statistics/

[133] flashdba. 2026. SLOB – The Silly Little Oracle Benchmark. *flashdba*. Retrieved September 5, 2026 from https://flashdba.com/slob/

[134] Martin Fowler. 2006. GUI Architectures. *martinfowler.com*. Retrieved September 5, 2026 from https://martinfowler.com/eaaDev/uiArchs.html

[135] Rico Fritzsche. 2024. Bounded Contexts: Behavior Over Data Structures - Part II. *Rico Fritzsche*. Retrieved April 20, 2026 from https://ricofritzsche.me/ddd-modularization-concepts-aggregates-part-ii/

[136] Matthieu Gaillard-Midol. Souveraineté Numérique : L'ère du « Sovereignty First » et du Cloud de Résistance. *Devoteam*. Retrieved September 5, 2026 from https://www.devoteam.com/fr/expert-view/souverainete-numerique-lere-du-sovereignty-first-et-du-cloud-de-resistance/

[137] Jatin Garg. 2025. Types of Database Indexes: When to Use B-Tree, Hash, and Full-Text Indexes. *Gocodeo*. Retrieved September 5, 2026 from https://www.gocodeo.com/post/types-of-database-indexes-when-to-use-b-tree-hash-and-full-text-indexes

[138] Rahul Garg. 2022. Demystifying software architecture patterns. *Thoughtworks*. Retrieved April 17, 2026 from https://www.thoughtworks.com/insights/blog/architecture/demystify-software-architecture-patterns

[139] GeekBench. Geekbench 7 - Cross-Platform Benchmark. *GeekBench*. Retrieved September 4, 2026 from https://www.geekbench.com/

[140] GeeksforGeeks. 2025. Coupling and Cohesion - Software Engineering. *GeeksforGeeks*. Retrieved February 26, 2026 from https://www.geeksforgeeks.org/software-engineering/software-engineering-coupling-and-cohesion/

[141] gelovolro. 2024. What are merge join, hash join, and nested loop? Example in PostgreSQL. *Medium*. Retrieved September 5, 2026 from https://gelovolro.medium.com/what-are-merge-join-hash-join-and-nested-loop-example-in-postgresql-29123ca18fd1

[142] George. 2023. Separation of Concerns between Use Cases and Entities. *Technical blog from UNIL engineering teams*. Retrieved September 4, 2026 from https://medium.com/unil-ci-software-engineering/separation-of-concern-between-use-cases-and-entities-e3a86cdee49e

[143] Hilal Gevrek. 2025. PART 1 — Understanding Keepalived for High Availability. *Medium*. Retrieved September 12, 2026 from https://medium.com/seynur/part-1-understanding-keepalived-for-high-availability-e88c40579f80

[144] Iddo Gino. 2025. Post on LinkedIn. *LinkedIn*. Retrieved September 4, 2026 from https://www.linkedin.com/posts/iddogino_do-we-still-need-rag-in-the-age-of-large-activity-7326648596185915392-Q2M8/

[145] Marlon Giona. 2025. Sequential vs Random I/O. *LinkedIn*. Retrieved September 5, 2026 from https://www.linkedin.com/pulse/sequential-vs-random-io-marlon-giona-dkswf/

[146] Dinu Gitlan. 2025. What Is SSL Termination And How Does It Work? *Medium*. Retrieved September 5, 2026 from https://medium.com/@munteanu210/what-is-ssl-termination-and-how-does-it-work-a194a77c2974

[147] Google. General Purpose VM pricing. *Google Cloud*. Retrieved September 5, 2026 from https://cloud.google.com/products/compute/pricing/general-purpose

[148] Google. Network bandwidth | Compute Engine. *Google Cloud Documentation*. Retrieved September 6, 2026 from https://docs.cloud.google.com/compute/docs/network-bandwidth

[149] Google. Storage pricing. *Google Cloud*. Retrieved September 6, 2026 from https://cloud.google.com/storage/pricing

[150] Google Cloud Tech and Smitha Kolan - Machine Learning Engineer. 2026. MCP vs API: Why traditional APIs are failing AI agents. *YouTube*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=185XGEMefgc

[151] Grant Thornton. 2024. IFRS Example Consolidated Financial Statements 2024. Retrieved from https://www.grantthornton.global/globalassets/1.-member-firms/global/insights/insight-content-blocks-and-media/ifrs/example-financial-statements-2024/ifrs-2024-example-financial-statements.pdf

[152] Gravitee. 2025. Categorizing Web APIs | Overview. *Gravitee Documentation*. Retrieved February 19, 2026 from https://documentation.gravitee.io/platform-overview/before-you-begin/api-fundamentals/categorizing-web-apis

[153] Harry Percival and Bob Gregory. 2020. cosmic_python - Simple patterns for building complex applications. *Cosmic Python*. Retrieved March 16, 2026 from https://www.cosmicpython.com

[154] Kamil Grzybek. 2024. Modular-monolith-with-ddd: Full Modular Monolith application with Domain-Driven Design approach. *GitHub*. Retrieved September 4, 2026 from https://github.com/kgrzybek/modular-monolith-with-ddd

[155] MOHAMMED HANAN M T P. 2026. aio-libs/aiohttp: Asynchronous HTTP client/server framework for asyncio and Python. Retrieved February 4, 2026 from https://github.com/aio-libs/aiohttp

[156] D. Hardt. 2012. *The OAuth 2.0 Authorization Framework*. Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

[157] Dzmitry Harunou. 2025. Gateway Unit in Clean Architecture for Frontend Applications. *DEV Community*. Retrieved September 4, 2026 from https://dev.to/harunou/gateway-unit-in-clean-architecture-for-frontend-applications-137f

[158] Anca Hellmann. 2026. How to Choose Modular Monolith vs. Microservices Architecture. *DO OK*. Retrieved September 4, 2026 from https://dook.pro/blog/tips-tricks/how-to-choose-modular-monolith-vs-microservices-architecture/

[159] Hesbon limo. 2025. Understanding File I/O in Python: Reading, Writing, and Managing Files. *DEV Community*. Retrieved February 2, 2026 from https://dev.to/limoh653/understanding-file-io-in-python-reading-writing-and-managing-files-2dof

[160] hgraca. 2018. More than concentric layers. *@hgraca*. Retrieved April 20, 2026 from https://herbertograca.com/2018/07/07/more-than-concentric-layers/

[161] Holori. p5.48xlarge pricing and specs - AWS | Holori. *Holori Cloud Calculator*. Retrieved September 27, 2026 from https://calculator.holori.com/aws/ec2/p5.48xlarge

[162] Kevin Hood. Scaling Stateless vs Stateful Services. Retrieved February 20, 2026 from https://blog.dreamfactory.com/scaling-stateless-vs-stateful-services

[163] Adam Hopkins. 2020. Oracle Licensing - NUP or CPU. *DSP*. Retrieved September 9, 2026 from https://content.dsp.co.uk/oracle-licensing-nup-or-cpu

[164] HTMX. hx-swap Attribute. *HTMX*. Retrieved September 4, 2026 from https://htmx.org/attributes/hx-swap/

[165] HTMX. htmx ~ Documentation. *HTMX*. Retrieved September 4, 2026 from https://htmx.org/docs/#requests

[166] Hugging Face. 2025. Qwen/Qwen3-VL-235B-A22B-Instruct. *Hugging Face*. Retrieved September 8, 2026 from https://huggingface.co/Qwen/Qwen3-VL-235B-A22B-Instruct

[167] Hugging Face. Use AI Models Locally. *Hugging Face*. Retrieved September 5, 2026 from https://huggingface.co/docs/hub/en/local-apps

[168] Hugging Face. Run Inference on servers. *Hugging Face*. Retrieved September 5, 2026 from https://huggingface.co/docs/huggingface_hub/en/guides/inference

[169] Hugging Face. How caching works. *Hugging Face*. Retrieved September 23, 2026 from https://huggingface.co/docs/transformers/main/cache_explanation

[170] Andreas Hütter. 2022. Answer to “What actually is a subdomain in domain-driven design?” *Stack Overflow*. Retrieved April 17, 2026 from https://stackoverflow.com/a/73080428

[171] Katerina Hynkova. 2025. Ultimate guide to FastAPI library in Python. *DeepNote*. Retrieved September 4, 2026 from https://deepnote.com/blog/ultimate-guide-to-fastapi-library-in-python

[172] IBM. 2025. IBM Cloud for Financial Services. *IBM*. Retrieved September 10, 2026 from https://www.ibm.com/products/cloud/financial-services

[173] IBM. 2026. Differences between static and dynamic SQL. *IBM*. Retrieved September 5, 2026 from https://www.ibm.com/docs/en/db2-for-zos/12.0.0?topic=program-differences-between-static-dynamic-sql

[174] IBM. IBM Cloud FedRAMP compliance. *IBM*. Retrieved September 15, 2026 from https://www.ibm.com/products/cloud/compliance/fedramp

[175] IBM. NVIDIA GPUs on IBM Cloud. *IBM*. Retrieved September 10, 2026 from https://www.ibm.com/products/gpu-ai-accelerator/nvidia

[176] Mostafa Ibrahim. 2024. OAuth vs JWT: Key Differences Explained | SuperTokens. Retrieved September 4, 2026 from https://supertokens.com/blog/oauth-vs-jwt

[177] IEEE. Network Fabric. *IEEE Technology Navigator*. Retrieved September 6, 2026 from https://technav.ieee.org/topic/network-fabric/

[178] Ignacy. 2026. vLLM vs TensorRT-LLM vs SGLang (2026): Which to Pick. *DTF*. Retrieved September 5, 2026 from https://decodethefuture.org/en/vllm-vs-tensorrt-llm-vs-sglang-2026/

[179] iku. 2025. Is JSON Outdated? The Reasons Why the New LLM-Era Format “TOON” Saves Tokens. *DEV Community*. Retrieved September 6, 2026 from https://dev.to/akari_iku/is-json-outdated-the-reasons-why-the-new-llm-era-format-toon-saves-tokens-31e5

[180] ImranzamanML. 2024. Here is how we can calculate the size of any LLM model: Each parameter in LLM…. *Hugging Face*. Retrieved September 5, 2026 from https://huggingface.co/posts/ImranzamanML/127269471333935

[181] IndustryNews. 2025. SSD Read and Write Speed Explained in Details - OSCOO. Retrieved September 5, 2026 from https://www.oscoo.com/news/ssd-read-and-write-speed-explained-in-details/

[182] Intel. Intel® Xeon® Gold 6330 Processor (42M Cache, 2.00 GHz) - Product Specifications. *Intel*. Retrieved September 6, 2026 from https://www.intel.com/content/www/us/en/products/sku/212458/intel-xeon-gold-6330-processor-42m-cache-2-00-ghz/specifications.html

[183] Intel. Intel® Xeon® 6980E+ processor (528M Cache, 2.10 GHz) - Product Specifications. *Intel*. Retrieved September 6, 2026 from https://www.intel.com/content/www/us/en/products/sku/246075/intel-xeon-6980e-processor-528m-cache-2-10-ghz/specifications.html

[184] ISO. ISO/IEC 27001:2022. *ISO*. Retrieved September 5, 2026 from https://www.iso.org/standard/27001

[185] ISO. ISO/IEC 42001:2023. *ISO*. Retrieved September 5, 2026 from https://www.iso.org/standard/42001

[186] IT Price. Dell 7.68 NVME Price - Dell Price List 2022. *IT Price*. Retrieved September 6, 2026 from https://itprice.com/dell-price-list/7.68%20nvme.html

[187] Thorben Janssen. 2023. Design Patterns Explained – Dependency Injection. *Stackify*. Retrieved March 1, 2026 from https://stackify.com/dependency-injection/

[188] Hossein Nejati Javaremi. 2026. The Anti-Corruption Layer: Protecting Your Domain from Legacy Systems. *Medium*. Retrieved September 4, 2026 from https://hosseinnejati.medium.com/the-anti-corruption-layer-protecting-your-domain-from-legacy-systems-6da58fc5f462

[189] Jettify. 2026. aio-libs/aioodbc. Retrieved February 4, 2026 from https://github.com/aio-libs/aioodbc

[190] Jinja. API — Jinja Documentation (3.1.x). *Jinja*. Retrieved September 4, 2026 from https://jinja.palletsprojects.com/en/stable/api/

[191] JusDB Team. 2023. Database Capacity Planning: From Metrics to Growth Projections. *JusDB*. Retrieved September 5, 2026 from https://www.jusdb.com/blog/database-capacity-planning-metrics-growth

[192] Ramesh Kannan s. 2026. Nginx Is Overkill for FastAPI: I Benchmarked Caddy and Traefik. *Medium*. Retrieved September 5, 2026 from https://medium.com/@rameshkannanyt0078/nginx-is-overkill-for-fastapi-i-benchmarked-caddy-and-traefik-846a0e6539eb

[193] Ayush Kaushik. 2026. How Many Uvicorn Workers Do You Actually Need? *LogicLoopTech*. Retrieved September 6, 2026 from https://www.logiclooptech.dev/how-many-uvicorn-workers-do-you-actually-need-fastapi-performance-guide/

[194] kdakan. 2020. DDD-Domain-Driven-Design/README.md at master · kdakan/DDD-Domain-Driven-Design. *GitHub*. Retrieved April 20, 2026 from https://github.com/kdakan/DDD-Domain-Driven-Design/blob/master/README.md

[195] KEEKLO. Server Colocation from $99.99/month. *KEEKLO*. Retrieved September 6, 2026 from https://keeklo.com/colocation.html

[196] KFSys. 2023. Navigating High Availability with Keepalived | DigitalOcean. *DigitalOcean*. Retrieved September 5, 2026 from https://www.digitalocean.com/community/questions/navigating-high-availability-with-keepalived

[197] Michael Kleehammer. pyodbc: DB API module for ODBC. Retrieved February 5, 2026 from https://github.com/mkleehammer/pyodbc

[198] Kludex. uvicorn/uvicorn/supervisors/multiprocess.py at main · Kludex/uvicorn. *GitHub*. Retrieved September 4, 2026 from https://github.com/Kludex/uvicorn/blob/main/uvicorn/supervisors/multiprocess.py

[199] Knime. Develop a KNIME Extension with Python. *Knime*. Retrieved September 5, 2026 from https://docs.knime.com/developers/latest/create_a_node_with_python/

[200] Sid Krishnan. Review: Stop Using JWT for Sessions. *Ducktypelabs*. Retrieved February 21, 2026 from https://www.ducktypelabs.com/review-stop-using-jwt-for-sessions/

[201] Kubernetes. Configure Liveness, Readiness and Startup Probes. *Kubernetes*. Retrieved September 5, 2026 from https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

[202] Raushani Kumari. 2025. Server vs. Serverless Architecture. *GeeksforGeeks*. Retrieved September 5, 2026 from https://www.geeksforgeeks.org/system-design/server-vs-serverless-architecture/

[203] Akshita Kumawat. 2026. Introduction to API (Application Programming Interface). *GeeksforGeeks*. Retrieved September 4, 2026 from https://www.geeksforgeeks.org/software-testing/what-is-an-api/

[204] Ahmet Eren Lapanta. 2026. From Threads to Coroutines: Modern Concurrency and Parallelism Explained. *HesapTech*. Retrieved September 4, 2026 from https://medium.com/hesaptech/from-threads-to-coroutines-modern-concurrency-and-parallelism-explained-ac5484377722

[205] LearnThatStack. 2026. How API Rate Limiting Actually Works and How to Build Your Own. *YouTube*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=s3wIoIN_-u4

[206] Ken Lee. 2024. Why Use A Linux Server In Enterprise Environments? | SUSE Blog. *Suse*. Retrieved September 10, 2026 from https://www.suse.com/c/why-use-linux/

[207] Newvick Lee. 2025. Python Concurrency: Threads, Processes, and asyncio Explained. *Newvick's blog*. Retrieved February 2, 2026 from https://newvick.com/posts/python-concurrency/

[208] R Leishman. 2007. Understanding Indexes. *OracleFAQ*. Retrieved September 5, 2026 from https://www.orafaq.com/node/1403

[209] Patroclos Lemoniatis. 2022. Stored Procedures vs ORM Framework: When to use each one and Why. *Medium*. Retrieved September 5, 2026 from https://patroclosdev.medium.com/stored-procedures-vs-orm-framework-when-to-use-each-one-and-why-b575600ee124

[210] Litestar. Sync vs. Async. *Litestar*. Retrieved February 1, 2026 from https://docs.litestar.dev/main/topics/sync-vs-async.html

[211] llamaIndex. 2026. Top Document Extraction Software: From Legacy OCR to Agentic AI. *llamaIndex*. Retrieved September 4, 2026 from https://www.llamaindex.ai/insights/top-document-extractaction-software

[212] LM-Kit.NET. LM-Kit.NET VLM OCR - Vision Model Text Extraction from Images and PDFs in C# .NET. *LM-Kit.NET*. Retrieved September 4, 2026 from https://docs.lm-kit.com/lm-kit-net/guides/samples-overview/vlm-ocr-demo.html

[213] long2ice. 2026. fastapi-limiter/pyproject.toml at main · long2ice/fastapi-limiter. *GitHub*. Retrieved September 4, 2026 from https://github.com/long2ice/fastapi-limiter/blob/main/pyproject.toml

[214] Zhen Lu. 2024. Spot vs. On-Demand Instances: What's the Difference? *Runpod*. Retrieved September 14, 2026 from https://www.runpod.io/blog/spot-vs-on-demand

[215] Bhavika M. 2026. Monolith vs Microservices: What Actually Matters (Lessons From Netflix, Amazon, Atlassian). *Medium*. Retrieved September 4, 2026 from https://aws.plainenglish.io/monolith-vs-microservices-what-actually-matters-lessons-from-netflix-amazon-atlassian-7cdc19dff4e4

[216] Cilia Madani. 2025. Making FastAPI Fast: A Beginner's Guide to Workers and Threads. *Medium*. Retrieved September 4, 2026 from https://medium.com/@ciliaMadani/making-fastapi-fast-a-beginners-guide-to-workers-and-threads-095d7f2b9575

[217] Sizan Mahmud. 2025. The Hidden Cost of Bad Caching: Why More Cache Isn't Always Better. *Medium*. Retrieved September 4, 2026 from https://medium.com/@sizanmahmud08/the-hidden-cost-of-bad-caching-why-more-cache-isnt-always-better-168b74af6715

[218] Shiladitya Majumder. 2026. Using SlowAPI in FastAPI: Mastering Rate Limiting Like a Pro. *Medium*. Retrieved September 4, 2026 from https://shiladityamajumder.medium.com/using-slowapi-in-fastapi-mastering-rate-limiting-like-a-pro-19044cb6062b

[219] Abir Mandal. 2026. State Corporate Income Tax Rates and Brackets, 2026. *Tax Foundation*. Retrieved September 16, 2026 from https://taxfoundation.org/data/all/state/state-corporate-income-tax-rates-brackets/

[220] Athreya aka Maneshwar. 2025. Domain Driven Design. *DEV Community*. Retrieved March 2, 2026 from https://dev.to/lovestaco/domain-driven-design-3i2j

[221] Jim Manico, Jakub Maćkowski, and Gabriel Corona. Cross Site Scripting Prevention. *OWASP Cheat Sheet Series*. Retrieved September 4, 2026 from https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[222] C Martin. 2012. Clean Architecture. *The Clean Blog*. Retrieved from https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

[223] Lucas McGregor. 2026. Comparing Different LLM Models For Data Extraction. *Medium*. Retrieved September 4, 2026 from https://medium.com/the-leading-indicator/comparing-different-llm-models-for-data-extraction-f067d56bed54

[224] MDN. 2025. Server-side rendering (SSR) - Glossary. *MDN Web Docs*. Retrieved September 16, 2026 from https://developer.mozilla.org/en-US/docs/Glossary/SSR

[225] MDN. 2026. HTTP caching - HTTP. *MDN Web Docs*. Retrieved September 5, 2026 from https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching

[226] MDN. 2026. Responsive web design - Learn web development. *MDN Web Docs*. Retrieved September 5, 2026 from https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design

[227] MDN. <input type="password"> HTML attribute value - HTML. *MDN Web Docs*. Retrieved September 5, 2026 from https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/password

[228] MDN. Client-side form validation - Learn web development. *MDN Web Docs*. Retrieved September 5, 2026 from https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation

[229] P M Mell and T Grance. 2011. *The NIST definition of cloud computing* (0 ed.). National Institute of Standards and Technology, Gaithersburg, MD. https://doi.org/10.6028/NIST.SP.800-145

[230] Mitrasish. 2026. Self-Host Embeddings and Rerankers: TEI on GPU Cloud (2026). *Spheron*. Retrieved September 6, 2026 from https://www.spheron.network/blog/self-host-embedding-reranker-tei-gpu-cloud/

[231] David Muraya. 2025. A Practical Guide to FastAPI Security. Retrieved September 4, 2026 from https://www.davidmuraya.com/blog/fastapi-security-guide

[232] NCS London. 2025. Oracle 23ai Databases: Free or Enterprise Guide. *NCS London*. Retrieved September 9, 2026 from https://www.ncs-london.com/blog/oracle-23ai-free-vs-enterprise-which-version-is-right-for-your-business/

[233] ISAAC NDUBUISI. 2022. Should you use Raw SQL or an ORM? *CodeX*. Retrieved September 4, 2026 from https://medium.com/codex/should-you-use-raw-sql-or-an-orm-18468c11c1a9

[234] NebulaGraph. 2022. Difference Between Relational and Non Relational Database. *NebulaGraph*. Retrieved September 4, 2026 from https://nebula-graph.io/posts/difference-between-relational-and-non-relational-database

[235] NebulaGraph. 2024. Choosing a Database for Your Project: Step by Step. *NebulaGraph*. Retrieved September 4, 2026 from https://nebula-graph.io/posts/choosing-a-database-for-your-project-step-by-step

[236] Nexora. 2026. I built a rate limiter that survives process restarts — no Redis required. *DEV Community*. Retrieved September 4, 2026 from https://dev.to/nexoradev/i-built-a-rate-limiter-that-survives-process-restarts-no-redis-required-g4n

[237] Duy Nguyen. 2025. Understanding Parallelisms in vLLM: Case Study with Qwen3–30B-A3B-Thinking-2507 on A10G GPUs. *Medium*. Retrieved September 25, 2026 from https://medium.com/@justinduy/understanding-parallelisms-in-vllm-case-study-with-qwen3-30b-a3b-thinking-2507-on-a10g-gpus-59821cb20c6e

[238] Nitin. 2023. Serverful vs Serverless: Which One is Right for Your Needs? *Locofy Blogs*. Retrieved September 5, 2026 from https://www.locofy.ai/blog/serverful-vs-serverless

[239] Olivia Novak. 2025. Asynchronous vs. Synchronous Functions in FastAPI When to Pick Which. *Leapcell*. Retrieved February 1, 2026 from https://leapcell.io/blog/asynchronous-vs-synchronous-functions-in-fastapi-when-to-pick-which

[240] Nvidia. Performance Archive — Megatron Bridge. *Nvidia*. Retrieved September 25, 2026 from https://docs.nvidia.com/nemo/megatron-bridge/0.6.2/performance-summary-archive.html

[241] NVIDIA. NVIDIA Data Center Products: Next Wave of AI Innovation. *NVIDIA Data Center Products*. Retrieved September 6, 2026 from https://www.nvidia.com/en-gb/data-center/products/

[242] NVIDIA. NVIDIA-Certified Systems Configuration Guide. *NVIDIA Certification Programs Documentation*. Retrieved September 6, 2026 from https://docs.nvidia.com/certification-programs/latest/nvidia-certified-configuration-guide.html

[243] NVIDIA. NVIDIA Grace CPU Superchip Datasheet. *NVIDIA*. Retrieved September 6, 2026 from https://resources.nvidia.com/en-us-grace-cpu/data-center-datasheet

[244] Nvidia. NVIDIA DGX H100. *NVIDIA*. Retrieved September 26, 2026 from https://www.nvidia.com/en-gb/data-center/dgx-h100/

[245] Nvidia. Introduction to NVIDIA DGX H100/H200 Systems — NVIDIA DGX H100/H200 User Guide. *Nvidia*. Retrieved September 27, 2026 from https://docs.nvidia.com/dgx/dgxh100-user-guide/introduction-to-dgxh100.html

[246] OLlama. 2024. bge-m3. *OLlama*. Retrieved September 23, 2026 from https://ollama.com/bge-m3

[247] Romuald Oluwatobi. 2025. Python Concurrency Model Comparison For CPU And IO Bound Execution: Asyncio vs Threads vs Sync. *Medium*. Retrieved February 19, 2026 from https://medium.com/@romualdoluwatobi/python-concurrency-model-comparison-for-cpu-and-io-bound-execution-asyncio-vs-threads-vs-sync-35c114fc0045

[248] OpenMetal IaaS. Bare Metal Dedicated Servers. *OpenMetal IaaS*. Retrieved September 5, 2026 from https://openmetal.io/products/bare-metal/

[249] OpterSoft. Security. *OpterSoft*. Retrieved September 4, 2026 from https://academy.optersoft.com/python/fastapi/security

[250] Oracel and affiliates. Concurrent Programming with asyncio. *python-oracledb 2.3.0 documentation*. Retrieved February 5, 2026 from https://python-oracledb.readthedocs.io/en/v2.3.0/user_guide/asyncio.html

[251] Oracel and affiliates. 8. Using Bind Variables. *Python-oracledb 26.0.0b1 Documentation*. Retrieved September 7, 2026 from https://python-oracledb.readthedocs.io/en/latest/user_guide/bind.html

[252] Oracel and affiliates. 7. API: Cursor Objects. *Python-oracledb 26.0.0b1 Documentation*. Retrieved September 5, 2026 from https://python-oracledb.readthedocs.io/en/latest/api_manual/cursor.html

[253] Oracle. 2025. *Oracle Linux: The Ideal Choice for Running Oracle Database*. Oracle. Retrieved from https://www.oracle.com/cn/a/ocom/docs/linux/oracle-linux-the-ideal-choice-for-running-oracle-database.pdf

[254] Oracle. 2025. Oracle License Definitions and Rules Booklet. Retrieved from https://www.oracle.com/contracts/docs/lic_definitions_and_rules_v061525.pdf

[255] Oracle. 2026. Secret Management Service. *Oracle*. Retrieved September 4, 2026 from https://docs.oracle.com/en-us/iaas/Content/secret-management/home.htm

[256] Oracle. 2026. Oracle Processor Core Factor Table. Retrieved from https://www.oracle.com/contracts/docs/processor-core-factor-table-070634.pdf?download=false

[257] Oracle. 2026. Oracle Technology Global Price List. Retrieved from https://www.oracle.com/europe/a/ocom/docs/corporate/pricing/technology-price-list-070617.pdf

[258] Oracle. Oracle Instant Client - Free tools and libraries for connecting to Oracle Database. *Oracle*. Retrieved February 5, 2026 from https://www.oracle.com/database/technologies/instant-client.html

[259] Oracle. About Oracle Data Guard. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/en/cloud/paas/base-database/data-guard/index.html

[260] Oracle. Oracle AI Database 26ai Technical Architecture - Log Writer Process (LGWR). *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/dbiad/db_LGWR.html

[261] Oracle. Database Concepts - 3: Indexes and Index-Organized Tables. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/E11882_01/server.112/e40540/indexiot.htm#CNCPT721

[262] Oracle. Choosing the Best Applicable Index. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/other-databases/nosql-database/24.4/sqlreferencefornosql/choosing-best-applicable-index.html

[263] Oracle. DB_BLOCK_SIZE. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/B13789_01/server.101/b10755/initparams039.htm

[264] Oracle. Oracle Database Online Documentation 11g, Release 2 (11.2), Database Performance Tuning Guide - 8.2.6 Choosing Data Block Size. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/E11882_01/server.112/e41573/iodesign.htm#PFGRF94404

[265] Oracle. Transactions. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/21/cncpt/transactions.html

[266] Oracle. Data Blocks, Extents, and Segments. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/A57673_01/DOC/server/doc/SCN73/ch3.htm#data%20bl%20format

[267] Oracle. Oracle Database Online Documentation 12c Release 1 - Database Administrator’s Guide - 12: Managing Archived Redo Log Files. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/database/121/ADMIN/archredo.htm

[268] Oracle. Oracle® Database Administrator's Guide 11g Release 2 (11.2) - Planning the Redo Log. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/html/E25494_01/onlineredo002.htm

[269] Oracle. Oracle Database - Release 18 - Database Concepts - 15. Process Architecture. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/18/cncpt/process-architecture.html

[270] Oracle. Oracle7 Tuning, release 7.3.3 - Tuning I/O. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/A57673_01/DOC/server/doc/A48506/io.htm

[271] Oracle. SQL Language Reference. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/file_specification.html

[272] Oracle. Oracle Database - Release 18 - Database Performance Tuning Guide - 4 Configuring a Database for Performance. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/18/tgdba/configuring-database-for-performance.html

[273] Oracle. Base Database Features. *Oracle*. Retrieved September 9, 2026 from https://www.oracle.com/database/base-database-service/features/

[274] Oracle. Base Database Service Pricing. *Oracle*. Retrieved September 9, 2026 from https://www.oracle.com/database/base-database-service/pricing/

[275] Oracle and affiliates. 30. Appendix B: Differences between python-oracledb Thin and Thick Modes. *python-oracledb 26.0.0b1 documentation*. Retrieved September 5, 2026 from https://python-oracledb.readthedocs.io/en/latest/user_guide/appendix_b.html

[276] Oracle DBA. 2025. Understanding Temporary Tablespaces: Monitoring and Troubleshooting in Oracle. *Oracle DBA*. Retrieved September 5, 2026 from https://odba.in/understanding-temporary-tablespaces-monitoring-and-troubleshooting-in-Oracle/

[277] Alan Ouko. 2026. SQL ON DELETE CASCADE: Automatically Remove Dependent Data. *DataCamp*. Retrieved September 4, 2026 from https://www.datacamp.com/tutorial/sql-on-delete-cascade

[278] OVHcloud. Bare Metal Dedicated Servers. *OVHcloud*. Retrieved September 5, 2026 from https://us.ovhcloud.com/bare-metal/

[279] OVHcloud. Dedicated server prices - Explore our full server range. *OVHcloud*. Retrieved September 6, 2026 from https://www.ovhcloud.com/en/bare-metal/prices/

[280] Devrim Ozcay. 2026. We Built a REST API with 47 Endpoints. Should've Been 12. *Let's Code Future, on Medium*. Retrieved September 5, 2026 from https://medium.com/lets-code-future/we-built-a-rest-api-with-47-endpoints-shouldve-been-12-8185d99bc802

[281] Mehmet Ozkaya. 2024. The Bridge: Modular Monolith (Modulith) Architectures — Gateway Between Monolith and Microservices…. *Medium*. Retrieved September 4, 2026 from https://mehmetozkaya.medium.com/the-bridge-modular-monolith-modulith-architectures-gateway-between-monolith-and-microservices-069338211de7

[282] Maria Paktiti. 2026. Picking a password hash: A developer's guide to argon2, bcrypt, and scrypt. *WorkOS*. Retrieved September 4, 2026 from https://workos.com/blog/picking-a-password-hash-argon2-bcrypt-scrypt

[283] Arpita Pal. 2025. LLM, VLM, and VLA. *Medium*. Retrieved September 4, 2026 from https://medium.com/@arpipal2/llm-vlm-and-vla-d758b91479eb

[284] Pallets. Template Designer Documentation. *Jinja Documentation (3.1.x)*. Retrieved September 4, 2026 from https://jinja.palletsprojects.com/en/stable/templates/

[285] Bugra Parlayan. 2024. Understanding and Optimizing the "db file sequential read" Oracle Wait Event - Bugra Parlayan | Oracle Database & Exadata Blog. *Bugra Parlayan's Blog*. Retrieved September 5, 2026 from https://www.bugraparlayan.com.tr/understanding-and-optimizing-the-db-file-sequential-read-oracle-wait-event.html

[286] Sagar Parmar. 2026. Understanding LLM Precision and Quantization. *Medium*. Retrieved September 23, 2026 from https://sagar-parmar.medium.com/understanding-llm-precision-and-quantization-08197e19ff4a

[287] Josh Parsons. 2025. Password Hashing: What Cyber Pros Need to Know. *Enzoic*. Retrieved September 4, 2026 from https://www.enzoic.com/blog/password-hashing-what-cyber-pros-need-to-know/

[288] Alejandro Penalba Tarrida. 2025. PyPex: Analizador Estático de Complejidad para Código Python. Universidad de Barcelona. Retrieved from https://diposit.ub.edu/server/api/core/bitstreams/fbc44f0c-3d69-40ff-8b7c-5cc1dd7e97f1/content

[289] pete@fedramp.gov. Scope of FedRAMP¶. *FedRamp*. Retrieved September 5, 2026 from https://fedramp.gov/2026/scope/

[290] Plainionist. 2018. Implementing Clean Architecture - Of controllers and presenters. *Plainionist*. Retrieved September 4, 2026 from https://plainionist.github.io/Implementing-Clean-Architecture-Controller-Presenter/

[291] Plainionist. 2020. Answer to “Do I need presenter in Clean Architecture?” *Stack Overflow*. Retrieved September 5, 2026 from https://stackoverflow.com/a/59589084

[292] Port. What is Domain-Driven Design? Benefits, Challenges & Implementation. *Port*. Retrieved March 6, 2026 from https://www.port.io/glossary/domain-driven-design

[293] Portable Technology Solutions. What is an ODBC compliant database? *Portable Technology Solutions*. Retrieved February 4, 2026 from https://www.ptsmobile.com/what-is-odbc.htm

[294] Padmaja Potineni, Kathy Rich, Andy Adams, Beldalker Anand, Chipper Brown, Larry Carpenter, Pieter Van Puymbroeck, Laurence Clarke Jin-Jwei Chen, Jeff Detjen, Ray Dutcher, B. G. Garin David Gagne, Mahesh Girkar, Joydip Kundu Yuhong Gu, Steven Lee, Steven Lim, Nitin Karkhanis, Jonghyun Lee Goutam Kulkarni, Bob McGuirk, Joe Meeks, Steve Moriarty, Muthu Olagappan, Ashish Ray, Mike Schloss, Mike Smith, Lawrence To, Doug Voss Stephen Vivian, Hongjie Yang, and Subhash Chandra. Getting Started with Oracle Data Guard. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/sbydb/getting-started-with-oracle-data-guard.html

[295] PriceOfElectricity.com. Electricity Cost Calculator: Any kWh (500=$92, 1,000=$184). *PriceOfElectricity.com*. Retrieved September 6, 2026 from https://www.priceofelectricity.com/electricity-cost-calculator

[296] Progress Community. 2023. What to consider when deciding which database block size to use. *Progress Community*. Retrieved September 5, 2026 from https://community.progress.com/s/article/18293

[297] PwC. 2024. United States - Corporate - Taxes on corporate income. *PwC*. Retrieved September 16, 2026 from https://taxsummaries.pwc.com/united-states/corporate/taxes-on-corporate-income

[298] Pydantic. Validators - Pydantic Validation. *Pydantic Docs*. Retrieved February 11, 2026 from https://docs.pydantic.dev/latest/concepts/validators/

[299] Pydantic. Models - Pydantic Validation. *Pydantic Docs*. Retrieved February 11, 2026 from https://docs.pydantic.dev/latest/concepts/models/

[300] Pydantic. Fields - Pydantic Validation. *Pydantic Docs*. Retrieved February 11, 2026 from https://docs.pydantic.dev/latest/concepts/fields/

[301] Python Software Foundation. Developing with asyncio. *Python documentation*. Retrieved September 4, 2026 from https://docs.python.org/3/library/asyncio-dev.html

[302] Python Software Foundation. dataclasses — Data Classes. *Python documentation*. Retrieved March 16, 2026 from https://docs.python.org/3/library/dataclasses.html

[303] Python Software Foundation. Coroutines and Tasks. *Python documentation*. Retrieved February 23, 2026 from https://docs.python.org/3/library/asyncio-task.html

[304] pythontutorials.net. 2025. Combining asyncio with Multi-Worker ProcessPoolExecutor: How to Run Blocking Functions Concurrently and Optimize Execution Time. *pythontutorials.net*. Retrieved February 6, 2026 from https://www.pythontutorials.net/blog/combining-asyncio-with-a-multi-worker-processpoolexecutor/

[305] Qwen. 2025. config.json · Qwen/Qwen3-VL-235B-A22B-Instruct at refs/pr/17. *Hugging Face*. Retrieved September 23, 2026 from https://huggingface.co/Qwen/Qwen3-VL-235B-A22B-Instruct/blob/refs%2Fpr%2F17/config.json

[306] Qwen. 2025. Qwen/Qwen3-4B. *Hugging Face*. Retrieved September 5, 2026 from https://huggingface.co/Qwen/Qwen3-4B

[307] Qwen. config.json · Qwen/Qwen3-VL-235B-A22B-Instruct at refs/pr/14. *Hugging Face*. Retrieved September 23, 2026 from https://huggingface.co/Qwen/Qwen3-VL-235B-A22B-Instruct/blame/refs%2Fpr%2F14/config.json

[308] Qwen Team. 2026. Qwen3-VL Technical Report. *GitHub*. Retrieved September 25, 2026 from https://chenshi221.github.io/Clippings/Qwen3-VL-Technical-Report

[309] raghu135ram. 2025. Asyncio Vs Threading In Python. *GeeksforGeeks*. Retrieved February 2, 2026 from https://www.geeksforgeeks.org/python/asyncio-vs-threading-in-python/

[310] Ujjwal Raj. 2025. Volatility-Based Decomposition: A System Design Example. *DEV Community*. Retrieved February 26, 2026 from https://dev.to/ujjwall-r/volatility-based-decomposition-a-system-design-example-an3

[311] Ramkumar Swaminathan. 2017. Redo Logs in an Oracle Database - DBArch Video 11. *YouTube*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=-oPdDx9JYp4

[312] Ramkumar Swaminathan. 2018. What should be the Database block size? - DBPerf Video 15. *YouTube*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=xGMaThyqvEs

[313] Bhagya Rana. 2026. 10 Decentralized GPU Clouds Taking on AWS. *Medium*. Retrieved September 6, 2026 from https://medium.com/@bhagyarana80/10-decentralized-gpu-clouds-taking-on-aws-7bfac993c86f

[314] Lilith Ray. 2026. SSD Sequential vs Random Speed: Which One Matters for Your PC? *OSCOO*. Retrieved September 5, 2026 from https://www.oscooshop.com/blogs/blogs/ssd-sequential-vs-random-speed

[315] RedHat. 2024. What's the difference between cloud and virtualization? *RedHat*. Retrieved September 5, 2026 from https://www.redhat.com/en/topics/cloud-computing/cloud-vs-virtualization

[316] RedHat. Containers explained: What they are and why you should care. *RedHat*. Retrieved September 6, 2026 from https://www.redhat.com/en/topics/containers

[317] Redis. Rate Limiting. *Redis*. Retrieved September 4, 2026 from https://redis.io/glossary/rate-limiting/

[318] Redis. Understanding Domain-Driven Design (DDD) for Developers. *Redis*. Retrieved September 4, 2026 from https://redis.io/glossary/domain-driven-design-ddd/

[319] RelationalDBDesign. How Oracle Uses Dirty Buffers | LRU List to Manage Cache Space. *RelationalDBDesign*. Retrieved September 5, 2026 from https://www.relationaldbdesign.com/database-architecture/module4/writing-dirty-buffers.php

[320] RelationalDBDesign. Oracle 23c Database Writer (DBWn): Dirty Buffer Management, Checkpoints, Performance. *RelationalDBDesign*. Retrieved September 5, 2026 from https://www.relationaldbdesign.com/database-architecture/module4/dbw0-oracle-database-writer.php

[321] Adejumo Ridwan Suleiman. 2026. Python Dependency Injection: A Guide for Cleaner Code Design. *DataCamp*. Retrieved March 16, 2026 from https://www.datacamp.com/tutorial/python-dependency-injection

[322] RisingStack Engineering. 2024. Concurrency and Parallelism: Understanding I/O. *RisingStack Engineering*. Retrieved February 6, 2026 from https://blog.risingstack.com/concurrency-and-parallelism-understanding-i-o/

[323] Mary Beth Roeser, Nipun Agarwal, Rick Anderson, Mark Bauer, Neerja Bhatt, Tudor Bosman, George Candea, Wilson Chan, Sumanta Chatterjee, Eugene Chong, Connie Dialeris Green, Harvey Eneman, Bruce Ernst, Ira Greenberg, Steve Harris, Thuvan Hoang, Lilian Hobbs, Namit Jain, Hakkan Jakobsson, Bob Jenkins, Mark Johnson, Jonathan Klein, Sushil Kumar, Tirthankar Lahiri, Bill Lee, Yunrui Li, Juan Loaiza, Rich Long, Diana Lorentz, Catherine Luu, Neil MacNaughton, Ari Mozes, Gopal Mulagund, Subramanian Muralidhar, Ravi Murthy, Karuna Muthiah, Arvind Nithrakashyap, Ronald Obermarck, Kant Patel, Christopher Racicot, Mark Ramacher, Suvendu Ray Shankar Raman, Ann Rhee, Kathy Rich, Vivian Schupmann, Debbie Steiner, Malai Stalin, Seema Sundara, Juan Tellez, Alex Tsukerman, Kothanda Umamageswaran, Randy Urbano, Steve Vivian, Eric Voss, Tak Wang, Steve Wertheimer, Andy Witkowski, Daniel Wong, Graham Wood, Brian Wright, Aravind Yalamanchi, Qin Yu, Mohamed Ziauddin, and Bert Rich. Oracle Database - Release 19 - V$SQLSTATS. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/19/refrn/V-SQLSTATS.html

[324] RunLLM. 2026. Understanding Multi Node Parallelization - General. *vLLM Forums*. Retrieved September 24, 2026 from https://discuss.vllm.ai/t/understanding-multi-node-parallelization/2639

[325] Runpod. Create a new endpoint. *Runpod*. Retrieved September 10, 2026 from https://docs.runpod.io/api-reference/endpoints/POST/endpoints

[326] Runpod. AI Infrastructure Security & Compliance. *Runpod*. Retrieved September 10, 2026 from https://www.runpod.io/legal/compliance

[327] Runpod. The AI Developer Cloud. *Runpod*. Retrieved September 10, 2026 from https://www.runpod.io/

[328] Runpod. Welcome to Runpod. *Runpod*. Retrieved September 10, 2026 from https://docs.runpod.io/overview

[329] RunvLLM. 2025. Run multiple models - General. *vLLM Forums*. Retrieved September 5, 2026 from https://discuss.vllm.ai/t/run-multiple-models/1181

[330] Moez SADOK. 2026. moez-sadok/chat-clean-architecture. Retrieved September 5, 2026 from https://github.com/moez-sadok/chat-clean-architecture

[331] Laurent Savaete. 2026. laurentS/slowapi. *GitHub*. Retrieved September 4, 2026 from https://github.com/laurentS/slowapi

[332] Sam Scott and Graham Neray. 2021. Best practices for REST API security: Authentication and authorization. *Stack Overflow*. Retrieved February 21, 2026 from https://stackoverflow.blog/2021/10/06/best-practices-for-authentication-and-authorization-for-rest-apis/

[333] ScyllaDB. What is Database Scalability? Definition & FAQs | ScyllaDB. *ScyllaDB*. Retrieved September 4, 2026 from https://www.scylladb.com/glossary/database-scalability/

[334] Deeksha Sehgal. 2025. Oracle Sharding:Enterprise-Grade Distributed Database. Retrieved September 4, 2026 from https://blogs.oracle.com/database/oracle-shardingenterprisegrade-distributed-database

[335] Craig Shallahamer. 2016. What's The Difference Between Direct Path Reads And DB File Scattered Reads? *Orapub*. Retrieved September 5, 2026 from https://blog.orapub.com/20160316/difference-between-oracle-database-direct-path-reads-and-db-file-scattered-reads.html

[336] Sharyu. 2025. " When your apps goes from 100 to 10,000 users : Vertical vs Horizontal Scaling ". *Medium*. Retrieved September 4, 2026 from https://medium.com/@sharyup01/when-your-apps-goes-from-100-to-10-000-users-vertical-vs-horizontal-scaling-ced1e8fb2d46

[337] sheepcraft7555. 2025. Distributed Inference with Multi Machine & Multi GPU Setup Deploying Large Models via vLLM & Ray ! *YouTube*. Retrieved September 26, 2026 from https://www.youtube.com/watch?v=1lbBrTqUTqE&t=335s

[338] Silicon Power. 2025. Here's Why HDDs are Still the Preferred Option Over SSDs! *Silicon Power*. Retrieved September 5, 2026 from https://www.silicon-power.com/knowledge-detail/ssd-vs-hdd/

[339] Oleg Sinavski. 2024. Interfaces: abc vs. Protocols. *Oleg Sinavski*. Retrieved March 17, 2026 from https://sinavski.com/post/1_abc_vs_protocols/

[340] Bhagya Sithumini. 2025. How To Implement Clean Architecture in FastAPI: A Step-by-Step Guide. *Medium*. Retrieved September 4, 2026 from https://medium.com/@bhagyasithumini/how-to-implement-clean-architecture-in-fastapi-a-step-by-step-guide-8b73a75c650b

[341] Brett Slatkin. 2019. Python Classes and Interfaces. In *Effective Python: 90 Specific Ways to Write Better Python* (2nd Edition). Retrieved March 16, 2026 from https://www.informit.com/articles/article.aspx?p=2992597&seqNum=2

[342] SlowApi. SlowApi Documentation. *SLowApi Documentation*. Retrieved September 4, 2026 from https://slowapi.readthedocs.io/en/stable/

[343] Mike Smith. 2026. tfb Archives. *TechEmpower*. Retrieved September 6, 2026 from https://www.techempower.com/blog/tag/tfb/

[344] Snowflake. 2026. OLTP vs. OLAP: Differences and Applications. *Snowflake*. Retrieved September 4, 2026 from https://www.snowflake.com/en/fundamentals/olap-vs-oltp-the-differences/

[345] Sobolevn. 2019. Enforcing Single Responsibility Principle in Python. *Sobolevn's personal blog*. Retrieved March 16, 2026 from https://sobolevn.me/2019/03/enforcing-srp

[346] Solarwinds. Database Caching Strategies – Database Guide. *Solarwinds*. Retrieved September 4, 2026 from https://www.solarwinds.com/database-optimization/database-caching

[347] SQLModel. SQLModel. *SQLModel*. Retrieved September 5, 2026 from https://sqlmodel.tiangolo.com/#editor-support-everywhere

[348] NETservis s.r.o. AIC EB202-CP - XP1-E202CP03. *Anafra*. Retrieved September 6, 2026 from https://smicro.eu/aic-eb202-cp-xp1-e202cp03-1

[349] STARTUP HAKK. 2024. Why PostGreSQL is better than MS SQL. *YouTube*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=wRRTZZH54Iw

[350] Jeffrey Steiner. 2024. Oracle database performance optimization and benchmarking. *NetApp*. Retrieved September 5, 2026 from https://docs.netapp.com/us-en/ontap-apps-dbs/oracle/oracle-notes-optimization-and-benchmarking.html

[351] Jared Still. 2021. How to Make Oracle Use the Correct Index. *Pythian*. Retrieved September 5, 2026 from https://www.pythian.com/blog/how-to-make-oracle-use-the-correct-index

[352] Stripe. Card verification checks. *Stripe docs*. Retrieved September 5, 2026 from https://docs.stripe.com/disputes/prevention/verification

[353] Stripe. Pricing & Fees. *Stripe*. Retrieved September 9, 2026 from https://stripe.com/en-fr/pricing

[354] Stripe. API keys. *Stripe docs*. Retrieved September 4, 2026 from https://docs.stripe.com/keys

[355] Stytch. JWTs vs. sessions: which authentication approach is right for you? *Stytch*. Retrieved February 21, 2026 from https://stytch.com/blog/jwts-vs-sessions-which-is-right-for-you/

[356] Daniel Suherman, Ashish Agrawal, Waleed Ahmed, Helen Altmar, Lance Ashdown, Pete Belknap, Supiti Buranawatanachoke, Romain Colle, Karl Dias, Kurt Engeleiter, Leonidas Galanis, Veeranjaneyulu Goli, Prabhaker Gongloor, Prakash Gupta, Shantanu Joshi, Prathiba Kalirengan, Karen McKeen, Mughees Minhas, Konstantinos Morfonios, Valarie Moore, Ravi Pattabhi, Bert Rich, Yujun Wang, Keith Wong, Qinyi Wu, Khaled Yagoub, Hailing Yu, Yury Berezin, and Sunil Surabhi. Using Workload Analysis. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/ratug/using-workload-analysis.html

[357] Ardit Sulce. 2025. Functions vs. Classes in Python: How to Know Which One to Use. *Daily Python Projects*. Retrieved March 16, 2026 from https://dailypythonprojects.substack.com/p/when-to-use-a-function-vs-a-class

[358] Michael Swengel. 2025. Linux IS Harder to Use. So Why Do People Like It? *Medium*. Retrieved September 10, 2026 from https://medium.com/@michaelswengel/linux-is-harder-to-use-so-why-do-people-like-it-14c86a6d4909

[359] Matt Tanner. 2025. The Ultimate Guide to the Best SQL Database Options in 2025. *SingleStore*. Retrieved September 4, 2026 from https://www.singlestore.com/blog/ultimate-guide-best-sql-database-options-2025/

[360] Sharoz Tanveer. 2024. Functions vs Classes: When to Use Which and Why? *DEV Community*. Retrieved March 16, 2026 from https://dev.to/sharoztanveer/functions-vs-classes-when-to-use-which-and-why-48d6

[361] EPM Information Development Team. Batch Processing. *Oracle Help Center*. Retrieved September 4, 2026 from https://docs.oracle.com/en/cloud/saas/enterprise-performance-management-common/erpia/batch_processing.html

[362] Morph Team. 2026. What Is AI Inference? How LLM Inference Turns a Prompt Into Tokens. *Morph*. Retrieved September 6, 2026 from https://www.morphllm.com/ai-inference

[363] Oracle Team and Franck Pachot. 2015. Does the block size matter? *dbi services*. Retrieved September 5, 2026 from https://www.dbi-services.com/blog/do-the-block-size-matter/

[364] The Postman Team. 2026. JSON Schema Data Types: A Complete Guide to Validation. *Postman Blog*. Retrieved February 11, 2026 from https://blog.postman.com/json-schema-data-types/

[365] Tech Field Day. 2021. Oracle Database Performance with NetApp. *YouTube*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=_YuSmeJmsco

[366] TechEmpower. 2022. TechEmpower Web Framework Performance Comparison. *TechEmpower*. Retrieved September 4, 2026 from https://www.techempower.com/benchmarks/#section=data-r21&test=db

[367] TechEmpower. 2023. TechEmpower Web Framework Performance Comparison - round 22 - single query. *www.techempower.com*. Retrieved September 6, 2026 from https://www.techempower.com/benchmarks/#section=data-r22&test=db

[368] TechEmpower. 2025. TechEmpower Web Framework Performance Comparison. *TechEmpower*. Retrieved September 4, 2026 from https://www.techempower.com/benchmarks/#section=data-r23&test=db&s=1&l=zijzen-pa7&w=zik0zj-zik0zj-zik0zj-hra0hr&a=2&o=b

[369] Intelegain Technologies. 2019. What are web frameworks and why you need them? *Medium*. Retrieved September 4, 2026 from https://intelegain-technologies.medium.com/what-are-web-frameworks-and-why-you-need-them-c4e8806bd0fb

[370] Vegavid Technology. Multimodal AI: Models, Types, Benefits & Use Cases. *Vegavid*. Retrieved September 23, 2026 from https://vegavid.com/blog/multimodal-ai

[371] The Engineers' Pitstop. 2026. Why Composition Over Inheritance Isn't Always Better. *YouTube*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=JsWQ0jFM-jc

[372] The Magic of SQL. 2015. What Impact Do Indexes Have on Inserts?: Finding All the Red Sweets Part 0. *YouTube*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=f3U9F_wbo1I

[373] T. Thomas, W. Weijermars, and E. Van Berkum. 2010. Predictions of Urban Volumes in Single Time Series. *IEEE Trans. Intell. Transp. Syst.* 11, 1 (March 2010), 71–80. https://doi.org/10.1109/tits.2009.2028149

[374] Tim. 2015. Multiple Block Sizes. *oracle-base*. Retrieved September 5, 2026 from https://oracle-base.com/articles/9i/multiple-block-sizes

[375] Cheuk Ting Ho. 2025. Faster Python: Concurrency in async/await and threading | The PyCharm Blog. *The JetBrains Blog*. Retrieved February 2, 2026 from https://blog.jetbrains.com/pycharm/2025/06/concurrency-in-async-await-and-threading/

[376] Cesar de la Torre, Bill Wagner, and Mark Rousos. 2023. Designing a microservice domain model - .NET. In *.NET Microservices: Architecture for Containerized .NET Applications* (EDITION v7.0). Microsoft Corporation, Redmond, Washington 98052-6399. Retrieved April 20, 2026 from https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/microservice-domain-model

[377] TQ. 2026. Vast AI Alternatives: Dedicated GPU for Enterprise AI. *Private AI Resource Center: Definitions, FAQs & Industry News*. Retrieved September 6, 2026 from https://www.onesourcecloud.net/cms/2026-vast-ai-alternatives.html

[378] Nicholas Tsim. 2020. Best practices for REST APIs: How to write endpoint URLs. *Nicholas Tsim*. Retrieved September 5, 2026 from https://ntsim.uk/posts/best-practices-for-rest-apis-endpoint-urls

[379] Unstract. Agentic AI Document Processing & Extraction. *Unstract*. Retrieved September 5, 2026 from https://unstract.com/

[380] Anu Upadhyay. 2026. SOLID Principles with Real Life Examples - GeeksforGeeks. *GeeksforGeeks*. Retrieved February 26, 2026 from https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/

[381] UserBenchmark.org. UserBenchmark - Real-time GPU benchmarking. *UserBenchmark*. Retrieved September 4, 2026 from https://userbenchmark.org/fr/

[382] Uvicorn. Server Behavior. *Uvicorn*. Retrieved September 6, 2026 from https://uvicorn.dev/server-behavior/#timeouts

[383] Gyanaa Vaibhav. 2025. Scaling Systems — How to Handle More Users Without Breaking Everything. *Medium*. Retrieved September 4, 2026 from https://medium.com/@gynanrudr0/scaling-systems-how-to-handle-more-users-without-breaking-everything-39b873a67dc3

[384] Yury Velikanov. 2013. What Are the Best Oracle Database Load Testing Tools? *Pythian*. Retrieved September 5, 2026 from https://www.pythian.com/blog/oracle-database-load-testing-tools-overview

[385] Velorum. 2025. 7 FastAPI Images Under 100MB (Without Pain). *Medium*. Retrieved September 6, 2026 from https://medium.com/@1nick1patel1/7-fastapi-images-under-100mb-without-pain-0e455912a603

[386] John Vester. 2024. FastAPI Got Me an OpenAPI Spec Really... Fast. *DEV Community*. Retrieved February 23, 2026 from https://dev.to/johnjvester/fastapi-got-me-an-openapi-spec-really-fast-4ck0

[387] Vijay. 2025. How to Build a Lightning-Fast API with FastAPI and AsyncIO. *Medium*. Retrieved February 1, 2026 from https://python.plainenglish.io/how-to-build-a-lightning-fast-api-with-fastapi-and-asyncio-9f4e290d7ca5

[388] K Vinay Kumar. 2025. Optimising Linux Performance on IBM Cloud: Best Practices for RHEL & Ubuntu Servers. *Medium*. Retrieved September 15, 2026 from https://medium.com/@vinaykumar1998gangadkar/optimising-linux-performance-on-ibm-cloud-best-practices-for-rhel-ubuntu-servers-91c066ce5554

[389] Alice Vitard. 2026. “Nous ne pouvons pas mettre ces données dans des clouds hors de France ”: Faute d’offres jugées suffisamment souveraines, le ministère de la Justice développe son propre assistant d’IA générative. *L’Usine Digitale*. Retrieved September 5, 2026 from https://www.usine-digitale.fr/souverainete/nous-ne-pouvons-pas-mettre-ces-donnees-dans-des-clouds-hors-de-france-faute-doffres-jugees-suffisamment-souveraines-le-ministere-de-la-justice-developpe-son-propre-assistant-dia-generative.LCEKHYMEIRF3DDBYBWR3GM2LVU.html

[390] vLLM. Offline Inference - vLLM. *vLLM*. Retrieved September 5, 2026 from https://docs.vllm.ai/en/latest/serving/offline_inference/

[391] vLLM. Integration with HuggingFace. *vLLM*. Retrieved September 9, 2026 from https://docs.vllm.cc/en/latest/design/huggingface_integration.html

[392] vLLM. vllm serve. *vLLM*. Retrieved September 24, 2026 from https://docs.vllm.ai/en/latest/cli/serve/

[393] vLLM. Parallelism and Scaling. *vLLM*. Retrieved September 24, 2026 from https://docs.vllm.ai/en/latest/serving/parallelism_scaling/

[394] W3 Schools. Web APIs - introduction. *W3 Schools*. Retrieved September 4, 2026 from https://www.w3schools.com/js/js_api_intro.asp

[395] Andrew Way and Gagan Gill. 2026. Inside multi-node training: How to scale model training across GPU clusters. *together.ai*. Retrieved September 26, 2026 from https://www.together.ai/blog/multi-node-gpu-training

[396] Wealth of Wisdom. 2019. 048 Oracle DBA Complete Tutorial - Temporary Tablespace Management. *YouTube*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=9WR3-22KNQ0

[397] Web Dev Simplified and Kyle Cook from Web Dev Simplified. 2026. The Best Local Agentic Coding Workflow (Complete Guide). *YouTube*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=UngVdAsQEiU

[398] Jannik Wempe. 2021. Domain-Driven Design: Distinguish Entities, Value Objects. *Jannik Wempe*. Retrieved April 21, 2026 from https://wempe.dev/blog/domain-driven-design-entities-value-objects

[399] Indika Wimalasuriya. 2023. Amazon Prime Video's 90% Cost Reduction throuh moving to Monolithic. *DEV Community*. Retrieved September 4, 2026 from https://dev.to/indika_wimalasuriya/amazon-prime-videos-90-cost-reduction-throuh-moving-to-monolithic-k4a

[400] WIPO. 2020. Annual financial report and financial statements 2020. Retrieved from HTTPs://www.wipo.int/edocs/pubdocs/en/wipo_pub_rn2021_18e.pdf

[401] Scott Wlaschin. Functional Programming Design Patterns. *F# for fun and profit*. Retrieved March 6, 2026 from https://fsharpforfunandprofit.com/fppatterns/

[402] Michał Wojciechowski. 2025. Clean Architecture and DDD in Practice 2025 - Complete Guide. *wojciechowski.app*. Retrieved April 20, 2026 from https://wojciechowski.app/en/articles/clean-architecture-domain-driven-design-2025

[403] Paul Wolf. 2021. Class Design — Python Guidelines 0.1.0 documentation. *Python Guidelines*. Retrieved March 17, 2026 from https://python-coding-guidelines.readthedocs.io/en/latest/class_design.html

[404] Wondelai. clean-architecture. *Skills*. Retrieved September 5, 2026 from https://www.skills.sh/wondelai/skills/clean-architecture

[405] Sherlock Xu. 2025. Multimodal AI: The Best Open-Source Vision Language Models in 2026. *BentoML*. Retrieved September 23, 2026 from https://www.bentoml.com/blog/multimodal-ai-a-guide-to-open-source-vision-language-models

[406] Ren Xuancheng. 2025. Qwen3/docs/source/getting_started/speed_benchmark.md at main · QwenLM/Qwen3. *GitHub*. Retrieved September 25, 2026 from https://github.com/QwenLM/Qwen3/blob/main/docs/source/getting_started/speed_benchmark.md

[407] Rayan Yassminh. 2024. Choosing the Right Cloud Service Model: A Guide from Simple to Advanced. *Medium*. Retrieved September 5, 2026 from https://medium.com/@ryassminh/choosing-the-right-cloud-service-model-a-guide-from-simple-to-advanced-68ac9d876d01

[408] YMI RackmountPro. Gigabyte 1U 4Bays Single Processor Rackmount Server - R163-S30. *YMI RackmountPro*. Retrieved September 6, 2026 from https://www.rackmountpro.com/product/4604/R163-S30.html

[409] Sławomir Zakrzewski. 2025. SQL Injection in the Age of ORM: Risks, Mitigations, and Best Practices. *Afine*. Retrieved September 4, 2026 from https://afine.com/sql-injection-in-the-age-of-orm-risks-mitigations-and-best-practices

[410] Jiayong Zhu, Jiangtong Li, Jinru Ding, Dawei Cheng, Jie Xu, and Feng Yu. 2026. FinDocMRE: A Benchmark for Document-Level Financial Multimodal Reasoning Evaluation. https://doi.org/10.48550/arXiv.2605.17962

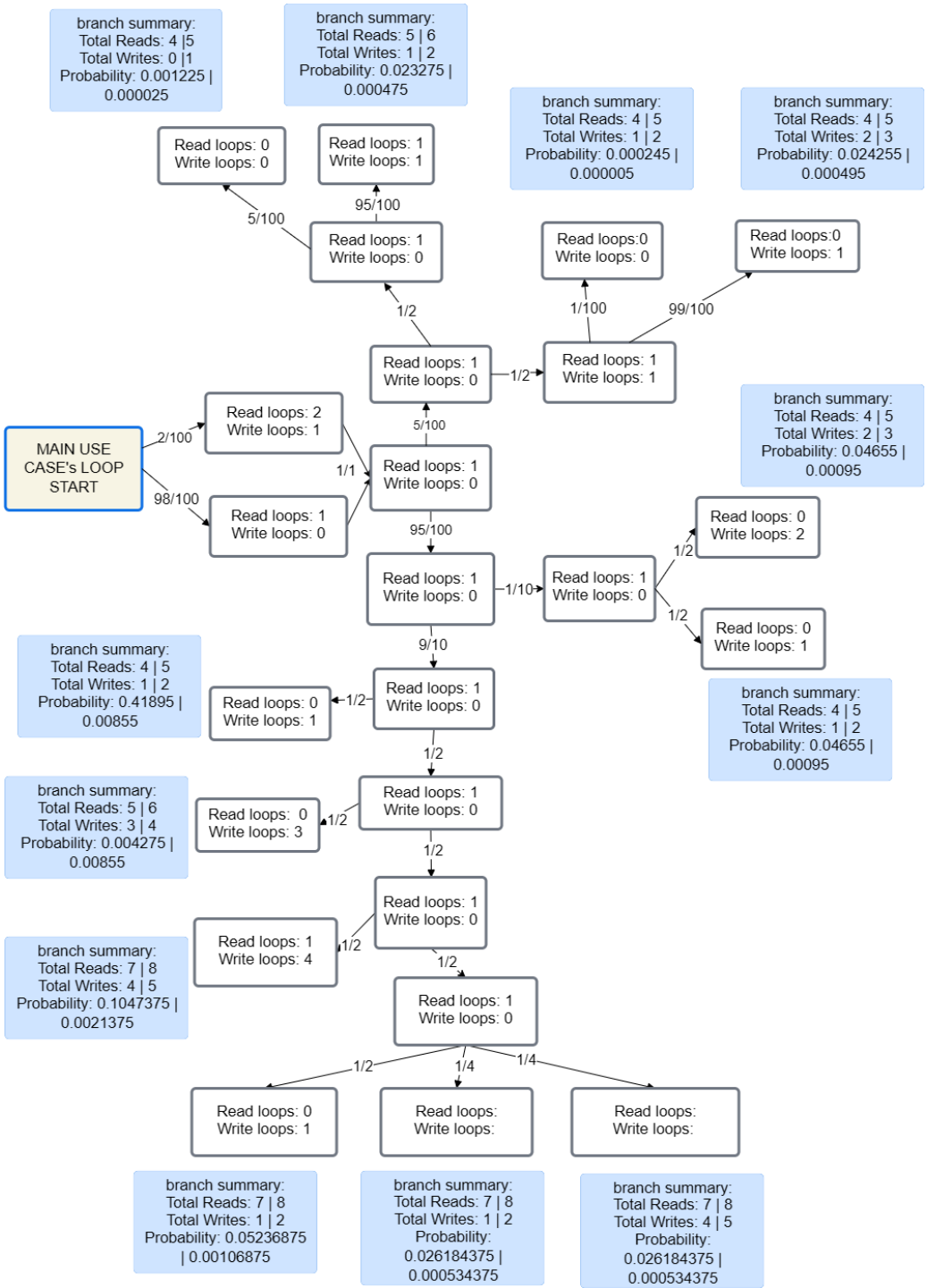
[411] zsolt.ero. 2023. What is a request in load balancing pricing? - Application Performance / Load Balancing. *Cloudflare Community*. Retrieved September 5, 2026 from https://community.cloudflare.com/t/what-is-a-request-in-load-balancing-pricing/588265

[412] 2022. What is SSO? | How single sign-on works. *Cloudflare*. Retrieved September 16, 2026 from https://www.cloudflare.com/learning/access-management/what-is-sso/

[413] Documentation, 12c Release 5 - Enterprise Manager ASM Config Assistant Context-Sensitive Online Help. *Oracle Help Center*. Retrieved September 8, 2026 from https://docs.oracle.com/cd/E24628_01/doc.121/e59369/usmca034.htm#USMCA186

Appendix 1: probabilistic tree of write and read loops happening during 1 loop of the main use case

The blue blocks sum up the read and write loops when the use case loop takes that orchestration path. After the first division in the execution of the use case's loop, I visually merge routes again because both logical tree sections from that point onwards are similar. Thus, each terminal branch displayed in the diagram represents two terminal branches, and the "|" separates data of these two branches.



Appendix 2: samples of the AWR Report of a DB of my Oracle contact:

For reminder, as said in the text the report shows two sessions (see left column of the first table under the “I#” cell, you can see session 1 and session 2) and both count in the analysis.

System Statistics

I#	Logical Reads	Physical Reads	Physical Writes	Redo Size (k)	Block Changes	User Calls	Execs	Parses	Logons	Txns
1	2,925,369	4,113	8,346	48,364	324,318	2,854	121,079	67,371	396	94
2	1,235,811	60	2,172	7,441	28,740	3,433	7,014	5,595	433	26
Sum	4,161,180	4,173	10,518	55,805	353,058	6,287	128,093	72,966	829	120
Avg	2,080,590	2,087	5,259	27,903	176,529	3,144	64,047	36,483	415	60
Std	1,194,698	2,866	4,366	28,937	209,005	409	80,656	43,682	26	48

Back to Global Activity Load Profile
Back to Top

System Statistics - Per Second

I#	Logical Reads/s	Physical Reads/s	Physical Writes/s	Redo Size (k)/s	Block Changes/s	User Calls/s	Execs/s	Parses/s	Logons/s	Txns/s
1	812.07	1.14	2.32	13.43	90.03	0.79	33.61	18.70	0.11	0.03
2	343.06	0.02	0.60	2.07	7.98	0.95	1.95	1.55	0.12	0.01
Sum	1,155.13	1.16	2.92	15.49	98.01	1.75	35.56	20.26	0.23	0.03
Avg	577.57	0.58	1.46	7.75	49.00	0.87	17.78	10.13	0.12	0.02
Std	331.64	0.80	1.21	8.03	58.02	0.11	22.39	12.13	0.01	0.01

Back to Global Activity Load Profile
Back to Top

System Statistics - Per Transaction

I#	Logical Reads/tx	Physical Reads/tx	Physical Writes/tx	Redo Size (k)/tx	Block Changes/tx	User Calls/tx	Execs/tx	Parses/tx	Logons/tx
1	31,120.95	43.76	88.79	514.51	3,450.19	30.36	1,288.07	716.71	4.21
2	47,531.19	2.31	83.54	286.21	1,105.38	132.04	269.77	215.19	16.65
Avg	39,326.07	23.03	86.16	400.36	2,277.79	81.20	778.92	465.95	10.43

				per Second			
Statistic	Total	per Second	per Trans	Average	Std Dev	Min	Max

physical read IO requests	2,780	0.77	23.17	0.39	0.52	0.02	0.76
physical read bytes	34,185,216	9,489.73	284,876.80	4,744.86	6,517.29	136.44	9,353.28
physical read total IO requests	34,409	9.55	286.74	4.78	0.59	4.36	5.19
physical read total bytes	651,311,616	180,802.39	5,427,596.80	90,401.20	27,118.06	71,225.83	109,576.56
physical read total multi block requests	156	0.04	1.30	0.04		0.04	0.04
physical reads	4,173	1.16	34.78	0.58	0.80	0.02	1.14
physical reads cache	4,173	1.16	34.78	0.58	0.80	0.02	1.14
physical reads cache prefetch	1,800	0.50	15.00	0.50		0.50	0.50
physical write IO requests	4,895	1.36	40.79	0.68	0.44	0.37	0.99
physical write bytes	86,163,456	23,918.75	718,028.80	11,959.37	9,927.89	4,939.30	18,979.45
physical write total IO requests	29,055	8.07	242.13	4.03	0.69	3.54	4.52
physical write total bytes	300,548,096	83,431.36	2,504,567.47	41,715.68	28,596.54	21,494.87	61,936.49
physical write total multi block requests	319	0.09	2.66	0.04	0.05	0.01	0.08
physical writes	10,518	2.92	87.65	1.46	1.21	0.60	2.32
physical writes direct	49	0.01	0.41	0.01	0.00	0.01	0.01
physical writes direct (lob)	4	0.00	0.03	0.00	0.00	0.00	0.00
physical writes direct temporary tablespace	45	0.01	0.38	0.01	0.00	0.01	0.01
physical writes from cache	10,469	2.91	87.24	1.45	1.21	0.60	2.31
physical writes non checkpoint	6,247	1.73	52.06	0.87	0.84	0.27	1.46
pinned cursors current	5	0.00	0.04	0.00	0.00	0.00	0.00
prefetched blocks aged out before use	2	0.00	0.02	0.00		0.00	0.00
process last non-idle time	7,199	2.00	59.99	1.00	0.00	1.00	1.00
queries parallelized	113	0.03	0.94	0.02	0.02	0.00	0.03
recursive calls	414,713	115.12	3,455.94	57.56	55.85	18.07	97.05
recursive cpu usage	11,333	3.15	94.44	1.57	0.68	1.09	2.05
redo blocks checksummed by FG (exclusive)	14,698	4.08	122.48	2.04	0.07	1.99	2.09
redo blocks written	121,979	33.86	1,016.49	16.93	16.68	5.13	28.73
redo blocks written (group 0)	120,808	33.54	1,006.73	16.77	16.61	5.02	28.52
redo blocks written (group 1)	1,171	0.33	9.76	0.16	0.07	0.11	0.21
redo entries	188,165	52.23	1,568.04	26.12	30.11	4.83	47.41
redo ordering marks	329	0.09	2.74	0.05	0.00	0.04	0.05
redo size	57,144,732	15,863.23	476,206.10	7,931.61	8,225.49	2,115.31	13,747.91
redo subscn max counts	306	0.08	2.55	0.04	0.02	0.03	0.06
redo synch time (usec)	32,201	8.94	268.34	4.47	5.85	0.33	8.61

				per Second			
Statistic	Total	per Second	per Trans	Average	Std Dev	Min	Max

redo blocks written	121,979	33.86	1,016.49	16.93	16.68	5.13	28.73
redo blocks written (group 0)	120,808	33.54	1,006.73	16.77	16.61	5.02	28.52
redo blocks written (group 1)	1,171	0.33	9.76	0.16	0.07	0.11	0.21
redo entries	188,165	52.23	1,568.04	26.12	30.11	4.83	47.41
redo ordering marks	329	0.09	2.74	0.05	0.00	0.04	0.05
redo size	57,144,732	15,863.23	476,206.10	7,931.61	8,225.49	2,115.31	13,747.91
redo subscn max counts	306	0.08	2.55	0.04	0.02	0.03	0.06
redo synch time (usec)	32,201	8.94	268.34	4.47	5.85	0.33	8.61
redo synch time overhead (usec)	2,570	0.71	21.42	0.36	0.45	0.04	0.68
redo synch time overhead count (2ms)	43	0.01	0.36	0.01	0.01	0.00	0.01
redo synch writes	318	0.09	2.65	0.04	0.00	0.04	0.04
redo wastage	3,260,508	905.11	27,170.90	452.55	8.19	446.77	458.34
redo write active strands	9,280	2.58	77.33	1.29	0.06	1.24	1.33
redo write broadcast ack count	173	0.05	1.44	0.02	0.01	0.02	0.03
redo write broadcast ack time	23,507	6.53	195.89	3.26	0.47	2.93	3.60
redo write finish time	8,671,097	2,407.07	72,259.14	1,203.54	57.35	1,162.98	1,244.09
redo write gather time	1,074,126	298.17	8,951.05	149.09	9.66	142.25	155.92
redo write info find	43	0.01	0.36	0.01	0.01	0.00	0.01
redo write issue time	1,274,434	353.78	10,620.28	176.89	9.19	170.39	183.39
redo write schedule time	1,266,898	351.69	10,557.48	175.84	9.19	169.34	182.34
redo write size count (4KB)	8,790	2.44	73.25	1.22	0.05	1.18	1.26
redo write size count (8KB)	125	0.03	1.04	0.02	0.00	0.02	0.02
redo write size count (16KB)	48	0.01	0.40	0.01	0.00	0.01	0.01
redo write size count (32KB)	25	0.01	0.21	0.00	0.00	0.00	0.00
redo write size count (128KB)	42	0.01	0.35	0.01	0.00	0.01	0.01
redo write size count (256KB)	13	0.00	0.11	0.00	0.00	0.00	0.00
redo write size count (512KB)	8	0.00	0.07	0.00	0.00	0.00	0.00
redo write size count (1024KB)	9	0.00	0.08	0.00	0.00	0.00	0.00
redo write size count (inf)	13	0.00	0.11	0.00		0.00	0.00
redo write time	880	0.24	7.33	0.12	0.01	0.12	0.13
redo write time (usec)	8,805,422	2,444.36	73,378.52	1,222.18	56.17	1,182.47	1,261.90
redo write total time	8,998,357	2,497.92	74,986.31	1,248.96	55.60	1,209.65	1,288.27
redo write worker delay (usec)	791,153	219.62	6,592.94	109.81	3.22	107.53	112.09
redo write worker delay count	9,121	2.53	76.01	1.27	0.06	1.23	1.31
redo writes	9,073	2.52	75.61	1.26	0.06	1.22	1.30
user commits	120	0.03	1.00	0.02	0.01	0.01	0.03
DBWR fusion writes	279	0.08	2.33	0.04	0.00	0.04	0.04
DBWR object drop buffers written	8	0.00	0.07	0.00		0.00	0.00
DBWR transaction table writes	388	0.11	3.23	0.05	0.01	0.05	0.06
DBWR undo block writes	4,089	1.14	34.08	0.57	0.47	0.23	0.90

Top SQL with Top Events

- Top SQL statements by DB Time along with the top events by DB Time for those SQLs.
- % Activity is the percentage of DB Time due to the SQL.
- % Event is the percentage of DB Time due to the event that the SQL is waiting on.
- % Row Source is the percentage of DB Time due to the row source for the SQL waiting on the event.
- Executions is the number of executions of the SQL that were sampled in ASH.

SQL ID	Plan Hash	Executions	% Activity	Event	% Event	Top Row Source	% Row Source	SQL Text
bgxtkrz2p3k08	4250213283	16	23.19	CPU + Wait for CPU	23.19	FIXED TABLE - FULL	20.29	SELECT VALUE FROM SYS.V_$PARAM...
gw788nxk7vbwg	4250213283	4	5.80	CPU + Wait for CPU	5.80	FIXED TABLE - FULL	4.35	SELECT VALUE FROM SYS.V_$PARAM...
cz8wbmy7k5bxn		3	4.35	CPU + Wait for CPU	4.35	** Row Source Not Available **	4.35	begin sys.dbms_aq_inv.internal...
69wffm3uch21b	426073554	2	2.90	CPU + Wait for CPU	2.90	FIXED TABLE - FULL	1.45	WITH events AS (SELECT instan...
c1kyf72yncjpw	3329590876	2	2.90	CPU + Wait for CPU	2.90	FIXED TABLE - FULL	2.90	WITH ash_rows AS (SELECT inst...

Back to Active Session History(ASH) Report
Back to Top

Top SQL with Top Row Sources

- Top SQL statements by DB Time along with the top row sources by DB Time for those SQLs.
- % Activity is the percentage of DB Time due to the SQL.
- % Row Source is the percentage of DB Time spent on the row source by that SQL.
- % Event is the percentage of DB Time spent on the event by the SQL executing the row source.
- Executions is the number of executions of the SQL that were sampled in ASH.

SQL ID	Plan Hash	Executions	% Activity	Row Source	% Row Source	Top Event	% Event	SQL Text
bgxtkrz2p3k08	4250213283	16	23.19	FIXED TABLE - FULL	20.29	CPU + Wait for CPU	20.29	SELECT VALUE FROM SYS.V_$PARAM...
				HASH JOIN	2.90	CPU + Wait for CPU	2.90	
gw788nxk7vbwg	4250213283	4	5.80	FIXED TABLE - FULL	4.35	CPU + Wait for CPU	4.35	SELECT VALUE FROM SYS.V_$PARAM...
				HASH JOIN	1.45	CPU + Wait for CPU	1.45	
cz8wbmy7k5bxn		3	4.35	** Row Source Not Available **	4.35	CPU + Wait for CPU	4.35	begin sys.dbms_aq_inv.internal...
69wffm3uch21b	426073554	2	2.90	FIXED TABLE - FULL	1.45	CPU + Wait for CPU	1.45	WITH events AS (SELECT instan...
				FIXED TABLE - FIXED INDEX	1.45	CPU + Wait for CPU	1.45	
c1kyf72yncjpw	3329590876	2	2.90	FIXED TABLE - FULL	2.90	CPU + Wait for CPU	2.90	WITH ash_rows AS (SELECT inst...

	Total								Per Execution							Percentage of Total							
SQL Id	Cluster (s)	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Rows	Execs	Cluster (s)	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Rows	Cluster	DB time	DB CPU	Gets	IO Wait	Reads	Execs	SQL Text
05s9358mm6vrr	0.59	39.67	35.70	0.94	1,192,000	3,572	1	1	0.59	39.67	35.70	0.94	1,192,000.00	3,572.00	1.00	13.06	63.47	59.64	63.16	28.65	85.60	0.00	begin dbms_feature_usage_inter...
fy4t9uzppnqbw	0.39	1.09	0.82	0.01	102,068	0	1	1	0.39	1.09	0.82	0.01	102,068.00	0.00	1.00	8.57	1.74	1.37	0.40	2.45	0.00	0.00	SELECT TO_CHAR(current_timesta...
3mkdy3ns38nxf	0.30	1.45	1.18	0.02	46,650	42	6,018	1	0.30	1.45	1.18	0.02	46,650.00	42.00	6,018.00	6.58	2.32	1.97	1.58	1.12	1.01	0.00	delete /* QOSH:PURGE_OLD_STS *...
1j6tnz8fcm4c3	0.23	6.57	6.17	0.17	377,005	312	40,240	5	0.05	1.31	1.23	0.03	75,401.00	62.40	8,048.00	5.00	10.51	10.30	11.45	9.06	7.48	0.00	delete /* QOSH:PURGE_OLD_STS *...
f7xnvp44nukp7	0.08	0.18	0.10	0.00	1,674	1	253	253	0.00	0.00	0.00	0.00	6.62	0.00	1.00	1.82	0.29	0.17	0.04	0.04	0.02	0.20	insert into wrm$_snapshot_deta...
43w0r9122v7jm	0.06	1.02	0.81	0.10	24,203	207	1	1	0.06	1.02	0.81	0.10	24,203.00	207.00	1.00	1.27	1.63	1.35	7.02	0.58	4.96	0.00	select max(bytes) from dba_seg...
03guhbfpak0w7	0.04	0.15	0.09	0.08	1,721	158	430	274	0.00	0.00	0.00	0.00	6.28	0.58	1.57	0.89	0.25	0.15	5.55	0.04	3.79	0.21	select /*+ index(idl_ub1$ i_id...
c179sut1vgpc8	0.04	2.60	2.55	0.00	150	0	69	24	0.00	0.11	0.11	0.00	6.25	0.00	2.88	0.81	4.16	4.26	0.00	0.00	0.00	0.02	INSERT /*+ LEADING(@"SEL$F5BB7...

	Total								Per Execution							Percentage of Total							
SQL Id	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Rows	Cluster (s)	Execs	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Rows	Cluster (s)	DB time	DB CPU	IO Wait	Gets	Reads	Cluster	Execs	SQL Text
0gwc9vhhygvbb	208.13	203.51	0.00	155,894	0	3,389	0.00	3,389	0.06	0.06	0.00	46.00	0.00	1.00	0.00	332.99	339.98	0.00	3.75	0.00	0.00	2.65	DECLARE po dbms_aqadm.aq$_purg...
3kqrku32p6sfn	0.00	5.63	5.53	38	0	0	0.00	4	0.00	1.41	1.38	9.50	0.00	0.00	0.00	0.00	9.00	9.23	0.00	0.00	0.00	0.00	MERGE /*+ OPT_PARAM('_pa

Some of the SQL Texts of my contact's db:

c179sut1vgpc8 INSERT /*+ LEADING(@"SEL$F5BB74E1" "H"@"SEL$2" "A"@"SEL$1") USE_NL(@"SEL$F5BB74E1" "A"@"SEL$1") */ INTO WRH$_ACTIVE_SESSION_HISTORY (snap_id, dbid, instance_number , sample_id, sample_time, sample_time_utc, usecs_per_row , session_id, session_serial#, session_type , flags , user_id , sql_id, sql_child_number, sql_opcode, force_matching_signature , top_level_sql_id, top_level_sql_opcode , sql_plan_hash_value, sql_full_plan_hash_value , sql_adaptive_plan_resolved , sql_plan_line_id , sql_plan_operation#, sql_plan_options# , sql_exec_id, sql_exec_start , plsql_entry_object_id, plsql_entry_subprogram_id , plsql_object_id, plsql_subprogram_id , qc_instance_id, qc_session_id, qc_session_serial#, px_flags , event_id, seq#, p1, p2 , p3 , wait_time, time_waited , blocking_session, blocking_session_serial#, blocking_inst_id , current_obj#, current_file#, current_block#, current_row# , top_level_call#, consumer_group_id, xid, remote_instance#, time_model , service_hash, program, module, action, client_id, machine, port, ecid , dbreplay_file_id, dbreplay_call_counter , con_dbid, per_pdb , tm_delta_time, tm_delta_cpu_time, tm_delta_db_time, delta_time, delta_read_io_requests, delta_write_io_requests, delta_read_io_bytes, delta_write_io_bytes, delta_interconnect_io_bytes, pga_allocated, temp_space_allocated , dbop_name, dbop_exec_id) (SELECT /*+ PARAM('_module_action_old_length', 0) */:snap_id, :dbid, :instance_number , a.sample_id, a.sample_time, a.sample_time_utc , a.usecs_per_row * :disk_filter_ratio , a.session_id, a.session_serial#, a.session_type , decode(a.flags, 0, to_number(NULL), a.flags) , a.user_id , a.sql_id, a.sql_child_number, a.sql_opcode, a .force_matching_signature , a.top_level_sql_id, a.top_level_sql_opcode , a.sql_plan_hash_value, a.sql_full_plan_hash_value , a.sql_adaptive_plan_resolved , a.sql_plan_line_id , a.sql_plan_operation#, a.sql_plan_options# , a.sql_exec_id, a.sql_exec_start , a.plsql_entry_object_id, a.plsql_entry_subprogram_id , a.plsql_object_id, a.plsql_subprogram_id , a.qc_instance_id, a.qc_session_id, a.qc_session_serial#, a.px_flags , a.event_id, a.seq#, a.p1, a.p2, a.p3 , a.wait_time, a.time_waited , a.blocking_session, a.blocking_session_serial#, a.blocking_inst_id , a.current_obj#, a.current_file#, a.current_block#, a.current_row# , a.top_level_call#, a.consumer_group_id, a.xid, a.remote_instance# , a.time_model , a.service_hash , substrb(a.program, 1, 64) , a.module, a.action, a.client_id, a.machine, a.port, a.ecid , a.dbreplay_file_id, a.dbreplay_call_counter , a.con_dbid, a.con_id , decode(a.tm_delta_time, 0, to_number(null), a.tm_delt ta_time), decode(a.tm_delta_time, 0, to_number(null), a.tm_delta_cpu_time), decode(a.tm_delta_time, 0, to_number(null), a.tm_delta_db_time), decode(a.delta_time, 0, to_number(null), a.delta_time), decode(a.delta_time, 0, to_number(null), decode(a.delta_read_io_requests, 0, to_number(null), a.delta_read_io_requests)), decode(a.delta_time, 0, to_number(null), decode(a.delta_write_io_requests, 0, to_number(null), a.delta_write_io_requests)), decode(a.delta_time, 0, to_number(null), decode(a.delta_read_io_bytes, 0, to_number(null), a.delta_read_io_bytes)), decode(a.delta_time, 0, to_number(null), decode(a.delta_write_io_bytes, 0, to_number(null), a.delta_write_io_bytes)), decode(a.delta_time, 0, to_number(null), decode(a.delta_interconnect_io_bytes, 0, to_number(null), a.delta_interconnect_io_bytes)), decode(a. pga_allocated, 0, to_number(null), a.pga_allocated), decode(a.pga_allocated, 0, to_number(null), decode(a.temp_space_allocated, 0, to_number(null), a.temp_space_allocated)) , a.dbop_name, a.dbop_exec_id FROM x$ash a, (SELECT h.sample_addr, h.sample_id FROM x$kewash h WHERE ((h.sample_id >= :begin_flushing) and (h.sample_id < :latest_sample_id)) and (nlssort(h.is_awr_sample, 'nls_sort=BINARY') = nlssort('Y', 'nls_sort=BINARY'))) shdr WHERE (1 = 1) and shdr.sample_addr = a.sample_addr and shdr.sample_id = a.sample_id and nlssort(a.need_awr_sample, 'nls_sort=BINARY') = nlssort('Y', 'nls_sort=BINARY'))

f7xnvp44nukp7 insert into wrm$_snapshot_details (snap_id, dbid, instance_number, table_id, begin_time, end_time, step_id) values (:snap_id, :dbid, :instance_number, :bind1, :bind2, :bind3, :step_id)

3mkdy3ns38nxf delete /* QOSH:PURGE_OLD_STS *//*+ dynamic_sampling(4) */ from sys.wri$_optstat_histhead_history where savtime < :1 and savtime > timestamp '1900-01-01 00:00:00 -0:0' and rownum <= :2

1j6tnz8fcm4c3 delete /* QOSH:PURGE_OLD_STS *//*+ dynamic_sampling(4) */ from sys.wri$_optstat_histgrm_history where savtime < :1 and savtime > timestamp '1900-01-01 00:00:00 -0:0' and rownum <= :2

3kqrku32p6sfn	MERGE /*+ OPT_PARAM('_parallel_syspls_obey_force' 'false') */ INTO OPTSTAT_USER_PREFS$ D USING (SELECT * FROM (SELECT O.OBJ#, SYSTIMESTAMP CHGTIME, ROUND(MAX(S.DELTA_READ_IO_BYTES/ S.DELTA_TIME), 3) SCANRATE FROM GV$ACTIVE_SESSION_HISTORY S, GV$SQL_PLAN P, OBJ$ O, USER$ U WHERE S.INST_ID = P.INST_ID AND S.SQL_ID = P.SQL_ID AND S.SQL_PLAN_HASH_VALUE = P.PLAN_HASH_VALUE AND S.SQL_PLAN_LINE_ID = P.ID AND P.OBJECT_NAME = O.NAME AND (:B4 IS NULL OR O.OBJ# = :B4) AND O.OWNER# = U.USER# AND P.OBJECT_OWNER = U.NAME AND O.TYPE# = 2 AND S.CON_ID = SYS_CONTEXT('userenv', 'con_id') AND P.CON_ID = SYS_CONTEXT('userenv', 'con_id') AND S.SQL_PLAN_OPERATION = 'TABLE ACCESS' AND P.OPERATION = 'TABLE ACCESS' AND S.DELTA_TIME > 0 AND S.DELTA_READ_IO_BYTES > 0 AND FROM_TZ(S.SAMPLE_TIME, :B3) > :B2 AND (:B1 = 'ON' OR (:B1 = 'HADOOP_ONLY' AND EXISTS (SELEC T NULL FROM SYS.EXTERNAL_TAB$ ET WHERE ET.OBJ# = O.OBJ# AND TYPE$ IN ('ORACLE_HIVE', 'ORACLE_HDFS', 'ORACLE_BIGDATA')))) GROUP BY O.OBJ#) C WHERE NOT EXISTS (SELECT NULL FROM OPTSTAT_USER_PREFS$ PR WHERE PNAME = 'SCAN_RATE' AND C.OBJ# = PR.OBJ# AND (PR.SPARE1 IS NULL OR C.SCANRATE <= TO_NUMBER(PR.VALCHAR))) SRC ON (D.OBJ#=SRC.OBJ# AND D.PNAME = 'SCAN_RATE') WHEN MATCHED THEN UPDATE SET D.VALCHAR= TO_CHAR(SRC.SCANRATE), D.CHGTIME=SRC.CHGTIME, D.SPARE1=1 WHEN NOT MATCHED THEN INSERT (OBJ#, PNAME, VALCHAR, CHGTIME, SPARE1) VALUES(SRC.OBJ#, 'SCAN_RATE', TO_CHAR(SRC.SCANRATE), SRC.CHGTIME, 1)

Top Timed Background Events

- % of bg time: % of background elapsed time
- Instance '*' - cluster wide summary
- '*' Waits, %Timeouts, Wait Time Total(s) : Cluster-wide total for the wait event
- '*' 'Wait Time Avg' : Cluster-wide average computed as (Wait Time Total / Event Waits)
- '*' Summary 'Avg Wait Time' : Per-instance 'Wait Time Avg' used to compute the following statistics
- '*' [Avg/Min/Max/Std Dev] : average/minimum/maximum/standard deviation of per-instance 'Wait Time
- '*' Cnt : count of instances with wait times for the event

	Wait		Event		Wait Time		
I#	Class	Event	Waits	%Timeouts	Total(s)	Avg Wait	%DB time
*		background cpu time			770.32		121.87
	Other	latch free	10,172	0.00	15.71	1.54ms	2.48
	Other	oracle thread bootstrap	249	0.00	13.64	54.78ms	2.16
	System I/O	control file sequential read	29,532	0.00	11.76	398.23us	1.86
	System I/O	log file parallel write	9,135	0.00	7.26	794.63us	1.15
	Other	LGWR all worker groups	7,229	0.00	7.23	1.00ms	1.14
	Other	IMR slave acknowledgement msg	21,606	0.00	6.21	287.57us	0.98
	Other	PX Deq: reap credit	254,182	100.00	3.31	13.03us	0.52
	System I/O	control file parallel write	2,758	0.00	2.23	806.90us	0.35
	System I/O	db file parallel write	4,790	0.00	2.15	448.43us	0.34
1		background cpu time			512.67		113.49
	Other	latch free	5,111	0.00	8.01	1.57ms	1.77
	Other	oracle thread bootstrap	127	0.00	7.14	56.18ms	1.58
	System I/O	control file sequential read	15,416	0.00	6.24	404.55us	1.38
	System I/O	log file parallel write	4,715	0.00	3.74	793.27us	0.83
	Other	LGWR all worker groups	3,592	0.00	3.63	1.01ms	0.80
	Other	IMR slave acknowledgement msg	10,803	0.00	3.17	293.71us	0.70
	Other	PX Deq: reap credit	129,529	100.00	1.67	12.87us	0.37
	Cluster	gc cr block 2-way	3,824	0.00	1.56	408.69us	0.35
	System I/O	db file parallel write	3,498	0.00	1.52	433.69us	0.34

Block Size
8192

Appendix 3: US revenue, profit, and pricing model

The pricing options of my service are to make the individual pay per user project, or pay per usage cycle, or to make the customers pay a monthly fee/yearly/weekly fee (based on activity or not), or indexing that fee on the number of requests or on the time spent on the user project or spent on the usage cycle, or indexing that fee on the number of tasks ran by the server for a user project.

First, due to the nature of the service, one customer can join and leave the pool of active users at any moment, and there is in fact no certitude that a given user will complete a user project at all times all year/month/week-round at the same frequency. Secondly, the usage cycle contains tasks aside of the user project (creating an account, designating billing methods, exporting results files, etc.). These side tasks would be priced per request, but that would likely deter users from accessing the website and considering creating user projects. Thirdly billing per user project makes each specific usage, which might vary in time and computing effort, be valued at a fixed priced. This means either using a middle-effort price, and top price, or a lowest-effort. Lowest effort price is simply unfair for myself. The app would often take way more effort than the lowest possible effort, and it would mean it usually work at a loss. Middle and highest-effort prices make no sense either, because users completing less complex project than the most complex imaginable (which is usually the case) would be unhappy overpaying. Also, opting for a per-HTTP request pricing is nonsense, because the complexity/AI redability and exhaustivity of the user-input documents directly influences the number of HTTP requests in a user project, therefore penalising users who cannot/do not have the time to control the 'quality' of their documents from the software's perspective.

The customer pays around 200$ for someone or themselves to complete a user project (secretary-like salary plus side company charges). Thus, they would be really reluctant of paying even $100 per user project, because that represents $300 of costs for them on average per usage project, which is often above what a low level equivalent human service provider would ask for a day's work (minimum $250). Therefore, I choose a pricing of $10. This way, customer charges for one usage cycle remain a bit under $250 at $210. This will also particularly attract customers working on complex projects, which would require thousands of dollars of financial consulting fees and many days or weeks of service delivery. The program has mechanisms to ensure that the delivery of the expertise happens only once per project (you need to validate the last step of the project to receive the expertise, and then you cannot go back in the project to modify it). **Therefore, I choose pricing of 10 US dollars per user project, paid at project creation.**

Knowing 1 usage cycle = 1 user project, and that in my forecast, the average number of new usage cycles a day per million inhabitants is 76.48, the profit before expenses is 76.48 ×350×10 dollars×21 workdays per month (because customers activity is calculated as if only on these days) = $5,621,280 per month in the US ($67,455,360 a year).